

April 17, 2025

1 Campaign for selling personal loans.

This case is about a bank (Thera Bank) which has a growing customer base. Majority of these customers are liability customers (depositors) with varying size of deposits. The number of customers who are also borrowers (asset customers) is quite small, and the bank is interested in expanding this base rapidly to bring in more loan business and in the process, earn more through the interest on loans. In particular, the management wants to explore ways of converting its liability customers to personal loan customers (while retaining them as depositors). A campaign that the bank ran last year for liability customers showed a healthy conversion rate of over 9% success. This has encouraged the retail marketing department to devise campaigns with better target marketing to increase the success ratio with minimal budget.

The department wants to build a model that will help them identify the potential customers who have higher probability of purchasing the loan. This will increase the success ratio while at the same time reduce the cost of the campaign.

The file Bank.xls contains data on 5000 customers. The data include customer demographic information (age, income, etc.), the customer's relationship with the bank (mortgage, securities account, etc.), and the customer response to the last personal loan campaign (Personal Loan). Among these 5000 customers, only 480 (= 9.6%) accepted the personal loan that was offered to them in the earlier campaign.

1.1 DataSet Description:

ID - Customer ID

Age - Customer's age in completed years

Experience - Number years of professional experience

Income - Annual income of the customer (US Dollars)

ZIPCode - Home Address ZIP code.

Family - Family size of the customer

CCAvg - Avg. spending on credit cards per month (US Dollars)

Education - Education Level. 1: Undergrad; 2: Graduate; 3: Advanced/Professional

Mortgage - Value of house mortgage if any. (US Dollars)

Personal Loan - Did this customer accept the personal loan offered in the last campaign?

Securities Account - Does the customer have a securities account with the bank?

CD Account - Does the customer have a certificate of deposit (CD) account with the bank?

Online - Does the customer use internet banking facilities?

CreditCard - Does the customer use a credit card issued by UniversalBank?

1.2 Task 1: Data Loading (5 Marks)

Perform the following tasks

1. Import the necessary libraries required
2. Read the csv into a dataframe
3. Display the first five rows of your dataframe.
4. Display the datatypes of the columns
5. Display the column names in the dataframe

1.2.1 1.1 Importing Libraries

```
[110]: # Import the necessary libraries required
import os

# data manipulation
import pandas as pd
import numpy as np

# deep learning tools
import tensorflow as tf
import keras_tuner as kt

# ML algorithms
from sklearn.linear_model import LogisticRegression, RidgeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier, RadiusNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier,
↳ GradientBoostingClassifier, BaggingClassifier
from catboost import CatBoostClassifier
from lightgbm import LGBMClassifier
from xgboost import XGBClassifier, XGBRFClassifier
```

```

from sklearn.tree import DecisionTreeClassifier

# feature engineering tools
from scipy.stats import pearsonr
from sklearn.feature_selection import RFECV
from imblearn.over_sampling import SMOTE
from sklearn.compose import ColumnTransformer

# algorithm metrics
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.metrics import classification_report, confusion_matrix, \
    accuracy_score, roc_curve, roc_auc_score, f1_score
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
import joblib

# visualization tools
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

```

1.2.2 1.2 Read dataframe

```

[3]: # Read the csv into a dataframe
personal_loan_df = pd.read_csv('../data/Bank_Personal_Loan_Modelling-1.csv')

pd.set_option('display.max_columns', None)

```

1.2.3 1.3 Display 1st 5 rows

```

[4]: # Display the first five rows of your dataframe.
personal_loan_df.head(5)

```

```

[4]:   ID  Age  Experience  Income  ZIP Code  Family  CCAvg  Education  Mortgage  \
0    1   25           1     49    91107      4    1.6           1           0
1    2   45          19     34    90089      3    1.5           1           0
2    3   39          15     11    94720      1    1.0           1           0
3    4   35           9    100    94112      1    2.7           2           0
4    5   35           8     45    91330      4    1.0           2           0

      Personal Loan  Securities Account  CD Account  Online  CreditCard
0                0                1            0        0         0
1                0                1            0        0         0
2                0                0            0        0         0
3                0                0            0        0         0

```

4 0 0 0 0 1

1.2.4 1.4 Dtypes of the columns

```
[5]: # Display the datatypes of the columns
personal_loan_df.dtypes
```

```
[5]: ID                int64
Age                int64
Experience         int64
Income            int64
ZIP Code          int64
Family            int64
CCAvg            float64
Education         int64
Mortgage          int64
Personal Loan     int64
Securities Account int64
CD Account        int64
Online            int64
CreditCard        int64
dtype: object
```

1.2.5 1.5 Column Names of the df

```
[6]: # Display the column names in the dataframe
print(list(personal_loan_df.columns))
```

```
['ID', 'Age', 'Experience', 'Income', 'ZIP Code', 'Family', 'CCAvg',
'Education', 'Mortgage', 'Personal Loan', 'Securities Account', 'CD Account',
'Online', 'CreditCard']
```

1.3 Task 2: Data Cleaning (10 Marks)

1. Drop the unnecessary columns in the dataframe(i.e. 'ID'). (2Marks)
2. Check for missing values in the dataframe and display the count in ascending order. Impute the missing values. (2Marks)
3. Check whether there are any duplicate values in the dataframe and if present remove them. (2Marks)
4. Display the total number of records in the dataframe after removing the duplicates. (2Marks)
5. Create a 5 Point summary of the data. (2Marks)

1.3.1 2.1 Unnecessary column drops

```
[7]: # Drop the unnecessary columns in the dataframe(i.e. 'ID')
personal_loan_df.drop(columns={"ID"},axis=1, inplace=True)
```

1.3.2 2.2 Handling missing Values

```
[8]: # a function for handling missing values
def missing_Val(df):
    # if df is a dataframe
    if isinstance(df, pd.DataFrame):
        missing_count = df.isna().sum().sum()
    # if df is a series
    elif isinstance(df, pd.Series):
        missing_count = df.isna().sum()
    # if df is neither dataframe or a series
    else:
        print("err! : not a valid data type")
        exit()

    if missing_count == 0:
        state = False
        print(f"{state}, no missing values present")
    else:
        state = True
        print(f"{state}, {missing_count} - Missing Values")
```

```
[9]: missing_Val(personal_loan_df)
```

False, no missing values present

```
[10]: # Check for missing values in the dataframe and display the count in ascending
      ↪ order. Impute the missing values
missing_Val(personal_loan_df)
```

False, no missing values present

1.3.3 2.3 Check for duplicates

```
[11]: personal_loan_df.shape
```

```
[11]: (5000, 13)
```

```
[12]: # Check whether there are any duplicate values in the dataframe and if present
      ↪ remove them
print(personal_loan_df.duplicated().sum())
```

0

1.3.4 2.4 Total No.Of records in df

```
[13]: # Display the total number of records in the dataframe after removing the
      ↪duplicates.
      personal_loan_df.drop_duplicates(inplace=True)
      print(personal_loan_df.duplicated().sum())
```

0

```
[14]: personal_loan_df.shape
```

```
[14]: (5000, 13)
```

- Since, there were no duplicate found , it's still 5000 records

1.3.5 2.5 Five Point Summary of the Data

```
[15]: personal_loan_df.describe()
```

```
[15]:
```

	Age	Experience	Income	ZIP Code	Family \
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000
mean	45.338400	20.104600	73.774200	93152.503000	2.396400
std	11.463166	11.467954	46.033729	2121.852197	1.147663
min	23.000000	-3.000000	8.000000	9307.000000	1.000000
25%	35.000000	10.000000	39.000000	91911.000000	1.000000
50%	45.000000	20.000000	64.000000	93437.000000	2.000000
75%	55.000000	30.000000	98.000000	94608.000000	3.000000
max	67.000000	43.000000	224.000000	96651.000000	4.000000

	CCAvg	Education	Mortgage	Personal Loan \
count	5000.000000	5000.000000	5000.000000	5000.000000
mean	1.937938	1.881000	56.498800	0.096000
std	1.747659	0.839869	101.713802	0.294621
min	0.000000	1.000000	0.000000	0.000000
25%	0.700000	1.000000	0.000000	0.000000
50%	1.500000	2.000000	0.000000	0.000000
75%	2.500000	3.000000	101.000000	0.000000
max	10.000000	3.000000	635.000000	1.000000

	Securities Account	CD Account	Online	CreditCard
count	5000.000000	5000.000000	5000.000000	5000.000000
mean	0.104400	0.06040	0.596800	0.294000
std	0.305809	0.23825	0.490589	0.455637
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	1.000000	0.000000
75%	0.000000	0.000000	1.000000	1.000000
max	1.000000	1.000000	1.000000	1.000000

```
[16]: personal_loan_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 5000 entries, 0 to 4999
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                    5000 non-null   int64
1   Experience              5000 non-null   int64
2   Income                 5000 non-null   int64
3   ZIP Code               5000 non-null   int64
4   Family                 5000 non-null   int64
5   CCAvg                  5000 non-null   float64
6   Education              5000 non-null   int64
7   Mortgage               5000 non-null   int64
8   Personal Loan          5000 non-null   int64
9   Securities Account     5000 non-null   int64
10  CD Account             5000 non-null   int64
11  Online                 5000 non-null   int64
12  CreditCard             5000 non-null   int64
dtypes: float64(1), int64(12)
memory usage: 546.9 KB
```

```
[17]: df_col_ls = personal_loan_df.columns.tolist()
df_col_ls

for i in df_col_ls:
    print(f"{i}, Unique Values: \n {personal_loan_df[i].unique()}")
```

Age, Unique Values:

```
[25 45 39 35 37 53 50 34 65 29 48 59 67 60 38 42 46 55 56 57 44 36 43 40
 30 31 51 32 61 41 28 49 47 62 58 54 33 27 66 24 52 26 64 63 23]
```

Experience, Unique Values:

```
[ 1 19 15  9  8 13 27 24 10 39  5 23 32 41 30 14 18 21 28 31 11 16 20 35
  6 25  7 12 26 37 17  2 36 29  3 22 -1 34  0 38 40 33  4 -2 42 -3 43]
```

Income, Unique Values:

```
[ 49  34  11 100  45  29  72  22  81 180 105 114  40 112 130 193  21  25
  63  62  43 152  83 158  48 119  35  41  18  50 121  71 141  80  84  60
132 104  52 194  8 131 190  44 139  93 188  39 125  32  20 115  69  85
135  12 133  19  82 109  42  78  51 113 118  64 161  94  15  74  30  38
  9  92  61  73  70 149  98 128  31  58  54 124 163  24  79 134  23  13
138 171 168  65  10 148 159 169 144 165  59  68  91 172  55 155  53  89
 28  75 170 120  99 111  33 129 122 150 195 110 101 191 140 153 173 174
 90 179 145 200 183 182  88 160 205 164  14 175 103 108 185 204 154 102
192 202 162 142  95 184 181 143 123 178 198 201 203 189 151 199 224 218]
```

ZIP Code, Unique Values:

```
[91107 90089 94720 94112 91330 92121 91711 93943 93023 94710 90277 93106
 94920 91741 95054 95010 94305 91604 94015 90095 91320 95521 95064 90064]
```

94539 94104 94117 94801 94035 92647 95814 94114 94115 92672 94122 90019
 95616 94065 95014 91380 95747 92373 92093 94005 90245 95819 94022 90404
 93407 94523 90024 91360 95670 95123 90045 91335 93907 92007 94606 94611
 94901 92220 93305 95134 94612 92507 91730 94501 94303 94105 94550 92612
 95617 92374 94080 94608 93555 93311 94704 92717 92037 95136 94542 94143
 91775 92703 92354 92024 92831 92833 94304 90057 92130 91301 92096 92646
 92182 92131 93720 90840 95035 93010 94928 95831 91770 90007 94102 91423
 93955 94107 92834 93117 94551 94596 94025 94545 95053 90036 91125 95120
 94706 95827 90503 90250 95817 95503 93111 94132 95818 91942 90401 93524
 95133 92173 94043 92521 92122 93118 92697 94577 91345 94123 92152 91355
 94609 94306 96150 94110 94707 91326 90291 92807 95051 94085 92677 92614
 92626 94583 92103 92691 92407 90504 94002 95039 94063 94923 95023 90058
 92126 94118 90029 92806 94806 92110 94536 90623 92069 92843 92120 95605
 90740 91207 95929 93437 90630 90034 90266 95630 93657 92038 91304 92606
 92192 90745 95060 94301 92692 92101 94610 90254 94590 92028 92054 92029
 93105 91941 92346 94402 94618 94904 9307 95482 91709 91311 94509 92866
 91745 94111 94309 90073 92333 90505 94998 94086 94709 95825 90509 93108
 94588 91706 92109 92068 95841 92123 91342 90232 92634 91006 91768 90028
 92008 95112 92154 92115 92177 90640 94607 92780 90009 92518 91007 93014
 94024 90027 95207 90717 94534 94010 91614 94234 90210 95020 92870 92124
 90049 94521 95678 95045 92653 92821 90025 92835 91910 94701 91129 90071
 96651 94960 91902 90033 95621 90037 90005 93940 91109 93009 93561 95126
 94109 93107 94591 92251 92648 92709 91754 92009 96064 91103 91030 90066
 95403 91016 95348 91950 95822 94538 92056 93063 91040 92661 94061 95758
 96091 94066 94939 95138 95762 92064 94708 92106 92116 91302 90048 90405
 92325 91116 92868 90638 90747 93611 95833 91605 92675 90650 95820 90018
 93711 95973 92886 95812 91203 91105 95008 90016 90035 92129 90720 94949
 90041 95003 95192 91101 94126 90230 93101 91365 91367 91763 92660 92104
 91361 90011 90032 95354 94546 92673 95741 95351 92399 90274 94087 90044
 94131 94124 95032 90212 93109 94019 95828 90086 94555 93033 93022 91343
 91911 94803 94553 95211 90304 92084 90601 92704 92350 94705 93401 90502
 94571 95070 92735 95037 95135 94028 96003 91024 90065 95405 95370 93727
 92867 95821 94566 95125 94526 94604 96008 93065 96001 95006 90639 92630
 95307 91801 94302 91710 93950 90059 94108 94558 93933 92161 94507 94575
 95449 93403 93460 95005 93302 94040 91401 95816 92624 95131 94965 91784
 91765 90280 95422 95518 95193 92694 90275 90272 91791 92705 91773 93003
 90755 96145 94703 96094 95842 94116 90068 94970 90813 94404 94598]

Family, Unique Values:

[4 3 1 2]

CCAvg, Unique Values:

[1.6	1.5	1.	2.7	0.4	0.3	0.6	8.9	2.4	0.1	3.8	2.5
2.	4.7	8.1	0.5	0.9	1.2	0.7	3.9	0.2	2.2	3.3	1.8
2.9	1.4	5.	2.3	1.1	5.7	4.5	2.1	8.	1.7	0.	2.8
3.5	4.	2.6	1.3	5.6	5.2	3.	4.6	3.6	7.2	1.75	7.4
2.67	7.5	6.5	7.8	7.9	4.1	1.9	4.3	6.8	5.1	3.1	0.8
3.7	6.2	0.75	2.33	4.9	0.67	3.2	5.5	6.9	4.33	7.3	4.2
4.4	6.1	6.33	6.6	5.3	3.4	7.	6.3	8.3	6.	1.67	8.6
7.6	6.4	10.	5.9	5.4	8.8	1.33	9.	6.7	4.25	6.67	5.8


```

    4.8    3.25  5.67  8.5    4.75  4.67  3.67  8.2    3.33  5.33  9.3    2.75]
Education, Unique Values:
[1 2 3]
Mortgage, Unique Values:
[  0 155 104 134 111 260 163 159  97 122 193 198 285 412 153 211 207 240
455 112 336 132 118 174 126 236 166 136 309 103 366 101 251 276 161 149
188 116 135 244 164  81 315 140  95  89  90 105 100 282 209 249  91  98
145 150 169 280  99  78 264 113 117 325 121 138  77 158 109 131 391  88
129 196 617 123 167 190 248  82 402 360 392 185 419 270 148 466 175 147
220 133 182 290 125 124 224 141 119 139 115 458 172 156 547 470 304 221
108 179 271 378 176  76 314  87 203 180 230 137 152 485 300 272 144  94
208 275  83 218 327 322 205 227 239  85 160 364 449  75 107  92 187 355
106 587 214 307 263 310 127 252 170 265 177 305 372  79 301 232 289 212
250  84 130 303 256 259 204 524 157 231 287 247 333 229 357 361 294  86
329 142 184 442 233 215 394 475 197 228 297 128 241 437 178 428 162 234
257 219 337 382 397 181 120 380 200 433 222 483 154 171 146 110 201 277
268 237 102  93 354 195 194 238 226 318 342 266 114 245 341 421 359 565
319 151 267 601 567 352 284 199  80 334 389 186 246 589 242 143 323 535
293 398 343 255 311 446 223 262 422 192 217 168 299 505 400 165 183 326
298 569 374 216 191 408 406 452 432 312 477 396 582 358 213 467 331 295
235 635 385 328 522 496 415 461 344 206 368 321 296 373 292 383 427 189
202  96 429 431 286 508 210 416 553 403 225 500 313 410 273 381 330 345
253 258 351 353 308 278 464 509 243 173 481 281 306 577 302 405 571 581
550 283 612 590 541]
Personal Loan, Unique Values:
[0 1]
Securities Account, Unique Values:
[1 0]
CD Account, Unique Values:
[0 1]
Online, Unique Values:
[0 1]
CreditCard, Unique Values:
[0 1]

```

```
[18]: # Create a 5 Point summary of the data
```

5 Point Summary of the Dataset (Personal Loan) * This dataset contains 5000 records * Binary Features => [Personal Loan,Securities Account,CD Account,Online,Credit Card] ('0' or '1'). * Numerical and Non-Continuous Values => [Income,ZIP Code,CCAVG,Mortgage] * Multi-Categorical Values => [Age,Family,Education] * Initially there were no Null or Duplicate Values found.

1.4 Task 3: Data Analysis and Visualization (20Marks)

1. Check for correlation of the attributes in the dataframe and plot the heatmap for the same. (4Marks)
2. Plot a pairplot for the attributes. Use "Personal Loan" as hue. (4Marks)
3. From the above visualization identify the attributes that are correlated and drop the unnec-

essary attribute(column) (4Marks)

4. Plot a boxplot to understand the distribution of data for the attributes such as “Age”, “Income”, Family“, ”CCAvg“, ”Education“, ”Mortgage” by taking x axis as “Personal Loan” (4Marks)
5. Display the unique values of attributes “Family”, “Education” and “Personal loan” and plot a bargraph. (4Marks)

CLEAN → (VISUALIZE) → FEATURE ENG → (TEMPORARY SCALE → CORRELATION / RFECV) → SPLIT DATA → APPLY SMOTE (train only) → SCALE (fit on train) → FINAL TRAINING → OPTIMIZE → EVALUATE → DEPLOY

1.4.1 3.1 Visualization

3.1.1 Correlation and Heatmap

```
[19]: personal_loan_df.corr()
```

```
[19]:
```

	Age	Experience	Income	ZIP Code	Family \
Age	1.000000	0.994215	-0.055269	-0.029216	-0.046418
Experience	0.994215	1.000000	-0.046574	-0.028626	-0.052563
Income	-0.055269	-0.046574	1.000000	-0.016410	-0.157501
ZIP Code	-0.029216	-0.028626	-0.016410	1.000000	0.011778
Family	-0.046418	-0.052563	-0.157501	0.011778	1.000000
CCAvg	-0.052012	-0.050077	0.645984	-0.004061	-0.109275
Education	0.041334	0.013152	-0.187524	-0.017377	0.064929
Mortgage	-0.012539	-0.010582	0.206806	0.007383	-0.020445
Personal Loan	-0.007726	-0.007413	0.502462	0.000107	0.061367
Securities Account	-0.000436	-0.001232	-0.002616	0.004704	0.019994
CD Account	0.008043	0.010353	0.169738	0.019972	0.014110
Online	0.013702	0.013898	0.014206	0.016990	0.010354
CreditCard	0.007681	0.008967	-0.002385	0.007691	0.011588

	CCAvg	Education	Mortgage	Personal Loan \
Age	-0.052012	0.041334	-0.012539	-0.007726
Experience	-0.050077	0.013152	-0.010582	-0.007413
Income	0.645984	-0.187524	0.206806	0.502462
ZIP Code	-0.004061	-0.017377	0.007383	0.000107
Family	-0.109275	0.064929	-0.020445	0.061367
CCAvg	1.000000	-0.136124	0.109905	0.366889
Education	-0.136124	1.000000	-0.033327	0.136722
Mortgage	0.109905	-0.033327	1.000000	0.142095
Personal Loan	0.366889	0.136722	0.142095	1.000000
Securities Account	0.015086	-0.010812	-0.005411	0.021954
CD Account	0.136534	0.013934	0.089311	0.316355
Online	-0.003611	-0.015004	-0.005995	0.006278
CreditCard	-0.006689	-0.011014	-0.007231	0.002802

	Securities Account	CD Account	Online	CreditCard
Age	-0.000436	0.008043	0.013702	0.007681

Experience	-0.001232	0.010353	0.013898	0.008967
Income	-0.002616	0.169738	0.014206	-0.002385
ZIP Code	0.004704	0.019972	0.016990	0.007691
Family	0.019994	0.014110	0.010354	0.011588
CCAvg	0.015086	0.136534	-0.003611	-0.006689
Education	-0.010812	0.013934	-0.015004	-0.011014
Mortgage	-0.005411	0.089311	-0.005995	-0.007231
Personal Loan	0.021954	0.316355	0.006278	0.002802
Securities Account	1.000000	0.317034	0.012627	-0.015028
CD Account	0.317034	1.000000	0.175880	0.278644
Online	0.012627	0.175880	1.000000	0.004210
CreditCard	-0.015028	0.278644	0.004210	1.000000

[20]: *# Check for correlation of the attributes in the dataframe and plot the heatmap*
↪ for the same.

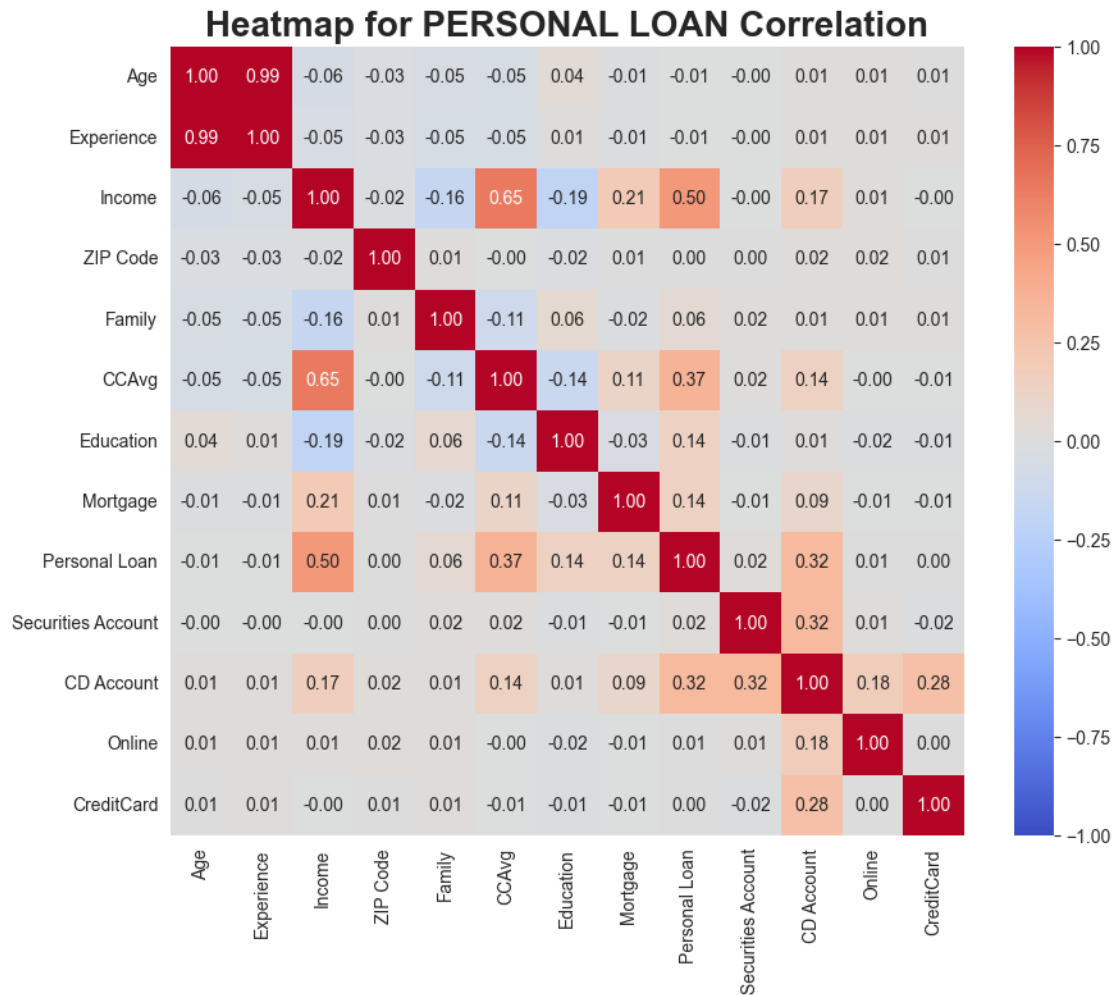
```
plt.figure(figsize = (10,8))

sns.heatmap(personal_loan_df.corr(), annot=True, vmin=-1, vmax=1,
↪ cmap="coolwarm" , fmt=".2f")

plt.title("Heatmap for PERSONAL LOAN Correlation",fontweight="bold",
↪ fontsize=20)

plt.show()

plt.savefig("../visualization/heatmap.png")
```



<Figure size 640x480 with 0 Axes>

3.1.2 Pairplot

```
[21]: # Plot a pairplot for the attributes. Use "Personal Loan" as hue.

plt.figure(figsize=(15,20))

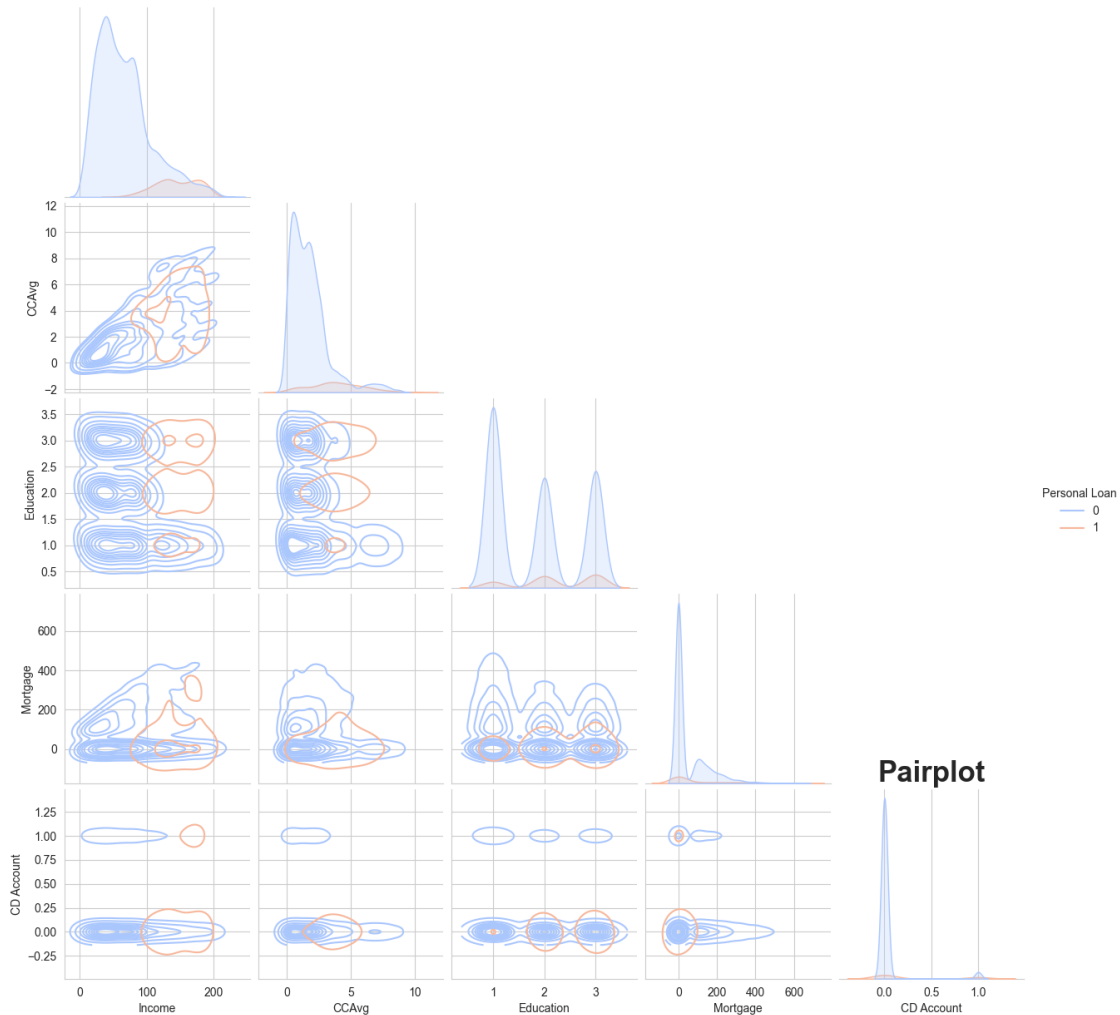
sns.pairplot(
    data = personal_loan_df, hue = "Personal Loan", palette = "coolwarm", kind_
    ↪ = "kde",
    x_vars=["Income","CCAvg","Education","Mortgage","CD Account"],
    y_vars=["Income","CCAvg","Education","Mortgage","CD Account"], corner = True
)

plt.title("Pairplot", fontweight="bold", fontsize=25)
```

```
plt.show()

plt.savefig("../visualization/pairplot.png")
```

<Figure size 1500x2000 with 0 Axes>



<Figure size 640x480 with 0 Axes>

3.1.3 Feature Identification

```
[22]: # From the above visualization identify the attributes that are correlated and
      ↪ drop the unnecessary attribute(column)
new_df = personal_loan_df.drop(columns={"Age", "Experience", "ZIP_
      ↪ Code", "Online", "CreditCard"}, axis=1)
new_df.head(5)
```

```
[22]:
```

	Income	Family	CCAvg	Education	Mortgage	Personal Loan	\
0	49	4	1.6	1	0	0	
1	34	3	1.5	1	0	0	
2	11	1	1.0	1	0	0	
3	100	1	2.7	2	0	0	
4	45	4	1.0	2	0	0	

	Securities Account	CD Account
0	1	0
1	1	0
2	0	0
3	0	0
4	0	0

3.1.4 Boxplot

```
[23]: # Plot a boxplot to understand the distribution of data for the attributes such
      ↪ as "Age", "Income", "Family", "CCAvg", "Education", "Mortgage" by taking x
      ↪ axis as "Personal Loan"

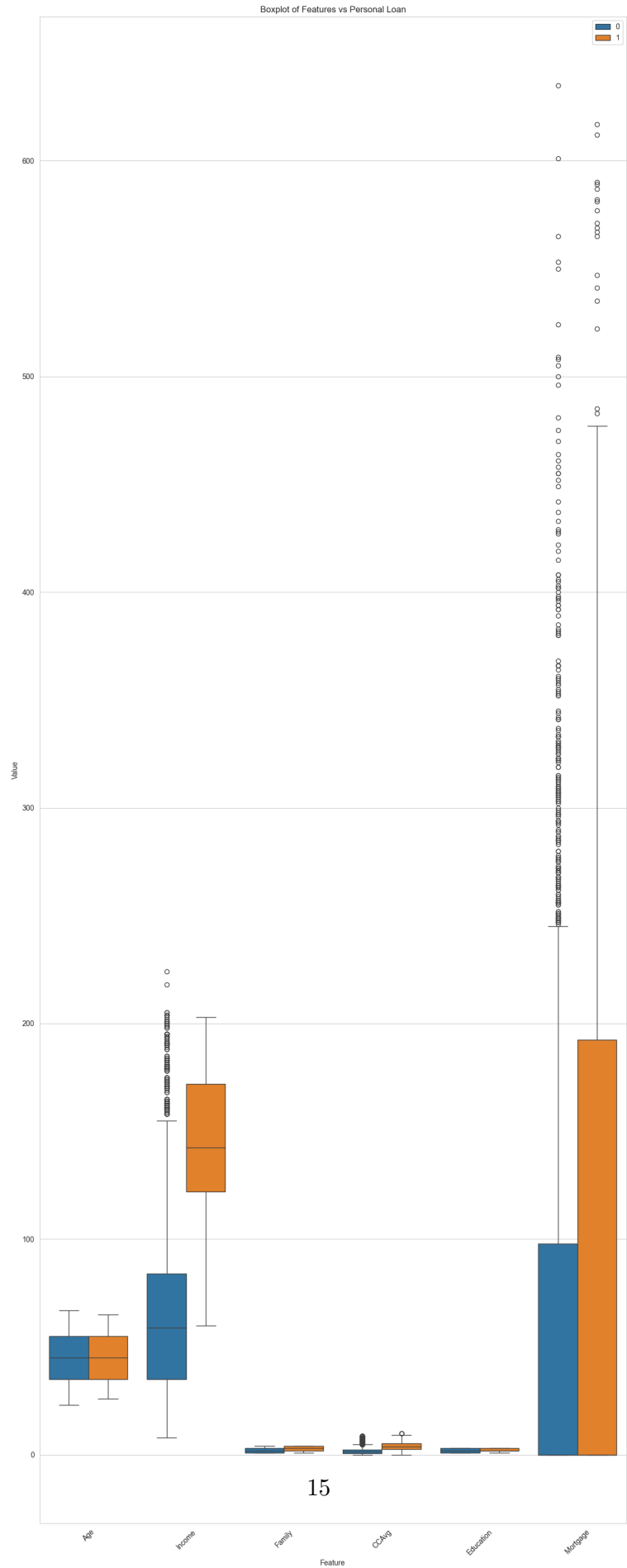
      # Select relevant columns
      cols = ["Age", "Income", "Family", "CCAvg", "Education", "Mortgage", "Personal_
      ↪ Loan"]
      df_melted = personal_loan_df[cols].melt(id_vars="Personal Loan",
      ↪ var_name="Feature", value_name="Value")

      # Create the boxplot
      plt.figure(figsize=(12, 30))

      sns.boxplot(data=df_melted, x="Feature", y="Value", hue="Personal Loan")

      plt.title("Boxplot of Features vs Personal Loan")
      plt.xticks(rotation=45)
      plt.tight_layout()
      plt.legend()
      plt.show()

      plt.savefig("../visualization/boxplots.png")
```



<Figure size 640x480 with 0 Axes>

3.1.5 Bargraph

```
[24]: # Display the unique values of attributes "Family", "Education" and "Personal_
↳ loan" and plot a bargraph.
ls = ["Family","Education","Personal Loan"]
for i in ls:
    print(f"Unique Values of {i},\n{personal_loan_df[i].unique()}")
```

```
Unique Values of Family,
[4 3 1 2]
Unique Values of Education,
[1 2 3]
Unique Values of Personal Loan,
[0 1]
```

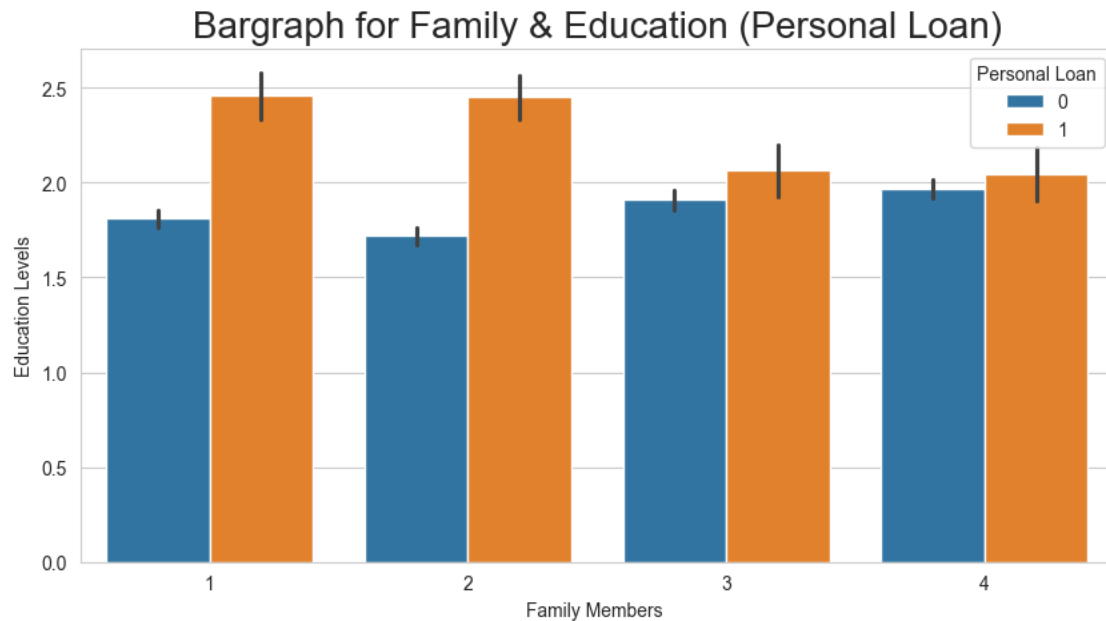
```
[25]: # bargraph plotting

plt.figure(figsize=(10,5))

sns.barplot(data = personal_loan_df, x="Family",y="Education",hue="Personal_
↳ Loan")

plt.title("Bargraph for Family & Education (Personal Loan)",fontsize=20)
plt.xlabel("Family Members ", fontsize=10)
plt.ylabel("Education Levels", fontsize=10)
plt.show()

plt.savefig("../visualization/Family Education.png")
```

<Figure size 640x480 with 0 Axes>

```
[26]: personal_loan_df.to_csv("../data/cleaned_data.csv")
```

1.4.2 3.2 Data Analysis

3.2.1 Pearson Correlation

```
[27]: # Finding the best features which has the best correlation to "Personal Loan"
personal_loan_X = personal_loan_df.drop(columns={"Personal Loan"},axis=1)
personal_loan_Y = personal_loan_df["Personal Loan"]

# initializing the variable dictionary
PL_corr = {}
features = list(personal_loan_X.columns)
# print(features)
# inserting elements into the dict
for i in features:
    corr , pval = pearsonr(personal_loan_X[i], personal_loan_Y)
    p_str = str()
    if pval <= 0.0001:
        p_str="strong"
    elif pval<= 0.05:
        p_str="moderate"
    elif pval<=0.1:
        p_str="weak"
    else:
```

```

    p_str="No Corr"
    PL_corr[i] = {"corr":corr, "pval":pval, "state":p_str}

PL_corr_df = pd.DataFrame.from_dict(PL_corr)

PL_corr_df = PL_corr_df.transpose()

```

```
[28]: PL_corr_df.head(6)
```

```

[28]:
           corr    pval    state
Age      -0.007726  0.584959  No Corr
Experience -0.007413  0.600236  No Corr
Income      0.502462    0.0  strong
ZIP Code    0.000107  0.993944  No Corr
Family      0.061367  0.000014  strong
CCAvg       0.366889    0.0  strong

```

```
[29]: PL_corr_df.tail(6)
```

```

[29]:
           corr    pval    state
Education    0.136722    0.0  strong
Mortgage     0.142095    0.0  strong
Securities Account 0.021954  0.120621  No Corr
CD Account    0.316355    0.0  strong
Online        0.006278  0.657186  No Corr
CreditCard    0.002802  0.843008  No Corr

```

- While using `pearsonr` we were able to see only 6 columns with strong correlation with the Personal Loan
- features Include : [Income, Family, CCAvg, Education, Mortgage, CD Account]

3.2.2 RFECV Feature Extraction RFECV - Recursive Feature Elimination with Cross Validation since RFECV needs a model to perform its duty to find best features we will move with 4 basic models:- * Logistic Regression * KNN (also not supported) * SVC (NOT Supported with RFECV) * Random Forest * Decision Tree * Gradient Boosting

```

[30]: # initializing the Models
models = {
    "logR": LogisticRegression(),
    # "SVC" : SVC(kernel='sigmoid', degree=3),
    "RandomForest": RandomForestClassifier(criterion="entropy"),
    # "KNN" : KNeighborsClassifier(n_neighbors=5, algorithm="brute"),
    "DecisionTree": DecisionTreeClassifier(splitter="best"),
    "GradientBoosting" : GradientBoostingClassifier()
}

```

```

# initiating cross validation
strat_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=3)

# best features
chosen_features_per_model = {}

# looping through every model
for name, model in models.items():
    # initiating rfecv
    rfecv_model = RFECV(estimator=model, cv=strat_cv, scoring='accuracy',
    ↪n_jobs=-1)

    # fitting the data into the model
    rfecv_model.fit(personal_loan_X, personal_loan_Y)

    # choosing the best features
    chosen_features = personal_loan_X.columns[rfecv_model.support_].tolist()
    # feature assignment
    chosen_features_per_model[name] = chosen_features
#
# # converting the feature dictionary to dataframe
# chosen_features_per_model_df = pd.DataFrame.
    ↪from_dict(chosen_features_per_model)

```

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:458: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:458: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

C:\ProgramData\anaconda3\Lib\site-

```
packages\sklearn\linear_model\_logistic.py:458: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
C:\ProgramData\anaconda3\Lib\site-
packages\sklearn\linear_model\_logistic.py:458: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
[31]: tuple(chosen_features_per_model.values())
```

```
[31]: (['Family',
      'CCAvg',
      'Education',
      'Securities Account',
      'CD Account',
      'Online',
      'CreditCard'],
      ['Experience', 'Income', 'Family', 'CCAvg', 'Education', 'CD Account'],
      ['Experience', 'Income', 'ZIP Code', 'Family', 'CCAvg', 'Education'],
      ['Income', 'Family', 'CCAvg', 'Education'])
```

3.2.3 Extracted Features From the list provided we can conclude that the features * Family * CCAvg * Education * Income * CD Account

These are the essential features for the model. However, we will also include Mortgage feature considering the Correlation for the Heatmap as well

```
[32]: # so our new dataset are :-
new_df = personal_loan_df[["Family", "CCAvg", "Education", "Income", "CD_
    Account", "Mortgage", "Personal Loan"]].copy()
```

3.2.4 Validating the Dataset

```
[33]: # checking for missing values
missing_Val(new_df)
```

False, no missing values present

```
[34]: # checking for duplicate values
new_df.duplicated().sum()
```

[34]: 469

```
[35]: print("Shape of the Data : ",new_df.shape)
print("Number of People who took Personal Loan =>", new_df[new_df["Personal_
↳Loan"]==1]["Personal Loan"].value_counts())
```

Shape of the Data : (5000, 7)
Number of People who took Personal Loan => 1 480
Name: Personal Loan, dtype: int64

```
[36]: # dropping the duplicated values from the new_data
new_df.drop_duplicates(inplace=True)
```

```
[37]: print("Shape of the Data : ",new_df.shape)
print("Number of People who took Personal Loan =>", new_df[new_df["Personal_
↳Loan"]==1]["Personal Loan"].value_counts())
```

Shape of the Data : (4531, 7)
Number of People who took Personal Loan => 1 480
Name: Personal Loan, dtype: int64

```
[38]: new_df.to_csv("../data/final_features_data_df.csv")
```

1.5 Task 4: Modelling (25Marks)

1. Split data into X and Y. (1Marks)
2. Split the data into training and test set in the ratio of 70:30 respectively (2Marks)
3. Train a Supervised Learning Classification base model using KNN classifier, Naive Bayes and Logistic Regression. Print the train and test accuracy for the same (12Marks)
4. Print the confusion matrix for all the above models (5Marks)
5. Give your reasoning on which is the best model in this case and why it performs better (5Marks)

1.5.1 4.1 Split X and y

```
[39]: # Split data into X and Y.
X = new_df.iloc[:, :6]
y = new_df["Personal Loan"]
```

1.5.2 4.2 Split Train & Test set

```
[40]: # Split the data into training and test set in the ratio of 70:30 respectively
# stratify means it will produce equal distribution on the test set as it did
↳on train set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
↳stratify=y)

# checkinng the train and test set shape and size
print(f"X_train shape : {X_train.shape}")
print(f"y_train shape : {y_train.shape}")
print(f"X_test shape : {X_test.shape}")
print(f"y_test shape : {y_test.shape}")
```

```
X_train shape : (3171, 6)
y_train shape : (3171,)
X_test shape : (1360, 6)
y_test shape : (1360,)
```

4.2.1 Saving the Train and Test Set

```
[41]: # Combine X_train (features) and y_train (target) into one DataFrame
train_set2 = pd.concat([pd.DataFrame(X_train), pd.DataFrame(y_train)], axis=1)
test_set = pd.concat([pd.DataFrame(X_test) , pd.DataFrame(y_test)], axis=1)

# Optional: name the target column properly
train_set2.columns = list(X_train.columns) + ["Personal Loan"]
test_set.columns = list(X_test.columns) + ["Personal Loan"]

# Save to CSV
train_set2.to_csv('../data/biased cleaned data/biased_train_set.csv',
↳index=False)
test_set.to_csv('../data/biased cleaned data/biased_test_set.csv', index=False)
```

4.2.2 Data Transformation

- ColumnTransformation : Applies different transformations to different columns in a single step.

```
[42]: # Transforming the data train set
print(f"X_train shape : {X_train.shape}")
print(f"X_test shape : {y_train.shape}")

numerical_cols = ["CCAvg", "Income", "Mortgage"]
categorical_cols = ["Family", "Education", "CD Account"]

# transforming categorical and numerical column with ColumnTransformer,
↳StandardScaler and OneHotEncoding
```

```

preprocessing = ColumnTransformer(
    transformers=[
        ('numericals', StandardScaler(), numerical_cols),
        ('categorical', OneHotEncoder(handle_unknown='ignore'), categorica_cols)
    ]
)

# fit and transform the trainset and just transform the test set
X_train_processed = preprocessing.fit_transform(X_train)
X_test_processed = preprocessing.transform(X_test)

```

```

X_train shape : (3171, 6)
X_test shape : (3171,)

```

```
[43]: X_train_processed.shape
```

```
[43]: (3171, 12)
```

4.2.3 SMOTE - Dealing with Minority Set

```

[44]: # Using the Smote() for resolving the imbalance learning from the train set
smt = SMOTE(random_state=333)

# fitting the smote() - Synthetic Minority Oversampling Technique
X_train_resampled , y_train_resampled = smt.fit_resample(X_train_processed,
↳ y_train)

# shape and size of the
print(f"Resampled X_train Shape {X_train_resampled.shape}")
print(f"Resample y_train shape {y_train_resampled.shape}")

```

```

Resampled X_train Shape (5670, 12)
Resample y_train shape (5670,)

```

```

[45]: # number of yes and no
print(f"The number of personal loans (yes) {(y_train_resampled==1).sum()} ")
# number of yes and no
print(f"The number of personal loans (no) {(y_train_resampled==0).sum()} ")

```

```

The number of personal loans (yes) 2835
The number of personal loans (no) 2835

```

- Saving the unbiased trainset

```
[46]: X_train.shape
```

```
[46]: (3171, 6)
```

```
[47]: X_train_resampled.shape
```

```
[47]: (5670, 12)
```

4.2.4 Saving Unbiased trainset

```
[48]: # gettin features name
feature_names = preprocessing.get_feature_names_out()

# Combine features + target
X_df = pd.DataFrame(X_train_resampled, columns=feature_names)
y_df = pd.DataFrame(y_train_resampled, columns=["target"])

# Combine into one DataFrame
train_set = pd.concat([X_df, y_df], axis=1)

# Save to CSV
train_set.to_csv("../data/unbiased dataset/unbiased_train_set.csv", index=False)
print("Saved unbiased train set to CSV!")
```

Saved unbiased train set to CSV!

1.5.3 4.3 Model Training (ML)

4.3.1 Models Description (ML Algo's) Machine learning models that are being used are :- 1. Logistic Regression - a type of regression model that predicts probability - Uses **sigmoid** function 2. SVC - Support Vector Classifier uses **Hyperplane** making the boundaries between 2 or more classification. 3. Random Forest Classifiers - Decision Trees are the building blocks for RFC and better performer than a single decision tree . - focuses on reducing variance by creating diverse, independent models and combining them. 4. KNearestNeighbour [KNN] - Instance Based classifier , chooses neighbours based on the **No.Of neighbours** that you want to choose. 5. RadiusNeighborsClassifier - Instance Based Classifier , chooses its neighbours based on the **distance(radius)** of the current point and what all neighbours **within the radius** are considered **neighbours**.

6. AdaBoostClassifier - Adaptive Boosting Classifier
 - combines multiple weak learners(shallow decision trees) **sequentially** from decision stumps.
7. BaggingClassifier
 - Trains **multiple decision trees** on different **bootstrapped samples** and aggregates their decision.
8. RidgeClassifier
 - **Minimizes squared loss** with **l2 regularization**, works better for small datasets.
9. Gradient Boosting Classifier
 - each new model tries to correct the **residual errors** of the combined previous models by **minimizing a loss function** using **gradient descent**. works best with medium-sized data.
 - focuses on reducing bias by correcting mistakes in sequential trees.
10. CatBoostClassifier - developed by Yandex
 - Categorical feature handling, and symmetric tree handling

11. LGBMClassifier - Light Gradient Boosting Classifier by Microsoft
 - uses **tree-based** learning algorithm, utilizes **leaf-wise growth**, finds more complex patterns.
12. XGBClassifier - eXtreme Gradient Boosting Classifier
 - boosts weak learner and corrects previous one's (trees) mistake.
13. XGBRFCClassifier - eXtreme Gradient Boosting Random Forest Classifier
 - trained with Random Forest and uses framework of gradient boosting.

4.3.2 Model Initialization

```
[49]: # Train a Supervised Learning Classification base model using KNN classifier,
      ↪ Naive Bayes and Logistic Regression. Print the train and test accuracy for
      ↪ the same

# some random parameters for each ML models
classification_ML_models = {
    "Logistic_Regression" : LogisticRegression(penalty="l2",
                                              solver="newton-cholesky"),
    "Support_Vector_Classifiers": SVC(probability=True,
                                      kernel = "sigmoid",
                                      degree=7),
    "Random_Forest_Classifiers" : RandomForestClassifier(max_depth=100 ,
                                                         n_estimators=500, ),
    "K-Nearest_Neighbor": KNeighborsClassifier( n_neighbors=5,
                                                algorithm="auto",
                                                metric="minkowski" ),
    "Radius_Neighbors_Classifiers": RadiusNeighborsClassifier(radius=2.50),
    "Adaptive Boosting Classifiers" : AdaBoostClassifier(n_estimators=100),
    "Bagging Classifiers" : BaggingClassifier( n_estimators=150,
                                              max_samples=6),

    # "RidgeClassifier" : RidgeClassifier(),
    "Gradient_Boosting_Classifiers" : GradientBoostingClassifier(
    ↪ n_estimators=500,
                                                                    max_depth=6,
                                                                    subsample=0.
    ↪ 8),
    "Cat_Boost_Classifier": CatBoostClassifier( iterations = 750,
                                                depth = 15,
                                                grow_policy = 'Depthwise' ),
    "Light_Gradient_Boosting_Machine_Classifiers": LGBMClassifier( n_estimators=
    ↪ 250,
                                                                    num_leaves=15,
                                                                    max_depth =
    ↪ 50 ),
    "eXtreme_Gradient_Boosting_Classifier": XGBClassifier( n_estimators= 750,
                                                            max_depth = 10,
                                                            subsample=0.9 ),
```

```

    "eXtreme_Gradient_Boosting_Random_Forest_Classifiers" :
    ↪XGBRFClassifier(colsample_bynode=1.0,

    ↪num_parallel_tree=150,

    ↪max_depth=8)
}

```

4.3.3 Model Choosing

```

[50]: # a new variable for storing the model accuracy for their train and test set

def ML_model_results(models_ls):
    # initializing an empty list
    model_result = []

    # for loop for training each model with metrics
    for model_name , model in models_ls.items():
        # Fit the model
        model_fit = model.fit(X_train_resampled, y_train_resampled)

        # Score using built-in model score method (usually returns accuracy for
        ↪classifiers)
        model_train_score = model_fit.score(X_train_resampled,
        ↪y_train_resampled)
        model_test_score = model_fit.score(X_test_processed, y_test)

        # Predictions
        y_train_pred = model_fit.predict(X_train_resampled)
        y_test_pred = model_fit.predict(X_test_processed)

        # For metrics that require probabilities (e.g., ROC-AUC)
        y_train_proba = model_fit.predict_proba(X_train_resampled)[: , 1]
        y_test_proba = model_fit.predict_proba(X_test_processed)[: , 1]

        # Accuracy
        model_train_accuracy = accuracy_score(y_train_resampled, y_train_pred)
        model_test_accuracy = accuracy_score(y_test, y_test_pred)

        # F1 Score
        model_f1_train = f1_score(y_train_resampled, y_train_pred)
        model_f1_test = f1_score(y_test, y_test_pred)

        # ROC AUC Score
        model_roc_auc_train = roc_auc_score(y_train_resampled, y_train_proba)
        model_roc_auc_test = roc_auc_score(y_test, y_test_proba)

```

```

# ROC Curve (returns fpr, tpr, thresholds - used for plotting)
fpr_train, tpr_train, thresholds_train = roc_curve(y_train_resampled,
↪y_train_proba)
fpr_test, tpr_test, thresholds_test = roc_curve(y_test, y_test_proba)

model_result.append({
    "Model_name": model_name,
    "train_scores" : model_train_score,
    "test_score" : model_test_score,
    "accuracy_train" : model_train_accuracy,
    "accuracy_test" : model_test_accuracy,
    "F1_score_train" : model_f1_train,
    "F1_score_test" : model_f1_test,
    "ROC_AUC_train" : model_roc_auc_train,
    "ROC_AUC_test" : model_roc_auc_test
})

return model_result

```

```

[51]: # Sorting through all the mentioned models to see which one is better
model_selection_df = pd.DataFrame(ML_model_results(classification_ML_models))

```

Learning rate set to 0.028137

0:	learn: 0.6347841	total: 301ms	remaining: 3m 45s
1:	learn: 0.5775648	total: 388ms	remaining: 2m 24s
2:	learn: 0.5277889	total: 469ms	remaining: 1m 56s
3:	learn: 0.4837246	total: 571ms	remaining: 1m 46s
4:	learn: 0.4441728	total: 667ms	remaining: 1m 39s
5:	learn: 0.4091546	total: 765ms	remaining: 1m 34s
6:	learn: 0.3754533	total: 866ms	remaining: 1m 31s
7:	learn: 0.3442619	total: 975ms	remaining: 1m 30s
8:	learn: 0.3169503	total: 1.07s	remaining: 1m 27s
9:	learn: 0.2915561	total: 1.17s	remaining: 1m 26s
10:	learn: 0.2686292	total: 1.29s	remaining: 1m 27s
11:	learn: 0.2492861	total: 1.39s	remaining: 1m 25s
12:	learn: 0.2318955	total: 1.49s	remaining: 1m 24s
13:	learn: 0.2137785	total: 1.59s	remaining: 1m 23s
14:	learn: 0.1979203	total: 1.68s	remaining: 1m 22s
15:	learn: 0.1849682	total: 1.8s	remaining: 1m 22s
16:	learn: 0.1729093	total: 1.9s	remaining: 1m 21s
17:	learn: 0.1623247	total: 2.01s	remaining: 1m 21s
18:	learn: 0.1509507	total: 2.12s	remaining: 1m 21s
19:	learn: 0.1408389	total: 2.23s	remaining: 1m 21s
20:	learn: 0.1313985	total: 2.33s	remaining: 1m 20s
21:	learn: 0.1229094	total: 2.44s	remaining: 1m 20s
22:	learn: 0.1150943	total: 2.53s	remaining: 1m 19s

23:	learn: 0.1082903	total: 2.66s	remaining: 1m 20s
24:	learn: 0.1017242	total: 2.75s	remaining: 1m 19s
25:	learn: 0.0961323	total: 2.88s	remaining: 1m 20s
26:	learn: 0.0908961	total: 3.02s	remaining: 1m 20s
27:	learn: 0.0860796	total: 3.13s	remaining: 1m 20s
28:	learn: 0.0816272	total: 3.23s	remaining: 1m 20s
29:	learn: 0.0775786	total: 3.33s	remaining: 1m 19s
30:	learn: 0.0740403	total: 3.42s	remaining: 1m 19s
31:	learn: 0.0703934	total: 3.54s	remaining: 1m 19s
32:	learn: 0.0670806	total: 3.67s	remaining: 1m 19s
33:	learn: 0.0637073	total: 3.91s	remaining: 1m 22s
34:	learn: 0.0607524	total: 4.11s	remaining: 1m 23s
35:	learn: 0.0581018	total: 4.27s	remaining: 1m 24s
36:	learn: 0.0554013	total: 4.41s	remaining: 1m 24s
37:	learn: 0.0530821	total: 4.55s	remaining: 1m 25s
38:	learn: 0.0507889	total: 4.68s	remaining: 1m 25s
39:	learn: 0.0488362	total: 4.83s	remaining: 1m 25s
40:	learn: 0.0467249	total: 4.97s	remaining: 1m 25s
41:	learn: 0.0448074	total: 5.08s	remaining: 1m 25s
42:	learn: 0.0431783	total: 5.22s	remaining: 1m 25s
43:	learn: 0.0415468	total: 5.35s	remaining: 1m 25s
44:	learn: 0.0400566	total: 5.48s	remaining: 1m 25s
45:	learn: 0.0385904	total: 5.63s	remaining: 1m 26s
46:	learn: 0.0372096	total: 5.76s	remaining: 1m 26s
47:	learn: 0.0359812	total: 5.92s	remaining: 1m 26s
48:	learn: 0.0347266	total: 6.05s	remaining: 1m 26s
49:	learn: 0.0336446	total: 6.21s	remaining: 1m 26s
50:	learn: 0.0324534	total: 6.35s	remaining: 1m 27s
51:	learn: 0.0313900	total: 6.5s	remaining: 1m 27s
52:	learn: 0.0304214	total: 6.66s	remaining: 1m 27s
53:	learn: 0.0293554	total: 6.82s	remaining: 1m 27s
54:	learn: 0.0284102	total: 7.01s	remaining: 1m 28s
55:	learn: 0.0275888	total: 7.21s	remaining: 1m 29s
56:	learn: 0.0267846	total: 7.36s	remaining: 1m 29s
57:	learn: 0.0260338	total: 7.53s	remaining: 1m 29s
58:	learn: 0.0252591	total: 7.65s	remaining: 1m 29s
59:	learn: 0.0245601	total: 7.8s	remaining: 1m 29s
60:	learn: 0.0239051	total: 7.96s	remaining: 1m 29s
61:	learn: 0.0232451	total: 8.13s	remaining: 1m 30s
62:	learn: 0.0226206	total: 8.27s	remaining: 1m 30s
63:	learn: 0.0220602	total: 8.41s	remaining: 1m 30s
64:	learn: 0.0214907	total: 8.52s	remaining: 1m 29s
65:	learn: 0.0209456	total: 8.61s	remaining: 1m 29s
66:	learn: 0.0204402	total: 8.77s	remaining: 1m 29s
67:	learn: 0.0199271	total: 8.92s	remaining: 1m 29s
68:	learn: 0.0194266	total: 9.09s	remaining: 1m 29s
69:	learn: 0.0189555	total: 9.21s	remaining: 1m 29s
70:	learn: 0.0185155	total: 9.36s	remaining: 1m 29s

71:	learn: 0.0181104	total: 9.52s	remaining: 1m 29s
72:	learn: 0.0177121	total: 9.68s	remaining: 1m 29s
73:	learn: 0.0173056	total: 9.82s	remaining: 1m 29s
74:	learn: 0.0168708	total: 9.97s	remaining: 1m 29s
75:	learn: 0.0164975	total: 10.1s	remaining: 1m 29s
76:	learn: 0.0161708	total: 10.2s	remaining: 1m 29s
77:	learn: 0.0158485	total: 10.3s	remaining: 1m 28s
78:	learn: 0.0155308	total: 10.4s	remaining: 1m 28s
79:	learn: 0.0152351	total: 10.6s	remaining: 1m 28s
80:	learn: 0.0149232	total: 10.7s	remaining: 1m 28s
81:	learn: 0.0146454	total: 10.9s	remaining: 1m 28s
82:	learn: 0.0143783	total: 11s	remaining: 1m 28s
83:	learn: 0.0140821	total: 11.2s	remaining: 1m 28s
84:	learn: 0.0138138	total: 11.3s	remaining: 1m 28s
85:	learn: 0.0135526	total: 11.4s	remaining: 1m 28s
86:	learn: 0.0133232	total: 11.6s	remaining: 1m 28s
87:	learn: 0.0130982	total: 11.7s	remaining: 1m 28s
88:	learn: 0.0128566	total: 11.8s	remaining: 1m 27s
89:	learn: 0.0126404	total: 12s	remaining: 1m 27s
90:	learn: 0.0124050	total: 12.1s	remaining: 1m 27s
91:	learn: 0.0121959	total: 12.2s	remaining: 1m 27s
92:	learn: 0.0119957	total: 12.4s	remaining: 1m 27s
93:	learn: 0.0118092	total: 12.5s	remaining: 1m 27s
94:	learn: 0.0116224	total: 12.6s	remaining: 1m 27s
95:	learn: 0.0114397	total: 12.8s	remaining: 1m 26s
96:	learn: 0.0112482	total: 12.9s	remaining: 1m 26s
97:	learn: 0.0110793	total: 13s	remaining: 1m 26s
98:	learn: 0.0109051	total: 13.2s	remaining: 1m 26s
99:	learn: 0.0107211	total: 13.3s	remaining: 1m 26s
100:	learn: 0.0105735	total: 13.4s	remaining: 1m 26s
101:	learn: 0.0104254	total: 13.5s	remaining: 1m 26s
102:	learn: 0.0102707	total: 13.7s	remaining: 1m 25s
103:	learn: 0.0101186	total: 13.8s	remaining: 1m 25s
104:	learn: 0.0099635	total: 14s	remaining: 1m 25s
105:	learn: 0.0098355	total: 14.1s	remaining: 1m 25s
106:	learn: 0.0096976	total: 14.2s	remaining: 1m 25s
107:	learn: 0.0095723	total: 14.4s	remaining: 1m 25s
108:	learn: 0.0094563	total: 14.5s	remaining: 1m 25s
109:	learn: 0.0093287	total: 14.7s	remaining: 1m 25s
110:	learn: 0.0091820	total: 14.9s	remaining: 1m 25s
111:	learn: 0.0090562	total: 15s	remaining: 1m 25s
112:	learn: 0.0089244	total: 15.2s	remaining: 1m 25s
113:	learn: 0.0088125	total: 15.3s	remaining: 1m 25s
114:	learn: 0.0086960	total: 15.4s	remaining: 1m 25s
115:	learn: 0.0085890	total: 15.6s	remaining: 1m 25s
116:	learn: 0.0084682	total: 15.7s	remaining: 1m 24s
117:	learn: 0.0083607	total: 15.8s	remaining: 1m 24s
118:	learn: 0.0082579	total: 16s	remaining: 1m 24s

119:	learn: 0.0081429	total: 16.1s	remaining: 1m 24s
120:	learn: 0.0080470	total: 16.2s	remaining: 1m 24s
121:	learn: 0.0079273	total: 16.4s	remaining: 1m 24s
122:	learn: 0.0078254	total: 16.5s	remaining: 1m 24s
123:	learn: 0.0077251	total: 16.6s	remaining: 1m 23s
124:	learn: 0.0076438	total: 16.8s	remaining: 1m 23s
125:	learn: 0.0075640	total: 16.9s	remaining: 1m 23s
126:	learn: 0.0074832	total: 17s	remaining: 1m 23s
127:	learn: 0.0074034	total: 17.2s	remaining: 1m 23s
128:	learn: 0.0073208	total: 17.3s	remaining: 1m 23s
129:	learn: 0.0072480	total: 17.5s	remaining: 1m 23s
130:	learn: 0.0071659	total: 17.6s	remaining: 1m 23s
131:	learn: 0.0070852	total: 17.7s	remaining: 1m 22s
132:	learn: 0.0070050	total: 17.8s	remaining: 1m 22s
133:	learn: 0.0069267	total: 18s	remaining: 1m 22s
134:	learn: 0.0068521	total: 18.1s	remaining: 1m 22s
135:	learn: 0.0067764	total: 18.2s	remaining: 1m 22s
136:	learn: 0.0067127	total: 18.4s	remaining: 1m 22s
137:	learn: 0.0066504	total: 18.5s	remaining: 1m 22s
138:	learn: 0.0065812	total: 18.7s	remaining: 1m 22s
139:	learn: 0.0065146	total: 18.8s	remaining: 1m 22s
140:	learn: 0.0064451	total: 19s	remaining: 1m 21s
141:	learn: 0.0063788	total: 19.1s	remaining: 1m 21s
142:	learn: 0.0063132	total: 19.2s	remaining: 1m 21s
143:	learn: 0.0062486	total: 19.4s	remaining: 1m 21s
144:	learn: 0.0061905	total: 19.5s	remaining: 1m 21s
145:	learn: 0.0061311	total: 19.6s	remaining: 1m 21s
146:	learn: 0.0060739	total: 19.7s	remaining: 1m 20s
147:	learn: 0.0060170	total: 19.9s	remaining: 1m 20s
148:	learn: 0.0059640	total: 20s	remaining: 1m 20s
149:	learn: 0.0059099	total: 20.2s	remaining: 1m 20s
150:	learn: 0.0058564	total: 20.3s	remaining: 1m 20s
151:	learn: 0.0058006	total: 20.4s	remaining: 1m 20s
152:	learn: 0.0057465	total: 20.6s	remaining: 1m 20s
153:	learn: 0.0056947	total: 20.7s	remaining: 1m 20s
154:	learn: 0.0056430	total: 20.9s	remaining: 1m 20s
155:	learn: 0.0055916	total: 21s	remaining: 1m 19s
156:	learn: 0.0055422	total: 21.1s	remaining: 1m 19s
157:	learn: 0.0054869	total: 21.3s	remaining: 1m 19s
158:	learn: 0.0054338	total: 21.4s	remaining: 1m 19s
159:	learn: 0.0053902	total: 21.6s	remaining: 1m 19s
160:	learn: 0.0053434	total: 21.7s	remaining: 1m 19s
161:	learn: 0.0052977	total: 21.8s	remaining: 1m 19s
162:	learn: 0.0052493	total: 21.9s	remaining: 1m 19s
163:	learn: 0.0052057	total: 22.1s	remaining: 1m 18s
164:	learn: 0.0051627	total: 22.2s	remaining: 1m 18s
165:	learn: 0.0051189	total: 22.3s	remaining: 1m 18s
166:	learn: 0.0050761	total: 22.4s	remaining: 1m 18s

167:	learn: 0.0050350	total: 22.6s	remaining: 1m 18s
168:	learn: 0.0049954	total: 22.7s	remaining: 1m 18s
169:	learn: 0.0049537	total: 22.8s	remaining: 1m 17s
170:	learn: 0.0049127	total: 22.9s	remaining: 1m 17s
171:	learn: 0.0048749	total: 23.1s	remaining: 1m 17s
172:	learn: 0.0048309	total: 23.2s	remaining: 1m 17s
173:	learn: 0.0047946	total: 23.4s	remaining: 1m 17s
174:	learn: 0.0047516	total: 23.5s	remaining: 1m 17s
175:	learn: 0.0047140	total: 23.7s	remaining: 1m 17s
176:	learn: 0.0046749	total: 23.8s	remaining: 1m 17s
177:	learn: 0.0046403	total: 24s	remaining: 1m 17s
178:	learn: 0.0046055	total: 24.3s	remaining: 1m 17s
179:	learn: 0.0045688	total: 24.4s	remaining: 1m 17s
180:	learn: 0.0045359	total: 24.7s	remaining: 1m 17s
181:	learn: 0.0044978	total: 25s	remaining: 1m 18s
182:	learn: 0.0044668	total: 25.2s	remaining: 1m 18s
183:	learn: 0.0044342	total: 25.4s	remaining: 1m 18s
184:	learn: 0.0043999	total: 25.6s	remaining: 1m 18s
185:	learn: 0.0043638	total: 25.8s	remaining: 1m 18s
186:	learn: 0.0043344	total: 26s	remaining: 1m 18s
187:	learn: 0.0043027	total: 26.2s	remaining: 1m 18s
188:	learn: 0.0042730	total: 26.4s	remaining: 1m 18s
189:	learn: 0.0042397	total: 26.6s	remaining: 1m 18s
190:	learn: 0.0042097	total: 26.7s	remaining: 1m 18s
191:	learn: 0.0041790	total: 26.9s	remaining: 1m 18s
192:	learn: 0.0041459	total: 27s	remaining: 1m 17s
193:	learn: 0.0041179	total: 27.1s	remaining: 1m 17s
194:	learn: 0.0040925	total: 27.2s	remaining: 1m 17s
195:	learn: 0.0040600	total: 27.4s	remaining: 1m 17s
196:	learn: 0.0040356	total: 27.5s	remaining: 1m 17s
197:	learn: 0.0040105	total: 27.7s	remaining: 1m 17s
198:	learn: 0.0039865	total: 27.8s	remaining: 1m 17s
199:	learn: 0.0039592	total: 28s	remaining: 1m 16s
200:	learn: 0.0039324	total: 28.1s	remaining: 1m 16s
201:	learn: 0.0039060	total: 28.3s	remaining: 1m 16s
202:	learn: 0.0038825	total: 28.4s	remaining: 1m 16s
203:	learn: 0.0038572	total: 28.5s	remaining: 1m 16s
204:	learn: 0.0038335	total: 28.7s	remaining: 1m 16s
205:	learn: 0.0038088	total: 28.8s	remaining: 1m 16s
206:	learn: 0.0037853	total: 28.9s	remaining: 1m 15s
207:	learn: 0.0037580	total: 29.1s	remaining: 1m 15s
208:	learn: 0.0037340	total: 29.2s	remaining: 1m 15s
209:	learn: 0.0037100	total: 29.4s	remaining: 1m 15s
210:	learn: 0.0036866	total: 29.6s	remaining: 1m 15s
211:	learn: 0.0036656	total: 29.8s	remaining: 1m 15s
212:	learn: 0.0036437	total: 30s	remaining: 1m 15s
213:	learn: 0.0036191	total: 30.1s	remaining: 1m 15s
214:	learn: 0.0035923	total: 30.3s	remaining: 1m 15s

215:	learn: 0.0035695	total: 30.4s	remaining: 1m 15s
216:	learn: 0.0035484	total: 30.6s	remaining: 1m 15s
217:	learn: 0.0035279	total: 30.8s	remaining: 1m 15s
218:	learn: 0.0035066	total: 30.9s	remaining: 1m 14s
219:	learn: 0.0034854	total: 31s	remaining: 1m 14s
220:	learn: 0.0034650	total: 31.2s	remaining: 1m 14s
221:	learn: 0.0034442	total: 31.3s	remaining: 1m 14s
222:	learn: 0.0034246	total: 31.4s	remaining: 1m 14s
223:	learn: 0.0034024	total: 31.6s	remaining: 1m 14s
224:	learn: 0.0033831	total: 31.7s	remaining: 1m 14s
225:	learn: 0.0033655	total: 31.9s	remaining: 1m 13s
226:	learn: 0.0033474	total: 32.1s	remaining: 1m 13s
227:	learn: 0.0033296	total: 32.2s	remaining: 1m 13s
228:	learn: 0.0033112	total: 32.4s	remaining: 1m 13s
229:	learn: 0.0032934	total: 32.5s	remaining: 1m 13s
230:	learn: 0.0032751	total: 32.6s	remaining: 1m 13s
231:	learn: 0.0032579	total: 32.8s	remaining: 1m 13s
232:	learn: 0.0032417	total: 32.9s	remaining: 1m 13s
233:	learn: 0.0032239	total: 33.1s	remaining: 1m 12s
234:	learn: 0.0032049	total: 33.2s	remaining: 1m 12s
235:	learn: 0.0031847	total: 33.3s	remaining: 1m 12s
236:	learn: 0.0031675	total: 33.4s	remaining: 1m 12s
237:	learn: 0.0031526	total: 33.6s	remaining: 1m 12s
238:	learn: 0.0031332	total: 33.7s	remaining: 1m 12s
239:	learn: 0.0031161	total: 33.8s	remaining: 1m 11s
240:	learn: 0.0030987	total: 33.9s	remaining: 1m 11s
241:	learn: 0.0030818	total: 34.1s	remaining: 1m 11s
242:	learn: 0.0030665	total: 34.2s	remaining: 1m 11s
243:	learn: 0.0030512	total: 34.4s	remaining: 1m 11s
244:	learn: 0.0030358	total: 34.5s	remaining: 1m 11s
245:	learn: 0.0030189	total: 34.7s	remaining: 1m 11s
246:	learn: 0.0030042	total: 34.9s	remaining: 1m 10s
247:	learn: 0.0029884	total: 35s	remaining: 1m 10s
248:	learn: 0.0029713	total: 35.2s	remaining: 1m 10s
249:	learn: 0.0029533	total: 35.3s	remaining: 1m 10s
250:	learn: 0.0029370	total: 35.4s	remaining: 1m 10s
251:	learn: 0.0029234	total: 35.6s	remaining: 1m 10s
252:	learn: 0.0029085	total: 35.7s	remaining: 1m 10s
253:	learn: 0.0028931	total: 35.9s	remaining: 1m 10s
254:	learn: 0.0028776	total: 36s	remaining: 1m 9s
255:	learn: 0.0028642	total: 36.1s	remaining: 1m 9s
256:	learn: 0.0028507	total: 36.3s	remaining: 1m 9s
257:	learn: 0.0028375	total: 36.4s	remaining: 1m 9s
258:	learn: 0.0028254	total: 36.6s	remaining: 1m 9s
259:	learn: 0.0028120	total: 36.7s	remaining: 1m 9s
260:	learn: 0.0027974	total: 36.8s	remaining: 1m 9s
261:	learn: 0.0027847	total: 37s	remaining: 1m 8s
262:	learn: 0.0027727	total: 37.1s	remaining: 1m 8s

263:	learn: 0.0027601	total: 37.3s	remaining: 1m 8s
264:	learn: 0.0027442	total: 37.4s	remaining: 1m 8s
265:	learn: 0.0027299	total: 37.5s	remaining: 1m 8s
266:	learn: 0.0027163	total: 37.6s	remaining: 1m 8s
267:	learn: 0.0027055	total: 37.8s	remaining: 1m 7s
268:	learn: 0.0026925	total: 37.9s	remaining: 1m 7s
269:	learn: 0.0026813	total: 38.1s	remaining: 1m 7s
270:	learn: 0.0026696	total: 38.2s	remaining: 1m 7s
271:	learn: 0.0026592	total: 38.4s	remaining: 1m 7s
272:	learn: 0.0026470	total: 38.5s	remaining: 1m 7s
273:	learn: 0.0026352	total: 38.7s	remaining: 1m 7s
274:	learn: 0.0026241	total: 38.8s	remaining: 1m 7s
275:	learn: 0.0026127	total: 39s	remaining: 1m 6s
276:	learn: 0.0026011	total: 39.1s	remaining: 1m 6s
277:	learn: 0.0025899	total: 39.2s	remaining: 1m 6s
278:	learn: 0.0025764	total: 39.3s	remaining: 1m 6s
279:	learn: 0.0025658	total: 39.5s	remaining: 1m 6s
280:	learn: 0.0025550	total: 39.6s	remaining: 1m 6s
281:	learn: 0.0025448	total: 39.8s	remaining: 1m 6s
282:	learn: 0.0025339	total: 39.9s	remaining: 1m 5s
283:	learn: 0.0025233	total: 40s	remaining: 1m 5s
284:	learn: 0.0025130	total: 40.2s	remaining: 1m 5s
285:	learn: 0.0025032	total: 40.3s	remaining: 1m 5s
286:	learn: 0.0024906	total: 40.5s	remaining: 1m 5s
287:	learn: 0.0024806	total: 40.6s	remaining: 1m 5s
288:	learn: 0.0024695	total: 40.8s	remaining: 1m 5s
289:	learn: 0.0024589	total: 40.9s	remaining: 1m 4s
290:	learn: 0.0024477	total: 41s	remaining: 1m 4s
291:	learn: 0.0024369	total: 41.2s	remaining: 1m 4s
292:	learn: 0.0024270	total: 41.3s	remaining: 1m 4s
293:	learn: 0.0024172	total: 41.5s	remaining: 1m 4s
294:	learn: 0.0024064	total: 41.6s	remaining: 1m 4s
295:	learn: 0.0023971	total: 41.8s	remaining: 1m 4s
296:	learn: 0.0023885	total: 41.9s	remaining: 1m 3s
297:	learn: 0.0023792	total: 42.1s	remaining: 1m 3s
298:	learn: 0.0023705	total: 42.3s	remaining: 1m 3s
299:	learn: 0.0023603	total: 42.5s	remaining: 1m 3s
300:	learn: 0.0023512	total: 42.6s	remaining: 1m 3s
301:	learn: 0.0023426	total: 42.8s	remaining: 1m 3s
302:	learn: 0.0023336	total: 43s	remaining: 1m 3s
303:	learn: 0.0023242	total: 43.2s	remaining: 1m 3s
304:	learn: 0.0023151	total: 43.3s	remaining: 1m 3s
305:	learn: 0.0023056	total: 43.5s	remaining: 1m 3s
306:	learn: 0.0022966	total: 43.6s	remaining: 1m 2s
307:	learn: 0.0022874	total: 43.8s	remaining: 1m 2s
308:	learn: 0.0022790	total: 43.9s	remaining: 1m 2s
309:	learn: 0.0022705	total: 44.1s	remaining: 1m 2s
310:	learn: 0.0022615	total: 44.3s	remaining: 1m 2s

311:	learn: 0.0022536	total: 44.5s	remaining: 1m 2s
312:	learn: 0.0022442	total: 44.6s	remaining: 1m 2s
313:	learn: 0.0022352	total: 44.8s	remaining: 1m 2s
314:	learn: 0.0022277	total: 44.9s	remaining: 1m 2s
315:	learn: 0.0022182	total: 45.1s	remaining: 1m 1s
316:	learn: 0.0022107	total: 45.3s	remaining: 1m 1s
317:	learn: 0.0022020	total: 45.5s	remaining: 1m 1s
318:	learn: 0.0021923	total: 45.7s	remaining: 1m 1s
319:	learn: 0.0021848	total: 45.9s	remaining: 1m 1s
320:	learn: 0.0021775	total: 46.2s	remaining: 1m 1s
321:	learn: 0.0021698	total: 46.4s	remaining: 1m 1s
322:	learn: 0.0021621	total: 46.6s	remaining: 1m 1s
323:	learn: 0.0021544	total: 46.7s	remaining: 1m 1s
324:	learn: 0.0021462	total: 46.9s	remaining: 1m 1s
325:	learn: 0.0021380	total: 47.1s	remaining: 1m 1s
326:	learn: 0.0021307	total: 47.3s	remaining: 1m 1s
327:	learn: 0.0021229	total: 47.4s	remaining: 1m 1s
328:	learn: 0.0021162	total: 47.6s	remaining: 1m
329:	learn: 0.0021083	total: 47.8s	remaining: 1m
330:	learn: 0.0021013	total: 48s	remaining: 1m
331:	learn: 0.0020939	total: 48.2s	remaining: 1m
332:	learn: 0.0020868	total: 48.4s	remaining: 1m
333:	learn: 0.0020799	total: 48.6s	remaining: 1m
334:	learn: 0.0020731	total: 48.8s	remaining: 1m
335:	learn: 0.0020665	total: 49s	remaining: 1m
336:	learn: 0.0020600	total: 49.2s	remaining: 1m
337:	learn: 0.0020531	total: 49.4s	remaining: 1m
338:	learn: 0.0020467	total: 49.7s	remaining: 1m
339:	learn: 0.0020398	total: 49.9s	remaining: 1m
340:	learn: 0.0020333	total: 50.1s	remaining: 1m
341:	learn: 0.0020259	total: 50.3s	remaining: 60s
342:	learn: 0.0020191	total: 50.5s	remaining: 59.9s
343:	learn: 0.0020118	total: 50.7s	remaining: 59.8s
344:	learn: 0.0020054	total: 50.9s	remaining: 59.8s
345:	learn: 0.0019990	total: 51.1s	remaining: 59.6s
346:	learn: 0.0019926	total: 51.3s	remaining: 59.5s
347:	learn: 0.0019869	total: 51.5s	remaining: 59.4s
348:	learn: 0.0019799	total: 51.6s	remaining: 59.3s
349:	learn: 0.0019732	total: 51.8s	remaining: 59.2s
350:	learn: 0.0019665	total: 52s	remaining: 59.1s
351:	learn: 0.0019594	total: 52.1s	remaining: 59s
352:	learn: 0.0019533	total: 52.3s	remaining: 58.9s
353:	learn: 0.0019476	total: 52.5s	remaining: 58.8s
354:	learn: 0.0019416	total: 52.8s	remaining: 58.7s
355:	learn: 0.0019354	total: 53.1s	remaining: 58.7s
356:	learn: 0.0019297	total: 53.3s	remaining: 58.6s
357:	learn: 0.0019244	total: 53.4s	remaining: 58.5s
358:	learn: 0.0019182	total: 53.6s	remaining: 58.4s

359:	learn: 0.0019123	total: 53.8s	remaining: 58.2s
360:	learn: 0.0019070	total: 53.9s	remaining: 58.1s
361:	learn: 0.0019017	total: 54.1s	remaining: 57.9s
362:	learn: 0.0018964	total: 54.3s	remaining: 57.8s
363:	learn: 0.0018900	total: 54.4s	remaining: 57.7s
364:	learn: 0.0018850	total: 54.7s	remaining: 57.7s
365:	learn: 0.0018802	total: 54.8s	remaining: 57.5s
366:	learn: 0.0018752	total: 55s	remaining: 57.4s
367:	learn: 0.0018700	total: 55.2s	remaining: 57.3s
368:	learn: 0.0018646	total: 55.4s	remaining: 57.2s
369:	learn: 0.0018595	total: 55.6s	remaining: 57.1s
370:	learn: 0.0018540	total: 55.8s	remaining: 57s
371:	learn: 0.0018491	total: 56s	remaining: 56.9s
372:	learn: 0.0018433	total: 56.3s	remaining: 56.9s
373:	learn: 0.0018390	total: 56.6s	remaining: 56.9s
374:	learn: 0.0018344	total: 56.8s	remaining: 56.8s
375:	learn: 0.0018299	total: 57s	remaining: 56.6s
376:	learn: 0.0018247	total: 57.1s	remaining: 56.5s
377:	learn: 0.0018205	total: 57.4s	remaining: 56.5s
378:	learn: 0.0018163	total: 57.6s	remaining: 56.4s
379:	learn: 0.0018120	total: 57.8s	remaining: 56.2s
380:	learn: 0.0018070	total: 57.9s	remaining: 56.1s
381:	learn: 0.0018026	total: 58.1s	remaining: 56s
382:	learn: 0.0017984	total: 58.3s	remaining: 55.9s
383:	learn: 0.0017949	total: 58.5s	remaining: 55.7s
384:	learn: 0.0017906	total: 58.7s	remaining: 55.7s
385:	learn: 0.0017864	total: 59.1s	remaining: 55.7s
386:	learn: 0.0017822	total: 59.4s	remaining: 55.7s
387:	learn: 0.0017783	total: 59.7s	remaining: 55.7s
388:	learn: 0.0017741	total: 60s	remaining: 55.7s
389:	learn: 0.0017699	total: 1m	remaining: 55.6s
390:	learn: 0.0017661	total: 1m	remaining: 55.5s
391:	learn: 0.0017611	total: 1m	remaining: 55.4s
392:	learn: 0.0017578	total: 1m	remaining: 55.3s
393:	learn: 0.0017545	total: 1m 1s	remaining: 55.3s
394:	learn: 0.0017511	total: 1m 1s	remaining: 55.2s
395:	learn: 0.0017496	total: 1m 1s	remaining: 55.1s
396:	learn: 0.0017457	total: 1m 1s	remaining: 55s
397:	learn: 0.0017420	total: 1m 2s	remaining: 54.9s
398:	learn: 0.0017380	total: 1m 2s	remaining: 54.8s
399:	learn: 0.0017340	total: 1m 2s	remaining: 54.7s
400:	learn: 0.0017303	total: 1m 2s	remaining: 54.6s
401:	learn: 0.0017267	total: 1m 2s	remaining: 54.5s
402:	learn: 0.0017225	total: 1m 3s	remaining: 54.4s
403:	learn: 0.0017183	total: 1m 3s	remaining: 54.2s
404:	learn: 0.0017146	total: 1m 3s	remaining: 54.1s
405:	learn: 0.0017112	total: 1m 3s	remaining: 54s
406:	learn: 0.0017081	total: 1m 3s	remaining: 53.8s

407:	learn: 0.0017048	total: 1m 4s	remaining: 53.7s
408:	learn: 0.0017015	total: 1m 4s	remaining: 53.6s
409:	learn: 0.0016982	total: 1m 4s	remaining: 53.4s
410:	learn: 0.0016951	total: 1m 4s	remaining: 53.3s
411:	learn: 0.0016923	total: 1m 4s	remaining: 53.2s
412:	learn: 0.0016894	total: 1m 5s	remaining: 53.2s
413:	learn: 0.0016860	total: 1m 5s	remaining: 53.1s
414:	learn: 0.0016832	total: 1m 5s	remaining: 53s
415:	learn: 0.0016799	total: 1m 5s	remaining: 52.9s
416:	learn: 0.0016766	total: 1m 6s	remaining: 52.8s
417:	learn: 0.0016731	total: 1m 6s	remaining: 52.8s
418:	learn: 0.0016698	total: 1m 6s	remaining: 52.6s
419:	learn: 0.0016672	total: 1m 6s	remaining: 52.5s
420:	learn: 0.0016642	total: 1m 7s	remaining: 52.5s
421:	learn: 0.0016611	total: 1m 7s	remaining: 52.4s
422:	learn: 0.0016611	total: 1m 7s	remaining: 52.3s
423:	learn: 0.0016575	total: 1m 7s	remaining: 52.2s
424:	learn: 0.0016575	total: 1m 8s	remaining: 52s
425:	learn: 0.0016559	total: 1m 8s	remaining: 51.9s
426:	learn: 0.0016559	total: 1m 8s	remaining: 51.9s
427:	learn: 0.0016524	total: 1m 8s	remaining: 51.8s
428:	learn: 0.0016493	total: 1m 9s	remaining: 51.7s
429:	learn: 0.0016467	total: 1m 9s	remaining: 51.5s
430:	learn: 0.0016438	total: 1m 9s	remaining: 51.4s
431:	learn: 0.0016406	total: 1m 9s	remaining: 51.3s
432:	learn: 0.0016406	total: 1m 9s	remaining: 51.2s
433:	learn: 0.0016380	total: 1m 10s	remaining: 51.2s
434:	learn: 0.0016380	total: 1m 10s	remaining: 51.1s
435:	learn: 0.0016348	total: 1m 10s	remaining: 51s
436:	learn: 0.0016348	total: 1m 11s	remaining: 50.9s
437:	learn: 0.0016348	total: 1m 11s	remaining: 50.8s
438:	learn: 0.0016318	total: 1m 11s	remaining: 50.6s
439:	learn: 0.0016318	total: 1m 11s	remaining: 50.5s
440:	learn: 0.0016318	total: 1m 12s	remaining: 50.5s
441:	learn: 0.0016293	total: 1m 12s	remaining: 50.4s
442:	learn: 0.0016293	total: 1m 12s	remaining: 50.3s
443:	learn: 0.0016293	total: 1m 12s	remaining: 50.2s
444:	learn: 0.0016293	total: 1m 13s	remaining: 50.1s
445:	learn: 0.0016293	total: 1m 13s	remaining: 50s
446:	learn: 0.0016293	total: 1m 13s	remaining: 49.9s
447:	learn: 0.0016293	total: 1m 13s	remaining: 49.8s
448:	learn: 0.0016293	total: 1m 14s	remaining: 49.8s
449:	learn: 0.0016293	total: 1m 14s	remaining: 49.7s
450:	learn: 0.0016293	total: 1m 14s	remaining: 49.6s
451:	learn: 0.0016293	total: 1m 15s	remaining: 49.5s
452:	learn: 0.0016293	total: 1m 15s	remaining: 49.4s
453:	learn: 0.0016293	total: 1m 15s	remaining: 49.3s
454:	learn: 0.0016267	total: 1m 15s	remaining: 49.2s

455:	learn: 0.0016267	total: 1m 16s	remaining: 49.1s
456:	learn: 0.0016241	total: 1m 16s	remaining: 49s
457:	learn: 0.0016241	total: 1m 16s	remaining: 48.9s
458:	learn: 0.0016240	total: 1m 16s	remaining: 48.8s
459:	learn: 0.0016240	total: 1m 17s	remaining: 48.7s
460:	learn: 0.0016240	total: 1m 17s	remaining: 48.6s
461:	learn: 0.0016240	total: 1m 17s	remaining: 48.5s
462:	learn: 0.0016215	total: 1m 18s	remaining: 48.4s
463:	learn: 0.0016215	total: 1m 18s	remaining: 48.3s
464:	learn: 0.0016191	total: 1m 18s	remaining: 48.2s
465:	learn: 0.0016191	total: 1m 19s	remaining: 48.2s
466:	learn: 0.0016191	total: 1m 19s	remaining: 48.1s
467:	learn: 0.0016191	total: 1m 19s	remaining: 48s
468:	learn: 0.0016191	total: 1m 19s	remaining: 47.9s
469:	learn: 0.0016191	total: 1m 20s	remaining: 47.8s
470:	learn: 0.0016191	total: 1m 20s	remaining: 47.8s
471:	learn: 0.0016191	total: 1m 20s	remaining: 47.7s
472:	learn: 0.0016191	total: 1m 21s	remaining: 47.6s
473:	learn: 0.0016191	total: 1m 21s	remaining: 47.5s
474:	learn: 0.0016191	total: 1m 21s	remaining: 47.4s
475:	learn: 0.0016191	total: 1m 22s	remaining: 47.3s
476:	learn: 0.0016191	total: 1m 22s	remaining: 47.1s
477:	learn: 0.0016191	total: 1m 22s	remaining: 47s
478:	learn: 0.0016191	total: 1m 22s	remaining: 46.9s
479:	learn: 0.0016191	total: 1m 23s	remaining: 46.8s
480:	learn: 0.0016191	total: 1m 23s	remaining: 46.7s
481:	learn: 0.0016191	total: 1m 23s	remaining: 46.6s
482:	learn: 0.0016191	total: 1m 24s	remaining: 46.5s
483:	learn: 0.0016190	total: 1m 24s	remaining: 46.4s
484:	learn: 0.0016191	total: 1m 24s	remaining: 46.2s
485:	learn: 0.0016190	total: 1m 24s	remaining: 46.1s
486:	learn: 0.0016190	total: 1m 25s	remaining: 46s
487:	learn: 0.0016190	total: 1m 25s	remaining: 45.9s
488:	learn: 0.0016190	total: 1m 25s	remaining: 45.8s
489:	learn: 0.0016190	total: 1m 26s	remaining: 45.6s
490:	learn: 0.0016190	total: 1m 26s	remaining: 45.5s
491:	learn: 0.0016190	total: 1m 26s	remaining: 45.4s
492:	learn: 0.0016190	total: 1m 26s	remaining: 45.3s
493:	learn: 0.0016190	total: 1m 27s	remaining: 45.2s
494:	learn: 0.0016190	total: 1m 27s	remaining: 45.1s
495:	learn: 0.0016190	total: 1m 27s	remaining: 44.9s
496:	learn: 0.0016190	total: 1m 28s	remaining: 44.8s
497:	learn: 0.0016190	total: 1m 28s	remaining: 44.7s
498:	learn: 0.0016190	total: 1m 28s	remaining: 44.6s
499:	learn: 0.0016190	total: 1m 28s	remaining: 44.5s
500:	learn: 0.0016190	total: 1m 29s	remaining: 44.4s
501:	learn: 0.0016190	total: 1m 29s	remaining: 44.2s
502:	learn: 0.0016190	total: 1m 29s	remaining: 44.1s

503:	learn: 0.0016190	total: 1m 30s	remaining: 44s
504:	learn: 0.0016166	total: 1m 30s	remaining: 43.9s
505:	learn: 0.0016166	total: 1m 30s	remaining: 43.8s
506:	learn: 0.0016166	total: 1m 31s	remaining: 43.6s
507:	learn: 0.0016165	total: 1m 31s	remaining: 43.5s
508:	learn: 0.0016166	total: 1m 31s	remaining: 43.4s
509:	learn: 0.0016165	total: 1m 31s	remaining: 43.3s
510:	learn: 0.0016165	total: 1m 32s	remaining: 43.1s
511:	learn: 0.0016165	total: 1m 32s	remaining: 43s
512:	learn: 0.0016165	total: 1m 32s	remaining: 42.9s
513:	learn: 0.0016165	total: 1m 33s	remaining: 42.7s
514:	learn: 0.0016165	total: 1m 33s	remaining: 42.6s
515:	learn: 0.0016165	total: 1m 33s	remaining: 42.5s
516:	learn: 0.0016165	total: 1m 33s	remaining: 42.3s
517:	learn: 0.0016165	total: 1m 34s	remaining: 42.2s
518:	learn: 0.0016165	total: 1m 34s	remaining: 42.1s
519:	learn: 0.0016165	total: 1m 34s	remaining: 41.9s
520:	learn: 0.0016165	total: 1m 35s	remaining: 41.8s
521:	learn: 0.0016165	total: 1m 35s	remaining: 41.7s
522:	learn: 0.0016165	total: 1m 35s	remaining: 41.5s
523:	learn: 0.0016165	total: 1m 36s	remaining: 41.4s
524:	learn: 0.0016165	total: 1m 36s	remaining: 41.3s
525:	learn: 0.0016165	total: 1m 36s	remaining: 41.1s
526:	learn: 0.0016165	total: 1m 36s	remaining: 41s
527:	learn: 0.0016165	total: 1m 37s	remaining: 40.9s
528:	learn: 0.0016165	total: 1m 37s	remaining: 40.8s
529:	learn: 0.0016165	total: 1m 37s	remaining: 40.6s
530:	learn: 0.0016165	total: 1m 38s	remaining: 40.5s
531:	learn: 0.0016165	total: 1m 38s	remaining: 40.3s
532:	learn: 0.0016164	total: 1m 38s	remaining: 40.2s
533:	learn: 0.0016165	total: 1m 39s	remaining: 40.1s
534:	learn: 0.0016164	total: 1m 39s	remaining: 39.9s
535:	learn: 0.0016164	total: 1m 39s	remaining: 39.8s
536:	learn: 0.0016164	total: 1m 39s	remaining: 39.7s
537:	learn: 0.0016164	total: 1m 40s	remaining: 39.5s
538:	learn: 0.0016164	total: 1m 40s	remaining: 39.4s
539:	learn: 0.0016164	total: 1m 40s	remaining: 39.2s
540:	learn: 0.0016164	total: 1m 41s	remaining: 39.1s
541:	learn: 0.0016164	total: 1m 41s	remaining: 39s
542:	learn: 0.0016164	total: 1m 41s	remaining: 38.8s
543:	learn: 0.0016164	total: 1m 42s	remaining: 38.7s
544:	learn: 0.0016164	total: 1m 42s	remaining: 38.6s
545:	learn: 0.0016164	total: 1m 42s	remaining: 38.4s
546:	learn: 0.0016164	total: 1m 43s	remaining: 38.3s
547:	learn: 0.0016164	total: 1m 43s	remaining: 38.1s
548:	learn: 0.0016164	total: 1m 43s	remaining: 38s
549:	learn: 0.0016164	total: 1m 44s	remaining: 37.9s
550:	learn: 0.0016164	total: 1m 44s	remaining: 37.8s

551:	learn: 0.0016164	total: 1m 44s	remaining: 37.7s
552:	learn: 0.0016163	total: 1m 45s	remaining: 37.5s
553:	learn: 0.0016163	total: 1m 45s	remaining: 37.4s
554:	learn: 0.0016163	total: 1m 45s	remaining: 37.2s
555:	learn: 0.0016163	total: 1m 46s	remaining: 37.1s
556:	learn: 0.0016163	total: 1m 46s	remaining: 36.9s
557:	learn: 0.0016163	total: 1m 46s	remaining: 36.8s
558:	learn: 0.0016163	total: 1m 47s	remaining: 36.7s
559:	learn: 0.0016163	total: 1m 47s	remaining: 36.5s
560:	learn: 0.0016163	total: 1m 47s	remaining: 36.4s
561:	learn: 0.0016163	total: 1m 48s	remaining: 36.2s
562:	learn: 0.0016163	total: 1m 48s	remaining: 36.1s
563:	learn: 0.0016163	total: 1m 48s	remaining: 35.9s
564:	learn: 0.0016163	total: 1m 49s	remaining: 35.8s
565:	learn: 0.0016163	total: 1m 49s	remaining: 35.6s
566:	learn: 0.0016163	total: 1m 50s	remaining: 35.5s
567:	learn: 0.0016163	total: 1m 50s	remaining: 35.4s
568:	learn: 0.0016163	total: 1m 50s	remaining: 35.2s
569:	learn: 0.0016163	total: 1m 51s	remaining: 35.1s
570:	learn: 0.0016163	total: 1m 51s	remaining: 34.9s
571:	learn: 0.0016163	total: 1m 51s	remaining: 34.7s
572:	learn: 0.0016163	total: 1m 51s	remaining: 34.6s
573:	learn: 0.0016162	total: 1m 52s	remaining: 34.4s
574:	learn: 0.0016163	total: 1m 52s	remaining: 34.3s
575:	learn: 0.0016162	total: 1m 52s	remaining: 34.1s
576:	learn: 0.0016162	total: 1m 53s	remaining: 34s
577:	learn: 0.0016162	total: 1m 53s	remaining: 33.8s
578:	learn: 0.0016162	total: 1m 53s	remaining: 33.6s
579:	learn: 0.0016162	total: 1m 54s	remaining: 33.5s
580:	learn: 0.0016162	total: 1m 54s	remaining: 33.4s
581:	learn: 0.0016162	total: 1m 55s	remaining: 33.2s
582:	learn: 0.0016162	total: 1m 55s	remaining: 33.1s
583:	learn: 0.0016162	total: 1m 55s	remaining: 32.9s
584:	learn: 0.0016162	total: 1m 56s	remaining: 32.8s
585:	learn: 0.0016162	total: 1m 56s	remaining: 32.6s
586:	learn: 0.0016162	total: 1m 56s	remaining: 32.4s
587:	learn: 0.0016162	total: 1m 57s	remaining: 32.3s
588:	learn: 0.0016162	total: 1m 57s	remaining: 32.1s
589:	learn: 0.0016162	total: 1m 57s	remaining: 31.9s
590:	learn: 0.0016162	total: 1m 58s	remaining: 31.8s
591:	learn: 0.0016162	total: 1m 58s	remaining: 31.6s
592:	learn: 0.0016162	total: 1m 58s	remaining: 31.4s
593:	learn: 0.0016162	total: 1m 59s	remaining: 31.3s
594:	learn: 0.0016162	total: 1m 59s	remaining: 31.1s
595:	learn: 0.0016162	total: 1m 59s	remaining: 31s
596:	learn: 0.0016162	total: 2m	remaining: 30.8s
597:	learn: 0.0016162	total: 2m	remaining: 30.6s
598:	learn: 0.0016162	total: 2m	remaining: 30.5s

599:	learn: 0.0016162	total: 2m 1s	remaining: 30.3s
600:	learn: 0.0016162	total: 2m 1s	remaining: 30.1s
601:	learn: 0.0016162	total: 2m 1s	remaining: 30s
602:	learn: 0.0016162	total: 2m 2s	remaining: 29.8s
603:	learn: 0.0016162	total: 2m 2s	remaining: 29.6s
604:	learn: 0.0016162	total: 2m 2s	remaining: 29.5s
605:	learn: 0.0016162	total: 2m 3s	remaining: 29.3s
606:	learn: 0.0016162	total: 2m 3s	remaining: 29.1s
607:	learn: 0.0016162	total: 2m 3s	remaining: 28.9s
608:	learn: 0.0016162	total: 2m 4s	remaining: 28.8s
609:	learn: 0.0016162	total: 2m 4s	remaining: 28.6s
610:	learn: 0.0016162	total: 2m 4s	remaining: 28.4s
611:	learn: 0.0016162	total: 2m 5s	remaining: 28.2s
612:	learn: 0.0016162	total: 2m 5s	remaining: 28.1s
613:	learn: 0.0016162	total: 2m 5s	remaining: 27.9s
614:	learn: 0.0016162	total: 2m 6s	remaining: 27.7s
615:	learn: 0.0016162	total: 2m 6s	remaining: 27.6s
616:	learn: 0.0016162	total: 2m 7s	remaining: 27.4s
617:	learn: 0.0016162	total: 2m 7s	remaining: 27.2s
618:	learn: 0.0016162	total: 2m 7s	remaining: 27s
619:	learn: 0.0016162	total: 2m 8s	remaining: 26.9s
620:	learn: 0.0016162	total: 2m 8s	remaining: 26.7s
621:	learn: 0.0016162	total: 2m 8s	remaining: 26.5s
622:	learn: 0.0016162	total: 2m 9s	remaining: 26.3s
623:	learn: 0.0016162	total: 2m 9s	remaining: 26.2s
624:	learn: 0.0016162	total: 2m 9s	remaining: 26s
625:	learn: 0.0016162	total: 2m 10s	remaining: 25.8s
626:	learn: 0.0016162	total: 2m 10s	remaining: 25.6s
627:	learn: 0.0016162	total: 2m 11s	remaining: 25.5s
628:	learn: 0.0016162	total: 2m 11s	remaining: 25.3s
629:	learn: 0.0016162	total: 2m 11s	remaining: 25.1s
630:	learn: 0.0016162	total: 2m 12s	remaining: 24.9s
631:	learn: 0.0016162	total: 2m 12s	remaining: 24.8s
632:	learn: 0.0016162	total: 2m 12s	remaining: 24.6s
633:	learn: 0.0016162	total: 2m 13s	remaining: 24.4s
634:	learn: 0.0016162	total: 2m 13s	remaining: 24.2s
635:	learn: 0.0016162	total: 2m 14s	remaining: 24s
636:	learn: 0.0016162	total: 2m 14s	remaining: 23.9s
637:	learn: 0.0016162	total: 2m 14s	remaining: 23.7s
638:	learn: 0.0016162	total: 2m 15s	remaining: 23.5s
639:	learn: 0.0016139	total: 2m 15s	remaining: 23.3s
640:	learn: 0.0016139	total: 2m 15s	remaining: 23.1s
641:	learn: 0.0016139	total: 2m 16s	remaining: 22.9s
642:	learn: 0.0016139	total: 2m 16s	remaining: 22.8s
643:	learn: 0.0016139	total: 2m 17s	remaining: 22.6s
644:	learn: 0.0016139	total: 2m 17s	remaining: 22.4s
645:	learn: 0.0016139	total: 2m 18s	remaining: 22.2s
646:	learn: 0.0016139	total: 2m 18s	remaining: 22s

647:	learn: 0.0016139	total: 2m 18s	remaining: 21.8s
648:	learn: 0.0016139	total: 2m 19s	remaining: 21.7s
649:	learn: 0.0016139	total: 2m 19s	remaining: 21.5s
650:	learn: 0.0016139	total: 2m 20s	remaining: 21.3s
651:	learn: 0.0016139	total: 2m 20s	remaining: 21.2s
652:	learn: 0.0016139	total: 2m 21s	remaining: 21s
653:	learn: 0.0016139	total: 2m 21s	remaining: 20.8s
654:	learn: 0.0016139	total: 2m 22s	remaining: 20.6s
655:	learn: 0.0016139	total: 2m 22s	remaining: 20.4s
656:	learn: 0.0016139	total: 2m 22s	remaining: 20.2s
657:	learn: 0.0016139	total: 2m 23s	remaining: 20s
658:	learn: 0.0016139	total: 2m 23s	remaining: 19.8s
659:	learn: 0.0016139	total: 2m 23s	remaining: 19.6s
660:	learn: 0.0016139	total: 2m 24s	remaining: 19.4s
661:	learn: 0.0016139	total: 2m 24s	remaining: 19.2s
662:	learn: 0.0016139	total: 2m 24s	remaining: 19s
663:	learn: 0.0016139	total: 2m 24s	remaining: 18.8s
664:	learn: 0.0016139	total: 2m 25s	remaining: 18.6s
665:	learn: 0.0016139	total: 2m 25s	remaining: 18.4s
666:	learn: 0.0016139	total: 2m 25s	remaining: 18.2s
667:	learn: 0.0016139	total: 2m 26s	remaining: 17.9s
668:	learn: 0.0016139	total: 2m 26s	remaining: 17.7s
669:	learn: 0.0016139	total: 2m 26s	remaining: 17.5s
670:	learn: 0.0016139	total: 2m 27s	remaining: 17.3s
671:	learn: 0.0016139	total: 2m 27s	remaining: 17.1s
672:	learn: 0.0016139	total: 2m 27s	remaining: 16.9s
673:	learn: 0.0016139	total: 2m 28s	remaining: 16.7s
674:	learn: 0.0016139	total: 2m 28s	remaining: 16.5s
675:	learn: 0.0016139	total: 2m 28s	remaining: 16.3s
676:	learn: 0.0016139	total: 2m 29s	remaining: 16.1s
677:	learn: 0.0016139	total: 2m 29s	remaining: 15.9s
678:	learn: 0.0016139	total: 2m 29s	remaining: 15.7s
679:	learn: 0.0016139	total: 2m 30s	remaining: 15.4s
680:	learn: 0.0016139	total: 2m 30s	remaining: 15.2s
681:	learn: 0.0016139	total: 2m 30s	remaining: 15s
682:	learn: 0.0016139	total: 2m 31s	remaining: 14.8s
683:	learn: 0.0016139	total: 2m 31s	remaining: 14.6s
684:	learn: 0.0016139	total: 2m 31s	remaining: 14.4s
685:	learn: 0.0016139	total: 2m 32s	remaining: 14.2s
686:	learn: 0.0016139	total: 2m 32s	remaining: 14s
687:	learn: 0.0016139	total: 2m 32s	remaining: 13.8s
688:	learn: 0.0016139	total: 2m 33s	remaining: 13.6s
689:	learn: 0.0016139	total: 2m 33s	remaining: 13.3s
690:	learn: 0.0016139	total: 2m 33s	remaining: 13.1s
691:	learn: 0.0016139	total: 2m 34s	remaining: 12.9s
692:	learn: 0.0016139	total: 2m 34s	remaining: 12.7s
693:	learn: 0.0016139	total: 2m 34s	remaining: 12.5s
694:	learn: 0.0016139	total: 2m 35s	remaining: 12.3s

695:	learn: 0.0016139	total: 2m 35s	remaining: 12.1s
696:	learn: 0.0016139	total: 2m 35s	remaining: 11.8s
697:	learn: 0.0016139	total: 2m 36s	remaining: 11.6s
698:	learn: 0.0016139	total: 2m 36s	remaining: 11.4s
699:	learn: 0.0016139	total: 2m 36s	remaining: 11.2s
700:	learn: 0.0016139	total: 2m 37s	remaining: 11s
701:	learn: 0.0016139	total: 2m 37s	remaining: 10.8s
702:	learn: 0.0016139	total: 2m 38s	remaining: 10.6s
703:	learn: 0.0016139	total: 2m 38s	remaining: 10.3s
704:	learn: 0.0016139	total: 2m 38s	remaining: 10.1s
705:	learn: 0.0016139	total: 2m 39s	remaining: 9.91s
706:	learn: 0.0016139	total: 2m 39s	remaining: 9.69s
707:	learn: 0.0016139	total: 2m 39s	remaining: 9.48s
708:	learn: 0.0016139	total: 2m 40s	remaining: 9.26s
709:	learn: 0.0016139	total: 2m 40s	remaining: 9.05s
710:	learn: 0.0016139	total: 2m 40s	remaining: 8.83s
711:	learn: 0.0016139	total: 2m 41s	remaining: 8.61s
712:	learn: 0.0016139	total: 2m 41s	remaining: 8.39s
713:	learn: 0.0016139	total: 2m 41s	remaining: 8.16s
714:	learn: 0.0016139	total: 2m 42s	remaining: 7.94s
715:	learn: 0.0016139	total: 2m 42s	remaining: 7.72s
716:	learn: 0.0016139	total: 2m 43s	remaining: 7.5s
717:	learn: 0.0016139	total: 2m 43s	remaining: 7.28s
718:	learn: 0.0016139	total: 2m 43s	remaining: 7.06s
719:	learn: 0.0016139	total: 2m 44s	remaining: 6.84s
720:	learn: 0.0016139	total: 2m 44s	remaining: 6.62s
721:	learn: 0.0016139	total: 2m 44s	remaining: 6.39s
722:	learn: 0.0016138	total: 2m 45s	remaining: 6.17s
723:	learn: 0.0016138	total: 2m 45s	remaining: 5.94s
724:	learn: 0.0016139	total: 2m 45s	remaining: 5.72s
725:	learn: 0.0016139	total: 2m 46s	remaining: 5.5s
726:	learn: 0.0016139	total: 2m 46s	remaining: 5.27s
727:	learn: 0.0016138	total: 2m 46s	remaining: 5.04s
728:	learn: 0.0016139	total: 2m 47s	remaining: 4.82s
729:	learn: 0.0016139	total: 2m 47s	remaining: 4.59s
730:	learn: 0.0016139	total: 2m 48s	remaining: 4.37s
731:	learn: 0.0016139	total: 2m 48s	remaining: 4.14s
732:	learn: 0.0016139	total: 2m 48s	remaining: 3.91s
733:	learn: 0.0016139	total: 2m 49s	remaining: 3.68s
734:	learn: 0.0016139	total: 2m 49s	remaining: 3.46s
735:	learn: 0.0016139	total: 2m 49s	remaining: 3.23s
736:	learn: 0.0016138	total: 2m 50s	remaining: 3s
737:	learn: 0.0016138	total: 2m 50s	remaining: 2.77s
738:	learn: 0.0016138	total: 2m 50s	remaining: 2.54s
739:	learn: 0.0016138	total: 2m 51s	remaining: 2.31s
740:	learn: 0.0016138	total: 2m 51s	remaining: 2.08s
741:	learn: 0.0016138	total: 2m 51s	remaining: 1.85s
742:	learn: 0.0016138	total: 2m 52s	remaining: 1.62s

```

743:   learn: 0.0016138      total: 2m 52s   remaining: 1.39s
744:   learn: 0.0016138      total: 2m 52s   remaining: 1.16s
745:   learn: 0.0016138      total: 2m 53s   remaining: 928ms
746:   learn: 0.0016138      total: 2m 53s   remaining: 697ms
747:   learn: 0.0016139      total: 2m 53s   remaining: 465ms
748:   learn: 0.0016139      total: 2m 54s   remaining: 233ms
749:   learn: 0.0016138      total: 2m 54s   remaining: 0us
[LightGBM] [Info] Number of positive: 2835, number of negative: 2835
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.001243 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1345
[LightGBM] [Info] Number of data points in the train set: 5670, number of used
features: 12
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000

```

4.3.4 Model Selection df

```
[52]: model_selection_df.head(6)
```

```
[52]:
```

	Model_name	train_scores	test_score	accuracy_train \
0	Logistic_Regression	0.908995	0.905882	0.908995
1	Support_Vector_Classifiers	0.781305	0.753676	0.781305
2	Random_Forest_Classifiers	1.000000	0.986765	1.000000
3	K-Nearest_Neighbor	0.992416	0.966176	0.992416
4	Radius_Neighbors_Classifiers	0.885185	0.866176	0.885185
5	Adaptive Boosting Classifiers	0.952028	0.936029	0.952028

	accuracy_test	F1_score_train	F1_score_test	ROC_AUC_train	ROC_AUC_test
0	0.905882	0.909283	0.664921	0.966446	0.957317
1	0.753676	0.781382	0.378479	0.871639	0.840044
2	0.986765	1.000000	0.937500	1.000000	0.997764
3	0.966176	0.992460	0.846667	0.999788	0.958999
4	0.866176	0.887468	0.576744	0.951587	0.949813
5	0.936029	0.952941	0.752137	0.992553	0.978601

```
[53]: model_selection_df.tail(6)
```

```
[53]:
```

	Model_name	train_scores \
6	Bagging Classifiers	0.908818
7	Gradient_Boosting_Classifiers	1.000000
8	Cat_Boost_Classifier	1.000000
9	Light_Gradient_Boosting_Machine_Classifiers	1.000000
10	eXtreme_Gradient_Boosting_Classifier	1.000000
11	eXtreme_Gradient_Boosting_Random_Forest_Classi...	0.993474

	test_score	accuracy_train	accuracy_test	F1_score_train	F1_score_test \
--	------------	----------------	---------------	----------------	-----------------

6	0.884559	0.908818	0.884559	0.911000	0.627078
7	0.993382	1.000000	0.993382	1.000000	0.968198
8	0.987500	1.000000	0.987500	1.000000	0.939929
9	0.988235	1.000000	0.988235	1.000000	0.942857
10	0.988235	1.000000	0.988235	1.000000	0.943262
11	0.978676	0.993474	0.978676	0.993489	0.903654

	ROC_AUC_train	ROC_AUC_test
6	0.957181	0.948725
7	1.000000	0.999280
8	1.000000	0.998230
9	1.000000	0.998475
10	1.000000	0.998978
11	0.999886	0.996751

```
[54]: model_selection_df.to_excel("../models/ML_model_select.
      ↪xlsx", sheet_name="ML_model_selection")
```

From the above dataframe we can see that models like : * Random Forest Classifier * Gradient Boost * Cat Boost Classifier * LightGBM * XGBoost * XGBRF Classifier

Are having the least differences between the train and test metrics[F1_scores]

```
[60]: # a variable for final _ML model evaluation
      final_ML_model = {}
```

```
[61]: # function for evaluating the testing accuracies with the given model
def ml_model_test_eval(ml_model):
    """
    Evaluates a trained GridSearchCV model on the test data and plots key_
    ↪performance metrics.

    Parameters:
    - grid_model: trained GridSearchCV model
    - X_test: test feature set
    - y_test: true test labels
    """
    # Extract the best estimator from GridSearchCV
    best_model = ml_model.best_estimator_

    # Predict class labels
    y_pred = ml_model.predict(X_test_processed)

    # Check if predict_proba is available
    if hasattr(ml_model, "predict_proba"):
        y_proba = ml_model.predict_proba(X_test_processed)[: , 1]
    elif hasattr(ml_model, "decision_function"):
        # Some models use decision_function instead
```

```

        decision_scores = ml_model.decision_function(X_test_processed)
        y_proba = (decision_scores - decision_scores.min()) / (decision_scores.
↪max() - decision_scores.min()) # min-max scaling
    else:
        print("This model does not support probability or decision scores. ROC_
↪AUC will not be computed.")
        y_proba = None

    # Compute metrics
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    roc_auc = roc_auc_score(y_test, y_proba) if y_proba is not None else "N/A"
    cm = confusion_matrix(y_test, y_pred)
    report = classification_report(y_test, y_pred)

    final_ML_model[str(ml_model.estimator.__class__.__name__)] = f1

    # Display text results
    print("Best Parameters:\n", ml_model.best_params_)
    print("\nClassification Report:\n", report)
    print("Accuracy:", round(acc, 4))
    print("F1 Score:", round(f1, 4))
    print("ROC AUC Score:", round(roc_auc, 4) if roc_auc != "N/A" else "N/A")

    # Plotting
    fig, axs = plt.subplots(1, 3, figsize=(18, 5))

    # Confusion Matrix
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axs[0])
    axs[0].set_title('Confusion Matrix')
    axs[0].set_xlabel('Predicted')
    axs[0].set_ylabel('Actual')

    # ROC Curve
    if y_proba is not None:
        fpr, tpr, thresholds = roc_curve(y_test, y_proba)
        axs[1].plot(fpr, tpr, color='darkorange', lw=2, label=f"ROC curve (AUC_
↪{roc_auc:.2f})")
        axs[1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
        axs[1].set_title('ROC Curve')
        axs[1].set_xlabel('False Positive Rate')
        axs[1].set_ylabel('True Positive Rate')
        axs[1].legend(loc='lower right')

    # Threshold plot
    # Align shapes properly
    tpr_plot = tpr[:-1] if len(tpr) > len(thresholds) else tpr

```

```

fpr_plot = fpr[:-1] if len(fpr) > len(thresholds) else fpr

axs[2].plot(thresholds, tpr_plot, label='TPR', color='green')
axs[2].plot(thresholds, 1 - fpr_plot, label='1 - FPR', color='red')
axs[2].set_title('Threshold vs TPR & 1 - FPR')
axs[2].set_xlabel('Threshold')
axs[2].set_ylabel('Rate')
axs[2].legend()
else:
    axs[1].axis('off')
    axs[2].axis('off')

plt.tight_layout()
plt.show()

```

[62]: # 1/0

4.3.4.1 Random Forest Classifier

```

[63]: # Random Forest Classifier
random_forest_parameters = {
    'n_estimators': [700, 500, 450],
    'criterion': ['gini'],
    ↪solid
    'max_depth': [None, 10, 30],
    ↪shallow, mid-depth
    'min_samples_split': [4, 8],
    'min_samples_leaf': [2, 5],
    'min_impurity_decrease': [0.01],
    ↪reasonable value
    'max_features': ['sqrt'],
    ↪often close
    'max_leaf_nodes': [50, 100],
    'bootstrap': [True],
    ↪standard
    'ccp_alpha': [0.005],
    'max_samples': [0.7]
}

```

```

[64]: # initializing the random forest gridsearchCV
rf_grid = GridSearchCV(estimator=RandomForestClassifier(),
    ↪param_grid=random_forest_parameters,
    cv=4, n_jobs=-1, scoring="f1_macro", verbose=10)

# fitting the model
rf_grid.fit(X_train_resampled, y_train_resampled)

```

```

# saving the results into df
random_forest_cv_result = pd.DataFrame.from_dict(rf_grid.cv_results_)

# random forest best parameters
rf_best_params = rf_grid.best_params_

# saving the df as a csv for further analysis if needed
random_forest_cv_result.to_csv("../models/ML_trials/random_forest_models/
↳random_forest_cv_results2.csv")

```

Fitting 4 folds for each of 72 candidates, totalling 288 fits

```

[65]: # results
random_forest_cv_result[random_forest_cv_result['rank_test_score']==1]

```

```

[65]:      mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
13      3.484475      0.493664      0.228726      0.098114

      param_bootstrap param_ccp_alpha param_criterion param_max_depth  \
13              True      0.005              gini              None

      param_max_features param_max_leaf_nodes param_max_samples  \
13              sqrt              100              0.7

      param_min_impurity_decrease param_min_samples_leaf param_min_samples_split  \
13              0.01              2              4

      param_n_estimators              params  \
13              500  {'bootstrap': True, 'ccp_alpha': 0.005, 'crite...

      split0_test_score  split1_test_score  split2_test_score  \
13              0.96756              0.959794              0.961889

      split3_test_score  mean_test_score  std_test_score  rank_test_score
13              0.961888              0.962783              0.002888              1

```

```

[66]: # the best results for random forest
rf_best_params.values()

```

```

[66]: dict_values([True, 0.005, 'gini', None, 'sqrt', 100, 0.7, 0.01, 2, 4, 500])

```

```

[67]: # comparing the results
ml_model_test_eval(rf_grid)

```

Best Parameters:

```

{'bootstrap': True, 'ccp_alpha': 0.005, 'criterion': 'gini', 'max_depth': None,
'max_features': 'sqrt', 'max_leaf_nodes': 100, 'max_samples': 0.7,
'min_impurity_decrease': 0.01, 'min_samples_leaf': 2, 'min_samples_split': 4,

```

```
'n_estimators': 500}
```

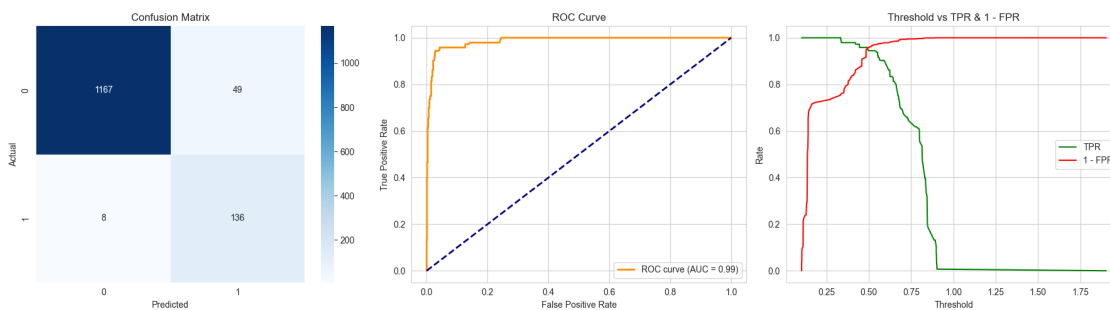
Classification Report:

	precision	recall	f1-score	support
0	0.99	0.96	0.98	1216
1	0.74	0.94	0.83	144
accuracy			0.96	1360
macro avg	0.86	0.95	0.90	1360
weighted avg	0.97	0.96	0.96	1360

Accuracy: 0.9581

F1 Score: 0.8267

ROC AUC Score: 0.986



4.3.4.2 Gradient Boosting Classifier

```
[70]: # passing all the parameters for the gradient boosting classifiers
gradBC_parameters = {

    'n_estimators': [400, 600, 800, 900],          # Adjusted range with ↵
    ↪good balance

    'learning_rate': [0.01, 0.05, 0.1],            # Added lower LR for ↵
    ↪finer updates

    'max_depth': [4, 6, 8],                          # Better range to prevent ↵
    ↪overfitting

    'min_samples_split': [2, 5, 8],                  # Balanced split ↵
    ↪sensitivity

    'min_samples_leaf': [1, 3, 5],                   # Helps control ↵
    ↪overfitting on small branches

    'subsample': [0.7, 0.85, 1.0],                  # Added more variation ↵
    ↪for bagging

    'max_features': ['sqrt', 'log2']                 # Removed None to promote ↵
    ↪feature randomness
}
```



```
}
```

```
[71]: # initializing the Gradient Boosting Classifier with gridsearchCV
gbc_grid = GridSearchCV(estimator=GradientBoostingClassifier(),
    ↪ param_grid=gradBC_parameters,
                        cv=4, n_jobs=-1 , scoring="f1_macro", verbose=10)

# fitting the model
gbc_grid.fit(X_train_resampled, y_train_resampled)

# saving the results into df
gbc_cv_result = pd.DataFrame.from_dict(gbc_grid.cv_results_)

# random forest best parameters
gbc_params = gbc_grid.best_params_

# saving the df as a csv for further analysis if needed
gbc_cv_result.to_csv("../models/ML_trials/gradientBoosting_model/
    ↪ gradient_boost_cv_results2.csv")
```

Fitting 4 folds for each of 1944 candidates, totalling 7776 fits

```
[72]: # results for the 1st rank model
gbc_cv_result[gbc_cv_result['rank_test_score']==1]
```

```
[72]:      mean_fit_time  std_fit_time  mean_score_time  std_score_time \
880      5.864138      0.24388      0.061189      0.007039

      param_learning_rate param_max_depth param_max_features \
880              0.05              6              sqrt

      param_min_samples_leaf param_min_samples_split param_n_estimators \
880              1              5              600

      param_subsample              params \
880      0.85  {'learning_rate': 0.05, 'max_depth': 6, 'max_f...

      split0_test_score  split1_test_score  split2_test_score \
880      0.988715      0.996474      0.997177

      split3_test_score  mean_test_score  std_test_score  rank_test_score
880      0.99506      0.994356      0.003345      1
```

```
[73]: print(gbc_params.values())
```

```
dict_values([0.05, 6, 'sqrt', 1, 5, 600, 0.85])
```

```
[74]: # Print the confusion matrix for all the above models
```

```
ml_model_test_eval(gbc_grid)
```

Best Parameters:

```
{'learning_rate': 0.05, 'max_depth': 6, 'max_features': 'sqrt',
'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 600, 'subsample':
0.85}
```

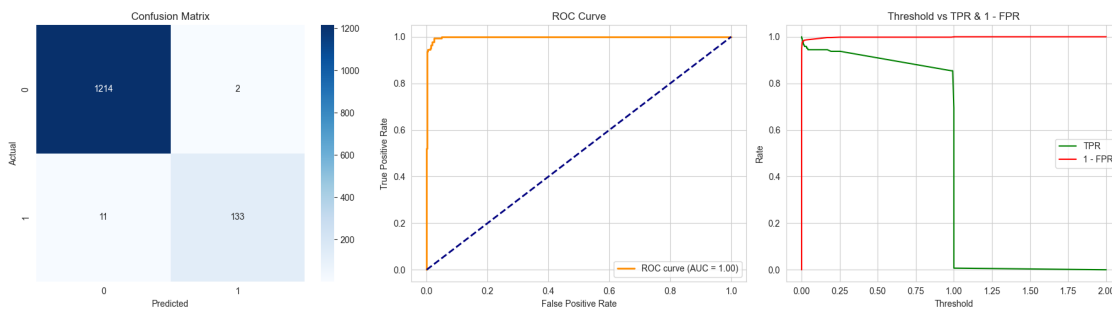
Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	0.99	1216
1	0.99	0.92	0.95	144
accuracy			0.99	1360
macro avg	0.99	0.96	0.97	1360
weighted avg	0.99	0.99	0.99	1360

Accuracy: 0.9904

F1 Score: 0.9534

ROC AUC Score: 0.9984



4.3.4.3 CatBoost Classifier

```
[75]: # parameters for cat boosting classifier
```

```
# Define base grid components
```

```
catBC_parameters = [ {
    'iterations': [300, 500],
    'learning_rate': [0.01, 0.05],
    'depth': [6, 8],
    'bootstrap_type': ['Bayesian'],
    'bagging_temperature': [0.2, 1, 2] # Only valid with Bayesian
},
{
```

```

        'iterations': [300, 500],
        'learning_rate': [0.01, 0.05],
        'depth': [6, 8],
        'bootstrap_type': ['Bernoulli'],
        'subsample': [0.6, 0.8, 1.0]           # Only valid with Bernoulli
    }
]

```

```

[76]: # initializing the Cat Boosting Classifier with gridsearchCV
catBC_grid = GridSearchCV(estimator=CatBoostClassifier(),
    ↪ param_grid=catBC_parameters,
                           cv=4, n_jobs=-1 , scoring="f1_macro", verbose=10)

# fitting the model
catBC_grid.fit(X_train_resampled, y_train_resampled)

# saving the results into df
catBC_cv_result = pd.DataFrame.from_dict(catBC_grid.cv_results_)

# random forest best parameters
catBC_params = catBC_grid.best_params_

# saving the df as a csv for further analysis if needed
catBC_cv_result.to_csv("../models/ML_trials/catBoost_model/
    ↪ Cat_boost_cv_results2.csv")

```

Fitting 4 folds for each of 48 candidates, totalling 192 fits

0:	learn: 0.5924776	total: 32.3ms	remaining: 16.1s
1:	learn: 0.4992951	total: 72.9ms	remaining: 18.2s
2:	learn: 0.4239790	total: 124ms	remaining: 20.6s
3:	learn: 0.3593198	total: 145ms	remaining: 17.9s
4:	learn: 0.3097294	total: 161ms	remaining: 16s
5:	learn: 0.2680175	total: 177ms	remaining: 14.6s
6:	learn: 0.2277468	total: 193ms	remaining: 13.6s
7:	learn: 0.1991604	total: 212ms	remaining: 13s
8:	learn: 0.1752255	total: 227ms	remaining: 12.4s
9:	learn: 0.1541306	total: 240ms	remaining: 11.8s
10:	learn: 0.1352651	total: 256ms	remaining: 11.4s
11:	learn: 0.1227542	total: 268ms	remaining: 10.9s
12:	learn: 0.1118135	total: 277ms	remaining: 10.4s
13:	learn: 0.1026632	total: 286ms	remaining: 9.94s
14:	learn: 0.0932239	total: 295ms	remaining: 9.55s
15:	learn: 0.0853072	total: 305ms	remaining: 9.22s
16:	learn: 0.0788559	total: 314ms	remaining: 8.93s
17:	learn: 0.0740947	total: 330ms	remaining: 8.84s
18:	learn: 0.0703355	total: 345ms	remaining: 8.72s
19:	learn: 0.0655898	total: 358ms	remaining: 8.6s

20:	learn: 0.0614205	total: 371ms	remaining: 8.47s
21:	learn: 0.0583806	total: 383ms	remaining: 8.31s
22:	learn: 0.0551378	total: 395ms	remaining: 8.18s
23:	learn: 0.0527116	total: 405ms	remaining: 8.03s
24:	learn: 0.0502985	total: 415ms	remaining: 7.89s
25:	learn: 0.0480537	total: 425ms	remaining: 7.75s
26:	learn: 0.0463753	total: 436ms	remaining: 7.63s
27:	learn: 0.0447313	total: 445ms	remaining: 7.51s
28:	learn: 0.0433654	total: 455ms	remaining: 7.39s
29:	learn: 0.0419742	total: 466ms	remaining: 7.3s
30:	learn: 0.0410697	total: 477ms	remaining: 7.21s
31:	learn: 0.0398721	total: 489ms	remaining: 7.15s
32:	learn: 0.0388634	total: 502ms	remaining: 7.1s
33:	learn: 0.0379530	total: 515ms	remaining: 7.05s
34:	learn: 0.0373086	total: 529ms	remaining: 7.03s
35:	learn: 0.0368524	total: 542ms	remaining: 6.99s
36:	learn: 0.0359842	total: 554ms	remaining: 6.93s
37:	learn: 0.0352432	total: 566ms	remaining: 6.88s
38:	learn: 0.0343683	total: 580ms	remaining: 6.86s
39:	learn: 0.0335449	total: 594ms	remaining: 6.83s
40:	learn: 0.0330368	total: 604ms	remaining: 6.76s
41:	learn: 0.0325738	total: 615ms	remaining: 6.7s
42:	learn: 0.0318968	total: 622ms	remaining: 6.61s
43:	learn: 0.0315130	total: 630ms	remaining: 6.53s
44:	learn: 0.0311055	total: 637ms	remaining: 6.44s
45:	learn: 0.0305903	total: 645ms	remaining: 6.37s
46:	learn: 0.0300710	total: 654ms	remaining: 6.3s
47:	learn: 0.0296350	total: 664ms	remaining: 6.25s
48:	learn: 0.0293816	total: 673ms	remaining: 6.19s
49:	learn: 0.0287397	total: 680ms	remaining: 6.12s
50:	learn: 0.0280685	total: 689ms	remaining: 6.06s
51:	learn: 0.0273734	total: 699ms	remaining: 6.02s
52:	learn: 0.0268664	total: 706ms	remaining: 5.96s
53:	learn: 0.0264327	total: 714ms	remaining: 5.9s
54:	learn: 0.0260860	total: 723ms	remaining: 5.85s
55:	learn: 0.0254145	total: 731ms	remaining: 5.8s
56:	learn: 0.0249709	total: 740ms	remaining: 5.75s
57:	learn: 0.0244294	total: 748ms	remaining: 5.7s
58:	learn: 0.0242283	total: 757ms	remaining: 5.65s
59:	learn: 0.0238841	total: 764ms	remaining: 5.6s
60:	learn: 0.0235297	total: 773ms	remaining: 5.56s
61:	learn: 0.0232153	total: 780ms	remaining: 5.51s
62:	learn: 0.0229829	total: 789ms	remaining: 5.47s
63:	learn: 0.0224896	total: 798ms	remaining: 5.43s
64:	learn: 0.0222426	total: 806ms	remaining: 5.39s
65:	learn: 0.0219691	total: 815ms	remaining: 5.36s
66:	learn: 0.0216870	total: 824ms	remaining: 5.33s
67:	learn: 0.0213810	total: 831ms	remaining: 5.28s

68:	learn: 0.0210809	total: 841ms	remaining: 5.25s
69:	learn: 0.0208627	total: 848ms	remaining: 5.21s
70:	learn: 0.0205998	total: 856ms	remaining: 5.17s
71:	learn: 0.0202835	total: 866ms	remaining: 5.15s
72:	learn: 0.0200451	total: 875ms	remaining: 5.12s
73:	learn: 0.0198654	total: 883ms	remaining: 5.08s
74:	learn: 0.0196404	total: 890ms	remaining: 5.04s
75:	learn: 0.0194499	total: 898ms	remaining: 5.01s
76:	learn: 0.0192029	total: 905ms	remaining: 4.97s
77:	learn: 0.0188834	total: 912ms	remaining: 4.93s
78:	learn: 0.0186646	total: 920ms	remaining: 4.9s
79:	learn: 0.0185354	total: 930ms	remaining: 4.88s
80:	learn: 0.0184229	total: 937ms	remaining: 4.85s
81:	learn: 0.0181919	total: 948ms	remaining: 4.83s
82:	learn: 0.0179832	total: 956ms	remaining: 4.8s
83:	learn: 0.0177372	total: 964ms	remaining: 4.77s
84:	learn: 0.0176191	total: 972ms	remaining: 4.74s
85:	learn: 0.0174107	total: 981ms	remaining: 4.72s
86:	learn: 0.0172226	total: 990ms	remaining: 4.7s
87:	learn: 0.0170325	total: 999ms	remaining: 4.67s
88:	learn: 0.0168735	total: 1.01s	remaining: 4.65s
89:	learn: 0.0166676	total: 1.02s	remaining: 4.63s
90:	learn: 0.0165259	total: 1.02s	remaining: 4.61s
91:	learn: 0.0164231	total: 1.03s	remaining: 4.58s
92:	learn: 0.0162845	total: 1.04s	remaining: 4.56s
93:	learn: 0.0160726	total: 1.05s	remaining: 4.54s
94:	learn: 0.0159177	total: 1.06s	remaining: 4.54s
95:	learn: 0.0158503	total: 1.07s	remaining: 4.52s
96:	learn: 0.0157090	total: 1.08s	remaining: 4.51s
97:	learn: 0.0155613	total: 1.09s	remaining: 4.49s
98:	learn: 0.0154628	total: 1.1s	remaining: 4.47s
99:	learn: 0.0153434	total: 1.11s	remaining: 4.45s
100:	learn: 0.0151890	total: 1.12s	remaining: 4.43s
101:	learn: 0.0150518	total: 1.13s	remaining: 4.41s
102:	learn: 0.0149453	total: 1.14s	remaining: 4.39s
103:	learn: 0.0148837	total: 1.15s	remaining: 4.36s
104:	learn: 0.0147855	total: 1.15s	remaining: 4.34s
105:	learn: 0.0146618	total: 1.16s	remaining: 4.32s
106:	learn: 0.0145166	total: 1.17s	remaining: 4.3s
107:	learn: 0.0144174	total: 1.18s	remaining: 4.28s
108:	learn: 0.0142697	total: 1.19s	remaining: 4.26s
109:	learn: 0.0141419	total: 1.19s	remaining: 4.23s
110:	learn: 0.0140203	total: 1.2s	remaining: 4.21s
111:	learn: 0.0138687	total: 1.21s	remaining: 4.19s
112:	learn: 0.0137533	total: 1.22s	remaining: 4.17s
113:	learn: 0.0136454	total: 1.22s	remaining: 4.14s
114:	learn: 0.0135451	total: 1.23s	remaining: 4.12s
115:	learn: 0.0135105	total: 1.24s	remaining: 4.1s

116:	learn: 0.0133900	total: 1.25s	remaining: 4.08s
117:	learn: 0.0132822	total: 1.25s	remaining: 4.05s
118:	learn: 0.0132122	total: 1.26s	remaining: 4.03s
119:	learn: 0.0130854	total: 1.27s	remaining: 4.01s
120:	learn: 0.0130071	total: 1.27s	remaining: 3.99s
121:	learn: 0.0129136	total: 1.28s	remaining: 3.97s
122:	learn: 0.0127949	total: 1.29s	remaining: 3.95s
123:	learn: 0.0126911	total: 1.3s	remaining: 3.93s
124:	learn: 0.0126139	total: 1.3s	remaining: 3.91s
125:	learn: 0.0124723	total: 1.31s	remaining: 3.89s
126:	learn: 0.0123961	total: 1.32s	remaining: 3.87s
127:	learn: 0.0122302	total: 1.33s	remaining: 3.85s
128:	learn: 0.0121267	total: 1.33s	remaining: 3.84s
129:	learn: 0.0120535	total: 1.34s	remaining: 3.82s
130:	learn: 0.0119849	total: 1.35s	remaining: 3.79s
131:	learn: 0.0119192	total: 1.35s	remaining: 3.78s
132:	learn: 0.0117933	total: 1.36s	remaining: 3.76s
133:	learn: 0.0117165	total: 1.37s	remaining: 3.74s
134:	learn: 0.0116459	total: 1.38s	remaining: 3.72s
135:	learn: 0.0115665	total: 1.39s	remaining: 3.71s
136:	learn: 0.0115064	total: 1.39s	remaining: 3.69s
137:	learn: 0.0113627	total: 1.4s	remaining: 3.68s
138:	learn: 0.0112668	total: 1.41s	remaining: 3.66s
139:	learn: 0.0112148	total: 1.42s	remaining: 3.64s
140:	learn: 0.0111274	total: 1.42s	remaining: 3.62s
141:	learn: 0.0110274	total: 1.43s	remaining: 3.6s
142:	learn: 0.0109525	total: 1.44s	remaining: 3.58s
143:	learn: 0.0108471	total: 1.44s	remaining: 3.56s
144:	learn: 0.0107912	total: 1.45s	remaining: 3.55s
145:	learn: 0.0107170	total: 1.46s	remaining: 3.53s
146:	learn: 0.0106340	total: 1.46s	remaining: 3.52s
147:	learn: 0.0105628	total: 1.47s	remaining: 3.5s
148:	learn: 0.0104438	total: 1.48s	remaining: 3.48s
149:	learn: 0.0103439	total: 1.49s	remaining: 3.47s
150:	learn: 0.0102536	total: 1.49s	remaining: 3.45s
151:	learn: 0.0101726	total: 1.5s	remaining: 3.44s
152:	learn: 0.0100776	total: 1.51s	remaining: 3.42s
153:	learn: 0.0100159	total: 1.51s	remaining: 3.4s
154:	learn: 0.0099578	total: 1.52s	remaining: 3.39s
155:	learn: 0.0098968	total: 1.53s	remaining: 3.38s
156:	learn: 0.0098366	total: 1.54s	remaining: 3.36s
157:	learn: 0.0097898	total: 1.54s	remaining: 3.34s
158:	learn: 0.0097138	total: 1.55s	remaining: 3.33s
159:	learn: 0.0096495	total: 1.56s	remaining: 3.31s
160:	learn: 0.0096003	total: 1.56s	remaining: 3.29s
161:	learn: 0.0095040	total: 1.57s	remaining: 3.28s
162:	learn: 0.0094234	total: 1.58s	remaining: 3.26s
163:	learn: 0.0093661	total: 1.59s	remaining: 3.25s

164:	learn: 0.0092999	total: 1.59s	remaining: 3.24s
165:	learn: 0.0092352	total: 1.6s	remaining: 3.22s
166:	learn: 0.0091706	total: 1.61s	remaining: 3.2s
167:	learn: 0.0091087	total: 1.61s	remaining: 3.19s
168:	learn: 0.0090403	total: 1.62s	remaining: 3.17s
169:	learn: 0.0089639	total: 1.63s	remaining: 3.16s
170:	learn: 0.0088916	total: 1.63s	remaining: 3.14s
171:	learn: 0.0088574	total: 1.64s	remaining: 3.13s
172:	learn: 0.0087921	total: 1.64s	remaining: 3.11s
173:	learn: 0.0087198	total: 1.65s	remaining: 3.09s
174:	learn: 0.0086586	total: 1.66s	remaining: 3.08s
175:	learn: 0.0085642	total: 1.66s	remaining: 3.06s
176:	learn: 0.0084789	total: 1.67s	remaining: 3.04s
177:	learn: 0.0084034	total: 1.67s	remaining: 3.03s
178:	learn: 0.0083804	total: 1.68s	remaining: 3.01s
179:	learn: 0.0083237	total: 1.69s	remaining: 3s
180:	learn: 0.0082916	total: 1.7s	remaining: 2.99s
181:	learn: 0.0082232	total: 1.7s	remaining: 2.98s
182:	learn: 0.0081490	total: 1.71s	remaining: 2.96s
183:	learn: 0.0080647	total: 1.72s	remaining: 2.95s
184:	learn: 0.0080043	total: 1.73s	remaining: 2.94s
185:	learn: 0.0079591	total: 1.73s	remaining: 2.92s
186:	learn: 0.0079115	total: 1.74s	remaining: 2.91s
187:	learn: 0.0078741	total: 1.75s	remaining: 2.9s
188:	learn: 0.0078317	total: 1.75s	remaining: 2.89s
189:	learn: 0.0077967	total: 1.76s	remaining: 2.87s
190:	learn: 0.0077603	total: 1.77s	remaining: 2.86s
191:	learn: 0.0077250	total: 1.77s	remaining: 2.85s
192:	learn: 0.0076626	total: 1.78s	remaining: 2.83s
193:	learn: 0.0076047	total: 1.79s	remaining: 2.82s
194:	learn: 0.0075519	total: 1.8s	remaining: 2.81s
195:	learn: 0.0075005	total: 1.8s	remaining: 2.8s
196:	learn: 0.0074611	total: 1.81s	remaining: 2.78s
197:	learn: 0.0074283	total: 1.82s	remaining: 2.77s
198:	learn: 0.0073598	total: 1.82s	remaining: 2.76s
199:	learn: 0.0073071	total: 1.83s	remaining: 2.74s
200:	learn: 0.0072584	total: 1.83s	remaining: 2.73s
201:	learn: 0.0072151	total: 1.84s	remaining: 2.71s
202:	learn: 0.0071775	total: 1.85s	remaining: 2.7s
203:	learn: 0.0071399	total: 1.85s	remaining: 2.69s
204:	learn: 0.0070946	total: 1.86s	remaining: 2.68s
205:	learn: 0.0070469	total: 1.87s	remaining: 2.66s
206:	learn: 0.0070218	total: 1.87s	remaining: 2.65s
207:	learn: 0.0069726	total: 1.88s	remaining: 2.64s
208:	learn: 0.0069311	total: 1.88s	remaining: 2.62s
209:	learn: 0.0068807	total: 1.89s	remaining: 2.61s
210:	learn: 0.0068265	total: 1.9s	remaining: 2.6s
211:	learn: 0.0067955	total: 1.9s	remaining: 2.59s

212:	learn: 0.0067700	total: 1.91s	remaining: 2.58s
213:	learn: 0.0067252	total: 1.92s	remaining: 2.56s
214:	learn: 0.0066913	total: 1.93s	remaining: 2.55s
215:	learn: 0.0066495	total: 1.94s	remaining: 2.54s
216:	learn: 0.0066199	total: 1.94s	remaining: 2.53s
217:	learn: 0.0065734	total: 1.95s	remaining: 2.53s
218:	learn: 0.0065142	total: 1.96s	remaining: 2.52s
219:	learn: 0.0064897	total: 1.97s	remaining: 2.51s
220:	learn: 0.0064565	total: 1.98s	remaining: 2.5s
221:	learn: 0.0064190	total: 1.99s	remaining: 2.49s
222:	learn: 0.0063868	total: 2s	remaining: 2.48s
223:	learn: 0.0063516	total: 2.01s	remaining: 2.47s
224:	learn: 0.0063270	total: 2.01s	remaining: 2.46s
225:	learn: 0.0062981	total: 2.02s	remaining: 2.45s
226:	learn: 0.0062522	total: 2.03s	remaining: 2.44s
227:	learn: 0.0061992	total: 2.03s	remaining: 2.43s
228:	learn: 0.0061701	total: 2.04s	remaining: 2.42s
229:	learn: 0.0061362	total: 2.05s	remaining: 2.4s
230:	learn: 0.0061099	total: 2.05s	remaining: 2.39s
231:	learn: 0.0060652	total: 2.06s	remaining: 2.38s
232:	learn: 0.0060323	total: 2.07s	remaining: 2.37s
233:	learn: 0.0059898	total: 2.07s	remaining: 2.36s
234:	learn: 0.0059537	total: 2.08s	remaining: 2.34s
235:	learn: 0.0059321	total: 2.08s	remaining: 2.33s
236:	learn: 0.0059023	total: 2.09s	remaining: 2.32s
237:	learn: 0.0058622	total: 2.1s	remaining: 2.31s
238:	learn: 0.0058422	total: 2.11s	remaining: 2.3s
239:	learn: 0.0057961	total: 2.11s	remaining: 2.29s
240:	learn: 0.0057580	total: 2.12s	remaining: 2.28s
241:	learn: 0.0057216	total: 2.13s	remaining: 2.27s
242:	learn: 0.0056899	total: 2.13s	remaining: 2.26s
243:	learn: 0.0056609	total: 2.14s	remaining: 2.25s
244:	learn: 0.0056365	total: 2.15s	remaining: 2.24s
245:	learn: 0.0056049	total: 2.15s	remaining: 2.23s
246:	learn: 0.0055940	total: 2.16s	remaining: 2.21s
247:	learn: 0.0055631	total: 2.17s	remaining: 2.2s
248:	learn: 0.0055364	total: 2.18s	remaining: 2.19s
249:	learn: 0.0054871	total: 2.18s	remaining: 2.18s
250:	learn: 0.0054484	total: 2.19s	remaining: 2.17s
251:	learn: 0.0054212	total: 2.19s	remaining: 2.16s
252:	learn: 0.0053866	total: 2.2s	remaining: 2.15s
253:	learn: 0.0053564	total: 2.21s	remaining: 2.14s
254:	learn: 0.0053180	total: 2.21s	remaining: 2.13s
255:	learn: 0.0052825	total: 2.22s	remaining: 2.12s
256:	learn: 0.0052691	total: 2.23s	remaining: 2.11s
257:	learn: 0.0052457	total: 2.23s	remaining: 2.1s
258:	learn: 0.0052194	total: 2.24s	remaining: 2.08s
259:	learn: 0.0051902	total: 2.25s	remaining: 2.07s

260:	learn: 0.0051511	total: 2.25s	remaining: 2.06s
261:	learn: 0.0051197	total: 2.26s	remaining: 2.05s
262:	learn: 0.0050927	total: 2.27s	remaining: 2.04s
263:	learn: 0.0050694	total: 2.27s	remaining: 2.03s
264:	learn: 0.0050398	total: 2.28s	remaining: 2.02s
265:	learn: 0.0050212	total: 2.29s	remaining: 2.01s
266:	learn: 0.0049984	total: 2.29s	remaining: 2s
267:	learn: 0.0049692	total: 2.3s	remaining: 1.99s
268:	learn: 0.0049440	total: 2.31s	remaining: 1.98s
269:	learn: 0.0049134	total: 2.31s	remaining: 1.97s
270:	learn: 0.0048891	total: 2.32s	remaining: 1.96s
271:	learn: 0.0048692	total: 2.33s	remaining: 1.95s
272:	learn: 0.0048366	total: 2.33s	remaining: 1.94s
273:	learn: 0.0048058	total: 2.34s	remaining: 1.93s
274:	learn: 0.0047789	total: 2.35s	remaining: 1.92s
275:	learn: 0.0047550	total: 2.35s	remaining: 1.91s
276:	learn: 0.0047409	total: 2.36s	remaining: 1.9s
277:	learn: 0.0047169	total: 2.37s	remaining: 1.89s
278:	learn: 0.0047065	total: 2.38s	remaining: 1.88s
279:	learn: 0.0046886	total: 2.38s	remaining: 1.87s
280:	learn: 0.0046744	total: 2.39s	remaining: 1.86s
281:	learn: 0.0046570	total: 2.4s	remaining: 1.85s
282:	learn: 0.0046350	total: 2.4s	remaining: 1.84s
283:	learn: 0.0046153	total: 2.41s	remaining: 1.84s
284:	learn: 0.0045935	total: 2.42s	remaining: 1.83s
285:	learn: 0.0045657	total: 2.43s	remaining: 1.82s
286:	learn: 0.0045366	total: 2.44s	remaining: 1.81s
287:	learn: 0.0045129	total: 2.45s	remaining: 1.8s
288:	learn: 0.0044876	total: 2.46s	remaining: 1.79s
289:	learn: 0.0044578	total: 2.47s	remaining: 1.79s
290:	learn: 0.0044408	total: 2.48s	remaining: 1.78s
291:	learn: 0.0044233	total: 2.48s	remaining: 1.77s
292:	learn: 0.0044001	total: 2.49s	remaining: 1.76s
293:	learn: 0.0043848	total: 2.5s	remaining: 1.75s
294:	learn: 0.0043682	total: 2.51s	remaining: 1.74s
295:	learn: 0.0043485	total: 2.52s	remaining: 1.73s
296:	learn: 0.0043305	total: 2.52s	remaining: 1.72s
297:	learn: 0.0043110	total: 2.53s	remaining: 1.72s
298:	learn: 0.0042877	total: 2.54s	remaining: 1.71s
299:	learn: 0.0042735	total: 2.55s	remaining: 1.7s
300:	learn: 0.0042492	total: 2.56s	remaining: 1.69s
301:	learn: 0.0042322	total: 2.56s	remaining: 1.68s
302:	learn: 0.0042229	total: 2.57s	remaining: 1.67s
303:	learn: 0.0042037	total: 2.58s	remaining: 1.66s
304:	learn: 0.0041858	total: 2.59s	remaining: 1.65s
305:	learn: 0.0041675	total: 2.6s	remaining: 1.65s
306:	learn: 0.0041530	total: 2.6s	remaining: 1.64s
307:	learn: 0.0041363	total: 2.61s	remaining: 1.63s

308:	learn: 0.0041116	total: 2.62s	remaining: 1.62s
309:	learn: 0.0040952	total: 2.63s	remaining: 1.61s
310:	learn: 0.0040747	total: 2.63s	remaining: 1.6s
311:	learn: 0.0040576	total: 2.64s	remaining: 1.59s
312:	learn: 0.0040445	total: 2.65s	remaining: 1.58s
313:	learn: 0.0040243	total: 2.65s	remaining: 1.57s
314:	learn: 0.0040068	total: 2.66s	remaining: 1.56s
315:	learn: 0.0039919	total: 2.67s	remaining: 1.55s
316:	learn: 0.0039761	total: 2.67s	remaining: 1.54s
317:	learn: 0.0039600	total: 2.68s	remaining: 1.53s
318:	learn: 0.0039406	total: 2.69s	remaining: 1.53s
319:	learn: 0.0039256	total: 2.7s	remaining: 1.52s
320:	learn: 0.0039048	total: 2.71s	remaining: 1.51s
321:	learn: 0.0038843	total: 2.71s	remaining: 1.5s
322:	learn: 0.0038671	total: 2.72s	remaining: 1.49s
323:	learn: 0.0038516	total: 2.73s	remaining: 1.48s
324:	learn: 0.0038273	total: 2.73s	remaining: 1.47s
325:	learn: 0.0038143	total: 2.74s	remaining: 1.46s
326:	learn: 0.0037902	total: 2.75s	remaining: 1.46s
327:	learn: 0.0037783	total: 2.76s	remaining: 1.45s
328:	learn: 0.0037629	total: 2.77s	remaining: 1.44s
329:	learn: 0.0037415	total: 2.78s	remaining: 1.43s
330:	learn: 0.0037220	total: 2.79s	remaining: 1.42s
331:	learn: 0.0037091	total: 2.79s	remaining: 1.41s
332:	learn: 0.0036917	total: 2.8s	remaining: 1.41s
333:	learn: 0.0036756	total: 2.81s	remaining: 1.4s
334:	learn: 0.0036600	total: 2.82s	remaining: 1.39s
335:	learn: 0.0036473	total: 2.82s	remaining: 1.38s
336:	learn: 0.0036388	total: 2.83s	remaining: 1.37s
337:	learn: 0.0036234	total: 2.84s	remaining: 1.36s
338:	learn: 0.0036034	total: 2.85s	remaining: 1.35s
339:	learn: 0.0035847	total: 2.85s	remaining: 1.34s
340:	learn: 0.0035705	total: 2.86s	remaining: 1.33s
341:	learn: 0.0035571	total: 2.87s	remaining: 1.32s
342:	learn: 0.0035459	total: 2.88s	remaining: 1.32s
343:	learn: 0.0035375	total: 2.88s	remaining: 1.31s
344:	learn: 0.0035273	total: 2.9s	remaining: 1.3s
345:	learn: 0.0035108	total: 2.9s	remaining: 1.29s
346:	learn: 0.0034907	total: 2.91s	remaining: 1.28s
347:	learn: 0.0034772	total: 2.92s	remaining: 1.28s
348:	learn: 0.0034681	total: 2.93s	remaining: 1.27s
349:	learn: 0.0034574	total: 2.94s	remaining: 1.26s
350:	learn: 0.0034431	total: 2.95s	remaining: 1.25s
351:	learn: 0.0034317	total: 2.96s	remaining: 1.24s
352:	learn: 0.0034182	total: 2.96s	remaining: 1.24s
353:	learn: 0.0034182	total: 2.97s	remaining: 1.23s
354:	learn: 0.0034022	total: 2.98s	remaining: 1.22s
355:	learn: 0.0033879	total: 2.99s	remaining: 1.21s

356:	learn: 0.0033754	total: 3s	remaining: 1.2s
357:	learn: 0.0033501	total: 3.01s	remaining: 1.19s
358:	learn: 0.0033367	total: 3.02s	remaining: 1.19s
359:	learn: 0.0033244	total: 3.02s	remaining: 1.18s
360:	learn: 0.0033088	total: 3.03s	remaining: 1.17s
361:	learn: 0.0033087	total: 3.04s	remaining: 1.16s
362:	learn: 0.0033087	total: 3.05s	remaining: 1.15s
363:	learn: 0.0032907	total: 3.06s	remaining: 1.14s
364:	learn: 0.0032813	total: 3.07s	remaining: 1.13s
365:	learn: 0.0032709	total: 3.08s	remaining: 1.13s
366:	learn: 0.0032555	total: 3.08s	remaining: 1.12s
367:	learn: 0.0032474	total: 3.09s	remaining: 1.11s
368:	learn: 0.0032314	total: 3.1s	remaining: 1.1s
369:	learn: 0.0032192	total: 3.11s	remaining: 1.09s
370:	learn: 0.0032058	total: 3.12s	remaining: 1.08s
371:	learn: 0.0032058	total: 3.12s	remaining: 1.07s
372:	learn: 0.0031970	total: 3.13s	remaining: 1.07s
373:	learn: 0.0031823	total: 3.14s	remaining: 1.06s
374:	learn: 0.0031738	total: 3.15s	remaining: 1.05s
375:	learn: 0.0031627	total: 3.15s	remaining: 1.04s
376:	learn: 0.0031514	total: 3.16s	remaining: 1.03s
377:	learn: 0.0031438	total: 3.17s	remaining: 1.02s
378:	learn: 0.0031319	total: 3.17s	remaining: 1.01s
379:	learn: 0.0031235	total: 3.18s	remaining: 1s
380:	learn: 0.0031084	total: 3.19s	remaining: 996ms
381:	learn: 0.0030999	total: 3.19s	remaining: 987ms
382:	learn: 0.0030902	total: 3.2s	remaining: 978ms
383:	learn: 0.0030902	total: 3.21s	remaining: 969ms
384:	learn: 0.0030745	total: 3.21s	remaining: 960ms
385:	learn: 0.0030638	total: 3.22s	remaining: 951ms
386:	learn: 0.0030506	total: 3.23s	remaining: 943ms
387:	learn: 0.0030433	total: 3.24s	remaining: 935ms
388:	learn: 0.0030341	total: 3.24s	remaining: 926ms
389:	learn: 0.0030342	total: 3.25s	remaining: 917ms
390:	learn: 0.0030247	total: 3.26s	remaining: 908ms
391:	learn: 0.0030177	total: 3.26s	remaining: 900ms
392:	learn: 0.0030176	total: 3.27s	remaining: 891ms
393:	learn: 0.0030085	total: 3.28s	remaining: 882ms
394:	learn: 0.0029995	total: 3.28s	remaining: 873ms
395:	learn: 0.0029994	total: 3.29s	remaining: 864ms
396:	learn: 0.0029858	total: 3.3s	remaining: 855ms
397:	learn: 0.0029723	total: 3.3s	remaining: 847ms
398:	learn: 0.0029605	total: 3.31s	remaining: 838ms
399:	learn: 0.0029445	total: 3.32s	remaining: 829ms
400:	learn: 0.0029380	total: 3.32s	remaining: 821ms
401:	learn: 0.0029380	total: 3.33s	remaining: 812ms
402:	learn: 0.0029379	total: 3.34s	remaining: 803ms
403:	learn: 0.0029295	total: 3.35s	remaining: 795ms

404:	learn: 0.0029184	total: 3.35s	remaining: 787ms
405:	learn: 0.0029099	total: 3.36s	remaining: 778ms
406:	learn: 0.0029100	total: 3.37s	remaining: 770ms
407:	learn: 0.0028953	total: 3.38s	remaining: 761ms
408:	learn: 0.0028883	total: 3.38s	remaining: 753ms
409:	learn: 0.0028762	total: 3.39s	remaining: 744ms
410:	learn: 0.0028646	total: 3.4s	remaining: 736ms
411:	learn: 0.0028563	total: 3.41s	remaining: 728ms
412:	learn: 0.0028563	total: 3.42s	remaining: 719ms
413:	learn: 0.0028466	total: 3.42s	remaining: 712ms
414:	learn: 0.0028466	total: 3.43s	remaining: 703ms
415:	learn: 0.0028379	total: 3.44s	remaining: 695ms
416:	learn: 0.0028300	total: 3.45s	remaining: 686ms
417:	learn: 0.0028213	total: 3.45s	remaining: 678ms
418:	learn: 0.0028123	total: 3.46s	remaining: 669ms
419:	learn: 0.0028051	total: 3.47s	remaining: 661ms
420:	learn: 0.0027952	total: 3.48s	remaining: 652ms
421:	learn: 0.0027856	total: 3.48s	remaining: 644ms
422:	learn: 0.0027776	total: 3.49s	remaining: 635ms
423:	learn: 0.0027686	total: 3.5s	remaining: 627ms
424:	learn: 0.0027603	total: 3.5s	remaining: 618ms
425:	learn: 0.0027569	total: 3.51s	remaining: 610ms
426:	learn: 0.0027470	total: 3.52s	remaining: 601ms
427:	learn: 0.0027373	total: 3.52s	remaining: 593ms
428:	learn: 0.0027291	total: 3.53s	remaining: 584ms
429:	learn: 0.0027206	total: 3.54s	remaining: 576ms
430:	learn: 0.0027101	total: 3.54s	remaining: 568ms
431:	learn: 0.0027000	total: 3.55s	remaining: 559ms
432:	learn: 0.0026865	total: 3.56s	remaining: 551ms
433:	learn: 0.0026862	total: 3.57s	remaining: 543ms
434:	learn: 0.0026739	total: 3.58s	remaining: 535ms
435:	learn: 0.0026663	total: 3.58s	remaining: 526ms
436:	learn: 0.0026553	total: 3.59s	remaining: 518ms
437:	learn: 0.0026414	total: 3.6s	remaining: 510ms
438:	learn: 0.0026339	total: 3.61s	remaining: 502ms
439:	learn: 0.0026294	total: 3.62s	remaining: 494ms
440:	learn: 0.0026195	total: 3.63s	remaining: 485ms
441:	learn: 0.0026126	total: 3.63s	remaining: 477ms
442:	learn: 0.0026040	total: 3.64s	remaining: 469ms
443:	learn: 0.0026039	total: 3.65s	remaining: 461ms
444:	learn: 0.0025978	total: 3.66s	remaining: 452ms
445:	learn: 0.0025874	total: 3.67s	remaining: 444ms
446:	learn: 0.0025843	total: 3.67s	remaining: 435ms
447:	learn: 0.0025749	total: 3.68s	remaining: 427ms
448:	learn: 0.0025672	total: 3.69s	remaining: 419ms
449:	learn: 0.0025594	total: 3.69s	remaining: 410ms
450:	learn: 0.0025498	total: 3.7s	remaining: 402ms
451:	learn: 0.0025497	total: 3.7s	remaining: 393ms

452:	learn: 0.0025416	total: 3.71s	remaining: 385ms
453:	learn: 0.0025416	total: 3.72s	remaining: 377ms
454:	learn: 0.0025345	total: 3.72s	remaining: 368ms
455:	learn: 0.0025235	total: 3.73s	remaining: 360ms
456:	learn: 0.0025234	total: 3.74s	remaining: 352ms
457:	learn: 0.0025126	total: 3.75s	remaining: 343ms
458:	learn: 0.0025025	total: 3.75s	remaining: 335ms
459:	learn: 0.0025022	total: 3.76s	remaining: 327ms
460:	learn: 0.0024951	total: 3.77s	remaining: 319ms
461:	learn: 0.0024829	total: 3.77s	remaining: 310ms
462:	learn: 0.0024763	total: 3.78s	remaining: 302ms
463:	learn: 0.0024681	total: 3.79s	remaining: 294ms
464:	learn: 0.0024611	total: 3.79s	remaining: 286ms
465:	learn: 0.0024545	total: 3.8s	remaining: 278ms
466:	learn: 0.0024545	total: 3.81s	remaining: 269ms
467:	learn: 0.0024545	total: 3.82s	remaining: 261ms
468:	learn: 0.0024545	total: 3.83s	remaining: 253ms
469:	learn: 0.0024544	total: 3.84s	remaining: 245ms
470:	learn: 0.0024471	total: 3.85s	remaining: 237ms
471:	learn: 0.0024470	total: 3.85s	remaining: 229ms
472:	learn: 0.0024471	total: 3.86s	remaining: 221ms
473:	learn: 0.0024470	total: 3.87s	remaining: 212ms
474:	learn: 0.0024407	total: 3.88s	remaining: 204ms
475:	learn: 0.0024408	total: 3.89s	remaining: 196ms
476:	learn: 0.0024407	total: 3.9s	remaining: 188ms
477:	learn: 0.0024330	total: 3.91s	remaining: 180ms
478:	learn: 0.0024194	total: 3.92s	remaining: 172ms
479:	learn: 0.0024097	total: 3.94s	remaining: 164ms
480:	learn: 0.0024019	total: 3.94s	remaining: 156ms
481:	learn: 0.0023959	total: 3.95s	remaining: 148ms
482:	learn: 0.0023906	total: 3.96s	remaining: 139ms
483:	learn: 0.0023860	total: 3.97s	remaining: 131ms
484:	learn: 0.0023860	total: 3.98s	remaining: 123ms
485:	learn: 0.0023861	total: 3.98s	remaining: 115ms
486:	learn: 0.0023804	total: 3.99s	remaining: 107ms
487:	learn: 0.0023707	total: 4s	remaining: 98.4ms
488:	learn: 0.0023610	total: 4.01s	remaining: 90.2ms
489:	learn: 0.0023525	total: 4.02s	remaining: 82ms
490:	learn: 0.0023486	total: 4.03s	remaining: 73.8ms
491:	learn: 0.0023486	total: 4.03s	remaining: 65.6ms
492:	learn: 0.0023409	total: 4.04s	remaining: 57.4ms
493:	learn: 0.0023363	total: 4.05s	remaining: 49.2ms
494:	learn: 0.0023260	total: 4.06s	remaining: 41ms
495:	learn: 0.0023166	total: 4.07s	remaining: 32.8ms
496:	learn: 0.0023166	total: 4.07s	remaining: 24.6ms
497:	learn: 0.0023108	total: 4.08s	remaining: 16.4ms
498:	learn: 0.0023087	total: 4.09s	remaining: 8.2ms
499:	learn: 0.0023022	total: 4.1s	remaining: 0ms

```
[77]: # best parameters
print(catBC_params.values())
```

```
dict_values([1, 'Bayesian', 8, 500, 0.05])
```

```
[78]: # best result
catBC_cv_result[catBC_cv_result['rank_test_score']==1]
```

```
[78]:      mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
15      35.608853      0.491475      0.035249      0.006866

      param_bagging_temperature  param_bootstrap_type  param_depth  \
15                             1              Bayesian              8

      param_iterations  param_learning_rate  param_subsample  \
15                500                0.05                NaN

                                     params  split0_test_score  \
15  {'bagging_temperature': 1, 'bootstrap_type': '...'      0.990127

      split1_test_score  split2_test_score  split3_test_score  mean_test_score  \
15      0.994358      0.993648      0.995766      0.993475

      std_test_score  rank_test_score
15      0.002078      1
```

```
[79]: # evaluating the model
ml_model_test_eval(catBC_grid)
```

Best Parameters:

```
{'bagging_temperature': 1, 'bootstrap_type': 'Bayesian', 'depth': 8,
'iterations': 500, 'learning_rate': 0.05}
```

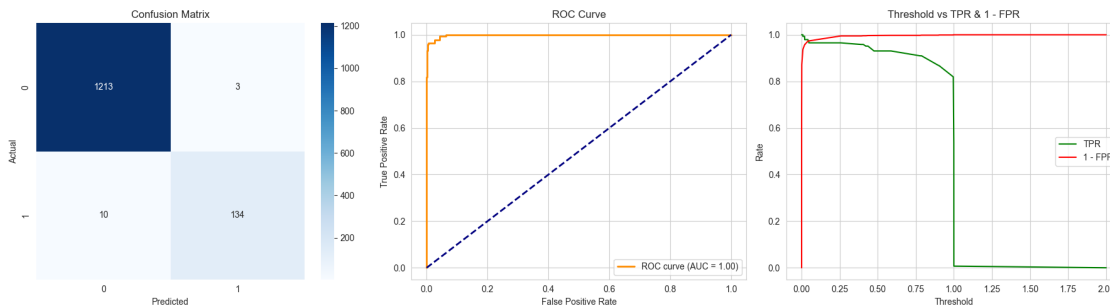
Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	0.99	1216
1	0.98	0.93	0.95	144
accuracy			0.99	1360
macro avg	0.98	0.96	0.97	1360
weighted avg	0.99	0.99	0.99	1360

Accuracy: 0.9904

F1 Score: 0.9537

ROC AUC Score: 0.9983



4.3.4.4 Light Boosting Gradient Machine Classifier

```
[80]: # LightGBM Classifier parameters
lgbmc_parameters = {
    'n_estimators': [400, 600, 800, 900],          # Increased estimators
    'learning_rate': [0.01, 0.05, 0.1],           # Added smaller learning
    'max_depth': [4, 6, 8],                        # Reduced max depth to
    'num_leaves': [15, 31, 45],                   # Number of leaves
    'min_child_samples': [5, 10, 20],              # Lower
    'subsample': [0.7, 0.85, 1.0],                # Added more randomness
    'colsample_bytree': [0.6, 0.8, 1.0],           # Added more variation in
    'reg_alpha': [0.0, 0.1, 0.3],                 # Adjust regularization
    'reg_lambda': [0.0, 0.1, 0.3]                 # Similar for lambda to
}

```

```
[81]: # initializing the Light Gradient Machine Boosting Classifier with gridsearchCV
lightgbmC_grid = GridSearchCV(estimator=LGBMClassifier(),
    param_grid=lgbmc_parameters,
    cv=4, n_jobs=-1, scoring="f1_macro", verbose=10)

# fitting the model
lightgbmC_grid.fit(X_train_resampled, y_train_resampled)

# saving the results into df
lightgbmC_grid_cv_result = pd.DataFrame.from_dict(lightgbmC_grid.cv_results_)

```

```

# LGBMc best parameters
lightgbmC_grid_params = lightgbmC_grid.best_params_

# saving the df as a csv for further analysis if needed
lightgbmC_grid_cv_result.to_csv("../models/ML_trials/LightGBMC_model/
↳lightgbmC_grid_cv_results2.csv")

```

Fitting 4 folds for each of 8748 candidates, totalling 34992 fits

[LightGBM] [Info] Number of positive: 2835, number of negative: 2835

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.006960 seconds.

You can set `force\_row\_wise=true` to remove the overhead.

And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 1345

[LightGBM] [Info] Number of data points in the train set: 5670, number of used features: 12

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[illegible]

[illegible]

[illegible]

[illegible]

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[82]: # printing the best parameters for lightgbmc
print(lightgbmC_grid_params.values())
```

```
dict_values([0.6, 0.1, 8, 20, 600, 31, 0.0, 0.7])
```

```
[83]: # best rank results
lightgbmC_grid_cv_result[lightgbmC_grid_cv_result['rank_test_score']==1]
```

```
[83]:      mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
2844      10.669130      0.844244      0.011879      0.002406
2845      12.037733      0.955392      0.010750      0.000830
2846      11.768932      0.803836      0.010752      0.001302

      param_colsample_bytree param_learning_rate param_max_depth  \
2844              0.6              0.1              8
2845              0.6              0.1              8
2846              0.6              0.1              8

      param_min_child_samples param_n_estimators param_num_leaves  \
2844              20              600              31
2845              20              600              31
2846              20              600              31

      param_reg_lambda param_subsample  \
2844              0.0              0.7
```

2845	0.0	0.85
2846	0.0	1.0

	params	split0_test_score \
2844	{'colsample_bytree': 0.6, 'learning_rate': 0.1...	0.987304
2845	{'colsample_bytree': 0.6, 'learning_rate': 0.1...	0.987304
2846	{'colsample_bytree': 0.6, 'learning_rate': 0.1...	0.987304

	split1_test_score	split2_test_score	split3_test_score \
2844	0.995063	0.995766	0.99506
2845	0.995063	0.995766	0.99506
2846	0.995063	0.995766	0.99506

	mean_test_score	std_test_score	rank_test_score
2844	0.993298	0.003473	1
2845	0.993298	0.003473	1
2846	0.993298	0.003473	1

```
[84]: # metric evaluation for the model
ml_model_test_eval(lightgbmC_grid)
```

Best Parameters:

```
{'colsample_bytree': 0.6, 'learning_rate': 0.1, 'max_depth': 8,
'min_child_samples': 20, 'n_estimators': 600, 'num_leaves': 31, 'reg_lambda':
0.0, 'subsample': 0.7}
```

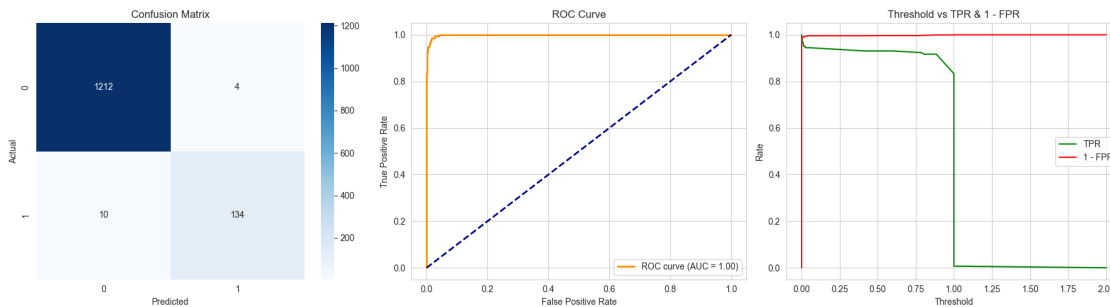
Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	0.99	1216
1	0.97	0.93	0.95	144
accuracy			0.99	1360
macro avg	0.98	0.96	0.97	1360
weighted avg	0.99	0.99	0.99	1360

Accuracy: 0.9897

F1 Score: 0.9504

ROC AUC Score: 0.9989



4.3.4.5 eXtreme Gradient Boosting Classifier

```
[85]: # eXtreme Gradient Boosting Classifiers Parameters
xgbc_parameters = {
    'learning_rate': [0.01, 0.05, 0.1],          # Lower learning rate for
    #better convergence control
    'n_estimators': [500, 750, 900],             # Added more variety to
    #the number of estimators
    'max_depth': [4, 6, 8],                      # Reduced max depth to
    #prevent overfitting with shallow trees
    'subsample': [0.7, 0.85, 1.0],               # Adjusted subsample
    #range for better generalization
    'colsample_bytree': [0.6, 0.8, 1.0],         # More feature randomness
    #for each tree
    'gamma': [0.1, 0.2, 0.3],                   # Added a higher gamma
    #value for further pruning
    'reg_lambda': [1.0, 1.5, 2.0],              # L2 regularization,
    #controlled for better complexity management
    'reg_alpha': [0.1, 0.5, 1.0]                # L1 regularization,
    #added more variety for sparsity
}
```

```
[86]: # initializing the eXtreme Gradient Boosting Classifier with gridsearchCV
xgbC_grid = GridSearchCV(estimator=XGBClassifier(), param_grid=xgbc_parameters,
                          cv=4, n_jobs=-1, scoring="f1_macro", verbose=10)

# fitting the model
xgbC_grid.fit(X_train_resampled, y_train_resampled)

# saving the results into df
xgbC_grid_cv_result = pd.DataFrame.from_dict(xgbC_grid.cv_results_)

# XGBcL best parameters
xgbC_grid_params = xgbC_grid.best_params_
```

```
# saving the df as a csv for further analysis if needed
xgbC_grid_cv_result.to_csv("../models/ML_trials/XGBClassifiers_model/
↳xgbC_grid_cv_results2.csv")
```

Fitting 4 folds for each of 6561 candidates, totalling 26244 fits

```
[87]: # printing the best parameters for XGBClassifier
print(xgbC_grid_params.values())
```

dict_values([0.8, 0.1, 0.05, 4, 900, 0.1, 1.5, 0.7])

```
[88]: # xgbClassifiers rank 1 scores and its parameters
xgbC_grid_cv_result[xgbC_grid_cv_result['rank_test_score']==1]
```

```
[88]:      mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
2487      1.131231      0.025975      0.039134      0.001443

      param_colsample_bytree param_gamma param_learning_rate param_max_depth  \
2487                      0.8          0.1              0.05              4

      param_n_estimators param_reg_alpha param_reg_lambda param_subsample  \
2487                900              0.1              1.5              0.7

                                     params  split0_test_score  \
2487  {'colsample_bytree': 0.8, 'gamma': 0.1, 'learn...      0.984484

      split1_test_score  split2_test_score  split3_test_score  \
2487      0.992948      0.994354      0.992943

      mean_test_score  std_test_score  rank_test_score
2487      0.991182      0.00391      1
```

```
[89]: ml_model_test_eval(xgbC_grid)
```

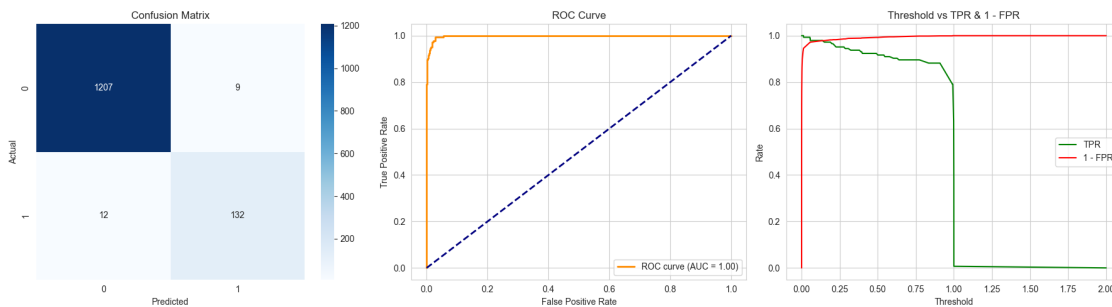
Best Parameters:

```
{'colsample_bytree': 0.8, 'gamma': 0.1, 'learning_rate': 0.05, 'max_depth': 4,
'n_estimators': 900, 'reg_alpha': 0.1, 'reg_lambda': 1.5, 'subsample': 0.7}
```

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	1216
1	0.94	0.92	0.93	144
accuracy			0.98	1360
macro avg	0.96	0.95	0.96	1360
weighted avg	0.98	0.98	0.98	1360

Accuracy: 0.9846
 F1 Score: 0.9263
 ROC AUC Score: 0.9981



4.3.4.6 eXtreme Gradient Boosting Random Forest Classifier

```
[90]: # Xtreme Gradient Boosting Random FOrest Classifiers
xgbrfC_parameters = {
    'n_estimators': [300, 450, 600],          # Optimized number of
    ↪estimators for performance balance
    'max_depth': [3, 5, 7],                    # Added intermediate
    ↪depth for more flexibility
    'learning_rate': [0.01, 0.05, 0.1],        # Lower learning rate
    ↪options for better fine-tuning
    'colsample_bytree': [0.5, 0.7, 1.0],        # Range of feature
    ↪subsample to promote diversity in trees
    'subsample': [0.6, 0.8, 1.0],              # Introduced
    ↪intermediate subsample values for more flexibility
    'min_child_weight': [1, 3, 5],              # Broader range to
    ↪control overfitting (lower values avoid pruning)
    'gamma': [0, 0.1, 0.3],                    # Expanded gamma for
    ↪better tree pruning
    'reg_lambda': [1.0, 1.5, 2.0]              # L2 regularization for
    ↪controlling model complexity
}
```

```
[91]: # initializing the eXtreme Gradient Boosting Classifier with gridsearchCV
xgbrfC_grid = GridSearchCV(estimator=XGBRFClassifier(),
    ↪param_grid=xgbrfC_parameters,
                           cv=4, n_jobs=-1, scoring="f1_macro", verbose=10)

# fitting the model
xgbrfC_grid.fit(X_train_resampled, y_train_resampled)

# saving the results into df
```

```
xgbrfC_grid_cv_result = pd.DataFrame.from_dict(xgbrfC_grid.cv_results_)

# Xtremem gradient Boosting random forest best parameters
xgbrfC_grid_params = xgbrfC_grid.best_params_

# saving the df as a csv for further analysis if needed
xgbrfC_grid_cv_result.to_csv("../models/ML_trials/XGBRFClassifiers_model/
↳xgbrfC_grid_cv_results2.csv")
```

Fitting 4 folds for each of 6561 candidates, totalling 26244 fits

```
[92]: # printing the best parameters for XGBRFClassifier
print(xgbrfC_grid_params.values())
```

```
dict_values([0.7, 0.3, 0.01, 7, 1, 300, 1.0, 0.8])
```

```
[93]: # xgbrfClassifiers rank 1 scores and its parameters
xgbrfC_grid_cv_result[xgbrfC_grid_cv_result['rank_test_score']==1]
```

```
[93]:      mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
3808      0.966566      0.160337      0.042886      0.01069

      param_colsample_bytree param_gamma param_learning_rate param_max_depth  \
3808              0.7          0.3              0.01              7

      param_min_child_weight param_n_estimators param_reg_lambda  \
3808              1              300              1.0

      param_subsample                                params  \
3808      0.8  {'colsample_bytree': 0.7, 'gamma': 0.3, 'learn...

      split0_test_score  split1_test_score  split2_test_score  \
3808      0.984485          0.989421          0.985886

      split3_test_score  mean_test_score  std_test_score  rank_test_score
3808      0.984474          0.986066          0.00202          1
```

```
[94]: ml_model_test_eval(xgbrfC_grid)
```

Best Parameters:

```
{'colsample_bytree': 0.7, 'gamma': 0.3, 'learning_rate': 0.01, 'max_depth': 7,
'min_child_weight': 1, 'n_estimators': 300, 'reg_lambda': 1.0, 'subsample': 0.8}
```

Classification Report:

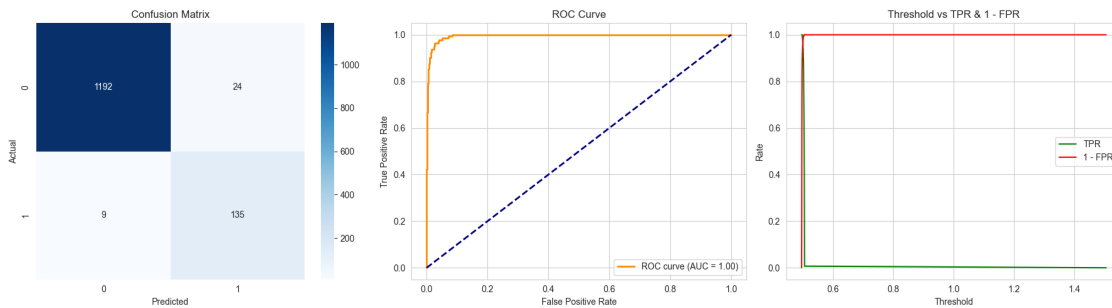
	precision	recall	f1-score	support
0	0.99	0.98	0.99	1216
1	0.85	0.94	0.89	144

accuracy			0.98	1360
macro avg	0.92	0.96	0.94	1360
weighted avg	0.98	0.98	0.98	1360

Accuracy: 0.9757

F1 Score: 0.8911

ROC AUC Score: 0.9954



1.5.4 4.4 Final ML Model

```
[95]: for model_name , score in final_ML_model.items():
      print(f"Model Name : {model_name} , F1_score : {score}")
```

```
Model Name : RandomForestClassifier , F1_score : 0.8267477203647416
Model Name : GradientBoostingClassifier , F1_score : 0.953405017921147
Model Name : CatBoostClassifier , F1_score : 0.9537366548042704
Model Name : LGBMClassifier , F1_score : 0.9503546099290779
Model Name : XGBClassifier , F1_score : 0.9263157894736843
Model Name : XGBRFClassifier , F1_score : 0.8910891089108911
```

```
[105]: # final machine learning models list
final_ML_model_df = pd.DataFrame.from_dict(final_ML_model, orient='index')
final_ML_model_df
```

```
[105]:
```

	0
RandomForestClassifier	0.826748
GradientBoostingClassifier	0.953405
CatBoostClassifier	0.953737
LGBMClassifier	0.950355
XGBClassifier	0.926316
XGBRFClassifier	0.891089

From the above output we can see that the higher test accuracies belongs to * Gradient Boosting= 95.34% , * Cat Boosting = 95.37% and * Light Gradient Boosting Machine = 95.03%

So I personally think it would be wise to follow with CatBoostClassifier

4.4.1 Initializing the FINAL model

```
[106]: # best parameters of the chosen model (CatBoost Classifier)
catBC_params
```

```
[106]: {'bagging_temperature': 1,
        'bootstrap_type': 'Bayesian',
        'depth': 8,
        'iterations': 500,
        'learning_rate': 0.05}
```

```
[111]: # training the catboost model with the best parameters
cat_boost_official = CatBoostClassifier(
    bagging_temperature=1, bootstrap_type = 'Bayesian', depth = 8, iterations = 700,
    learning_rate = 0.05, l2_leaf_reg = 3,
    grow_policy = 'SymmetricTree' ,
)

'''
I have also altered some parameters and increased some values.
'''

# fitting the model with the train set
cat_boost_official.fit( X_train_resampled, y_train_resampled )
```

0:	learn: 0.5924776	total: 19.8ms	remaining: 13.8s
1:	learn: 0.4992951	total: 29.7ms	remaining: 10.4s
2:	learn: 0.4239790	total: 39.3ms	remaining: 9.12s
3:	learn: 0.3593198	total: 46.1ms	remaining: 8.02s
4:	learn: 0.3097294	total: 53.3ms	remaining: 7.41s
5:	learn: 0.2680175	total: 60.8ms	remaining: 7.03s
6:	learn: 0.2277468	total: 67.1ms	remaining: 6.64s
7:	learn: 0.1991604	total: 72.6ms	remaining: 6.28s
8:	learn: 0.1752255	total: 77.1ms	remaining: 5.92s
9:	learn: 0.1541306	total: 81.8ms	remaining: 5.64s
10:	learn: 0.1352651	total: 86.3ms	remaining: 5.41s
11:	learn: 0.1227542	total: 91.2ms	remaining: 5.23s
12:	learn: 0.1118135	total: 95.9ms	remaining: 5.07s
13:	learn: 0.1026632	total: 101ms	remaining: 4.94s
14:	learn: 0.0932239	total: 106ms	remaining: 4.82s
15:	learn: 0.0853072	total: 111ms	remaining: 4.73s
16:	learn: 0.0788559	total: 117ms	remaining: 4.69s
17:	learn: 0.0740947	total: 122ms	remaining: 4.61s
18:	learn: 0.0703355	total: 127ms	remaining: 4.54s
19:	learn: 0.0655898	total: 133ms	remaining: 4.51s
20:	learn: 0.0614205	total: 137ms	remaining: 4.44s
21:	learn: 0.0583806	total: 143ms	remaining: 4.41s
22:	learn: 0.0551378	total: 148ms	remaining: 4.36s

23:	learn: 0.0527116	total: 153ms	remaining: 4.3s
24:	learn: 0.0502985	total: 157ms	remaining: 4.25s
25:	learn: 0.0480537	total: 162ms	remaining: 4.2s
26:	learn: 0.0463753	total: 166ms	remaining: 4.14s
27:	learn: 0.0447313	total: 172ms	remaining: 4.12s
28:	learn: 0.0433654	total: 177ms	remaining: 4.1s
29:	learn: 0.0419742	total: 182ms	remaining: 4.06s
30:	learn: 0.0410697	total: 187ms	remaining: 4.04s
31:	learn: 0.0398721	total: 193ms	remaining: 4.04s
32:	learn: 0.0388634	total: 200ms	remaining: 4.04s
33:	learn: 0.0379530	total: 205ms	remaining: 4.02s
34:	learn: 0.0373086	total: 212ms	remaining: 4.02s
35:	learn: 0.0368524	total: 217ms	remaining: 4.01s
36:	learn: 0.0359842	total: 222ms	remaining: 3.98s
37:	learn: 0.0352432	total: 227ms	remaining: 3.96s
38:	learn: 0.0343683	total: 234ms	remaining: 3.97s
39:	learn: 0.0335449	total: 242ms	remaining: 3.99s
40:	learn: 0.0330368	total: 248ms	remaining: 3.99s
41:	learn: 0.0325738	total: 257ms	remaining: 4.03s
42:	learn: 0.0318968	total: 264ms	remaining: 4.04s
43:	learn: 0.0315130	total: 270ms	remaining: 4.03s
44:	learn: 0.0311055	total: 276ms	remaining: 4.01s
45:	learn: 0.0305903	total: 281ms	remaining: 4s
46:	learn: 0.0300710	total: 286ms	remaining: 3.98s
47:	learn: 0.0296350	total: 292ms	remaining: 3.96s
48:	learn: 0.0293816	total: 298ms	remaining: 3.96s
49:	learn: 0.0287397	total: 305ms	remaining: 3.96s
50:	learn: 0.0280685	total: 312ms	remaining: 3.97s
51:	learn: 0.0273734	total: 318ms	remaining: 3.97s
52:	learn: 0.0268664	total: 325ms	remaining: 3.97s
53:	learn: 0.0264327	total: 331ms	remaining: 3.96s
54:	learn: 0.0260860	total: 337ms	remaining: 3.95s
55:	learn: 0.0254145	total: 342ms	remaining: 3.93s
56:	learn: 0.0249709	total: 347ms	remaining: 3.92s
57:	learn: 0.0244294	total: 354ms	remaining: 3.91s
58:	learn: 0.0242283	total: 358ms	remaining: 3.89s
59:	learn: 0.0238841	total: 364ms	remaining: 3.88s
60:	learn: 0.0235297	total: 370ms	remaining: 3.87s
61:	learn: 0.0232153	total: 375ms	remaining: 3.86s
62:	learn: 0.0229829	total: 380ms	remaining: 3.85s
63:	learn: 0.0224896	total: 385ms	remaining: 3.83s
64:	learn: 0.0222426	total: 390ms	remaining: 3.81s
65:	learn: 0.0219691	total: 396ms	remaining: 3.8s
66:	learn: 0.0216870	total: 401ms	remaining: 3.79s
67:	learn: 0.0213810	total: 406ms	remaining: 3.77s
68:	learn: 0.0210809	total: 412ms	remaining: 3.77s
69:	learn: 0.0208627	total: 419ms	remaining: 3.77s
70:	learn: 0.0205998	total: 425ms	remaining: 3.76s

71:	learn: 0.0202835	total: 432ms	remaining: 3.77s
72:	learn: 0.0200451	total: 439ms	remaining: 3.77s
73:	learn: 0.0198654	total: 446ms	remaining: 3.77s
74:	learn: 0.0196404	total: 453ms	remaining: 3.78s
75:	learn: 0.0194499	total: 461ms	remaining: 3.79s
76:	learn: 0.0192029	total: 467ms	remaining: 3.78s
77:	learn: 0.0188834	total: 475ms	remaining: 3.79s
78:	learn: 0.0186646	total: 482ms	remaining: 3.79s
79:	learn: 0.0185354	total: 489ms	remaining: 3.79s
80:	learn: 0.0184229	total: 496ms	remaining: 3.79s
81:	learn: 0.0181919	total: 503ms	remaining: 3.79s
82:	learn: 0.0179832	total: 509ms	remaining: 3.78s
83:	learn: 0.0177372	total: 516ms	remaining: 3.78s
84:	learn: 0.0176191	total: 523ms	remaining: 3.78s
85:	learn: 0.0174107	total: 530ms	remaining: 3.78s
86:	learn: 0.0172226	total: 536ms	remaining: 3.78s
87:	learn: 0.0170325	total: 543ms	remaining: 3.78s
88:	learn: 0.0168735	total: 551ms	remaining: 3.78s
89:	learn: 0.0166676	total: 557ms	remaining: 3.78s
90:	learn: 0.0165259	total: 563ms	remaining: 3.77s
91:	learn: 0.0164231	total: 570ms	remaining: 3.76s
92:	learn: 0.0162845	total: 578ms	remaining: 3.77s
93:	learn: 0.0160726	total: 586ms	remaining: 3.78s
94:	learn: 0.0159177	total: 595ms	remaining: 3.79s
95:	learn: 0.0158503	total: 600ms	remaining: 3.78s
96:	learn: 0.0157090	total: 607ms	remaining: 3.77s
97:	learn: 0.0155613	total: 615ms	remaining: 3.77s
98:	learn: 0.0154628	total: 623ms	remaining: 3.78s
99:	learn: 0.0153434	total: 630ms	remaining: 3.78s
100:	learn: 0.0151890	total: 640ms	remaining: 3.79s
101:	learn: 0.0150518	total: 646ms	remaining: 3.79s
102:	learn: 0.0149453	total: 653ms	remaining: 3.78s
103:	learn: 0.0148837	total: 659ms	remaining: 3.77s
104:	learn: 0.0147855	total: 666ms	remaining: 3.77s
105:	learn: 0.0146618	total: 674ms	remaining: 3.77s
106:	learn: 0.0145166	total: 680ms	remaining: 3.77s
107:	learn: 0.0144174	total: 686ms	remaining: 3.76s
108:	learn: 0.0142697	total: 692ms	remaining: 3.75s
109:	learn: 0.0141419	total: 699ms	remaining: 3.75s
110:	learn: 0.0140203	total: 706ms	remaining: 3.75s
111:	learn: 0.0138687	total: 713ms	remaining: 3.74s
112:	learn: 0.0137533	total: 722ms	remaining: 3.75s
113:	learn: 0.0136454	total: 728ms	remaining: 3.74s
114:	learn: 0.0135451	total: 736ms	remaining: 3.74s
115:	learn: 0.0135105	total: 743ms	remaining: 3.74s
116:	learn: 0.0133900	total: 749ms	remaining: 3.73s
117:	learn: 0.0132822	total: 756ms	remaining: 3.73s
118:	learn: 0.0132122	total: 762ms	remaining: 3.72s

119:	learn: 0.0130854	total: 768ms	remaining: 3.71s
120:	learn: 0.0130071	total: 773ms	remaining: 3.7s
121:	learn: 0.0129136	total: 778ms	remaining: 3.69s
122:	learn: 0.0127949	total: 784ms	remaining: 3.68s
123:	learn: 0.0126911	total: 791ms	remaining: 3.67s
124:	learn: 0.0126139	total: 797ms	remaining: 3.67s
125:	learn: 0.0124723	total: 803ms	remaining: 3.66s
126:	learn: 0.0123961	total: 809ms	remaining: 3.65s
127:	learn: 0.0122302	total: 816ms	remaining: 3.65s
128:	learn: 0.0121267	total: 823ms	remaining: 3.64s
129:	learn: 0.0120535	total: 828ms	remaining: 3.63s
130:	learn: 0.0119849	total: 835ms	remaining: 3.63s
131:	learn: 0.0119192	total: 840ms	remaining: 3.62s
132:	learn: 0.0117933	total: 848ms	remaining: 3.61s
133:	learn: 0.0117165	total: 854ms	remaining: 3.61s
134:	learn: 0.0116459	total: 860ms	remaining: 3.6s
135:	learn: 0.0115665	total: 866ms	remaining: 3.59s
136:	learn: 0.0115064	total: 872ms	remaining: 3.58s
137:	learn: 0.0113627	total: 879ms	remaining: 3.58s
138:	learn: 0.0112668	total: 884ms	remaining: 3.57s
139:	learn: 0.0112148	total: 889ms	remaining: 3.56s
140:	learn: 0.0111274	total: 896ms	remaining: 3.55s
141:	learn: 0.0110274	total: 903ms	remaining: 3.55s
142:	learn: 0.0109525	total: 910ms	remaining: 3.54s
143:	learn: 0.0108471	total: 916ms	remaining: 3.54s
144:	learn: 0.0107912	total: 922ms	remaining: 3.53s
145:	learn: 0.0107170	total: 929ms	remaining: 3.52s
146:	learn: 0.0106340	total: 934ms	remaining: 3.52s
147:	learn: 0.0105628	total: 940ms	remaining: 3.5s
148:	learn: 0.0104438	total: 945ms	remaining: 3.5s
149:	learn: 0.0103439	total: 952ms	remaining: 3.49s
150:	learn: 0.0102536	total: 958ms	remaining: 3.48s
151:	learn: 0.0101726	total: 965ms	remaining: 3.48s
152:	learn: 0.0100776	total: 971ms	remaining: 3.47s
153:	learn: 0.0100159	total: 977ms	remaining: 3.46s
154:	learn: 0.0099578	total: 983ms	remaining: 3.46s
155:	learn: 0.0098968	total: 988ms	remaining: 3.45s
156:	learn: 0.0098366	total: 994ms	remaining: 3.44s
157:	learn: 0.0097898	total: 999ms	remaining: 3.43s
158:	learn: 0.0097138	total: 1s	remaining: 3.42s
159:	learn: 0.0096495	total: 1.01s	remaining: 3.41s
160:	learn: 0.0096003	total: 1.01s	remaining: 3.4s
161:	learn: 0.0095040	total: 1.02s	remaining: 3.39s
162:	learn: 0.0094234	total: 1.02s	remaining: 3.38s
163:	learn: 0.0093661	total: 1.03s	remaining: 3.36s
164:	learn: 0.0092999	total: 1.03s	remaining: 3.35s
165:	learn: 0.0092352	total: 1.04s	remaining: 3.35s
166:	learn: 0.0091706	total: 1.04s	remaining: 3.34s

167:	learn: 0.0091087	total: 1.05s	remaining: 3.33s
168:	learn: 0.0090403	total: 1.06s	remaining: 3.32s
169:	learn: 0.0089639	total: 1.06s	remaining: 3.31s
170:	learn: 0.0088916	total: 1.07s	remaining: 3.3s
171:	learn: 0.0088574	total: 1.07s	remaining: 3.3s
172:	learn: 0.0087921	total: 1.08s	remaining: 3.29s
173:	learn: 0.0087198	total: 1.09s	remaining: 3.29s
174:	learn: 0.0086586	total: 1.09s	remaining: 3.28s
175:	learn: 0.0085642	total: 1.1s	remaining: 3.27s
176:	learn: 0.0084789	total: 1.1s	remaining: 3.27s
177:	learn: 0.0084034	total: 1.11s	remaining: 3.26s
178:	learn: 0.0083804	total: 1.12s	remaining: 3.25s
179:	learn: 0.0083237	total: 1.12s	remaining: 3.24s
180:	learn: 0.0082916	total: 1.13s	remaining: 3.23s
181:	learn: 0.0082232	total: 1.13s	remaining: 3.23s
182:	learn: 0.0081490	total: 1.14s	remaining: 3.22s
183:	learn: 0.0080647	total: 1.15s	remaining: 3.21s
184:	learn: 0.0080043	total: 1.15s	remaining: 3.21s
185:	learn: 0.0079591	total: 1.16s	remaining: 3.2s
186:	learn: 0.0079115	total: 1.17s	remaining: 3.2s
187:	learn: 0.0078741	total: 1.17s	remaining: 3.19s
188:	learn: 0.0078317	total: 1.18s	remaining: 3.18s
189:	learn: 0.0077967	total: 1.18s	remaining: 3.17s
190:	learn: 0.0077603	total: 1.19s	remaining: 3.16s
191:	learn: 0.0077250	total: 1.19s	remaining: 3.15s
192:	learn: 0.0076626	total: 1.2s	remaining: 3.14s
193:	learn: 0.0076047	total: 1.2s	remaining: 3.13s
194:	learn: 0.0075519	total: 1.21s	remaining: 3.12s
195:	learn: 0.0075005	total: 1.21s	remaining: 3.11s
196:	learn: 0.0074611	total: 1.22s	remaining: 3.11s
197:	learn: 0.0074283	total: 1.22s	remaining: 3.1s
198:	learn: 0.0073598	total: 1.23s	remaining: 3.09s
199:	learn: 0.0073071	total: 1.23s	remaining: 3.09s
200:	learn: 0.0072584	total: 1.24s	remaining: 3.08s
201:	learn: 0.0072151	total: 1.25s	remaining: 3.07s
202:	learn: 0.0071775	total: 1.25s	remaining: 3.06s
203:	learn: 0.0071399	total: 1.25s	remaining: 3.05s
204:	learn: 0.0070946	total: 1.26s	remaining: 3.05s
205:	learn: 0.0070469	total: 1.27s	remaining: 3.04s
206:	learn: 0.0070218	total: 1.27s	remaining: 3.04s
207:	learn: 0.0069726	total: 1.28s	remaining: 3.03s
208:	learn: 0.0069311	total: 1.29s	remaining: 3.02s
209:	learn: 0.0068807	total: 1.29s	remaining: 3.02s
210:	learn: 0.0068265	total: 1.3s	remaining: 3.02s
211:	learn: 0.0067955	total: 1.31s	remaining: 3.01s
212:	learn: 0.0067700	total: 1.31s	remaining: 3s
213:	learn: 0.0067252	total: 1.32s	remaining: 2.99s
214:	learn: 0.0066913	total: 1.32s	remaining: 2.99s

215:	learn: 0.0066495	total: 1.33s	remaining: 2.98s
216:	learn: 0.0066199	total: 1.34s	remaining: 2.98s
217:	learn: 0.0065734	total: 1.34s	remaining: 2.97s
218:	learn: 0.0065142	total: 1.35s	remaining: 2.97s
219:	learn: 0.0064897	total: 1.35s	remaining: 2.96s
220:	learn: 0.0064565	total: 1.36s	remaining: 2.95s
221:	learn: 0.0064190	total: 1.37s	remaining: 2.94s
222:	learn: 0.0063868	total: 1.37s	remaining: 2.93s
223:	learn: 0.0063516	total: 1.38s	remaining: 2.92s
224:	learn: 0.0063270	total: 1.38s	remaining: 2.92s
225:	learn: 0.0062981	total: 1.39s	remaining: 2.91s
226:	learn: 0.0062522	total: 1.39s	remaining: 2.9s
227:	learn: 0.0061992	total: 1.4s	remaining: 2.9s
228:	learn: 0.0061701	total: 1.4s	remaining: 2.89s
229:	learn: 0.0061362	total: 1.41s	remaining: 2.88s
230:	learn: 0.0061099	total: 1.42s	remaining: 2.88s
231:	learn: 0.0060652	total: 1.42s	remaining: 2.87s
232:	learn: 0.0060323	total: 1.43s	remaining: 2.86s
233:	learn: 0.0059898	total: 1.43s	remaining: 2.86s
234:	learn: 0.0059537	total: 1.44s	remaining: 2.85s
235:	learn: 0.0059321	total: 1.45s	remaining: 2.85s
236:	learn: 0.0059023	total: 1.45s	remaining: 2.84s
237:	learn: 0.0058622	total: 1.46s	remaining: 2.83s
238:	learn: 0.0058422	total: 1.46s	remaining: 2.82s
239:	learn: 0.0057961	total: 1.47s	remaining: 2.81s
240:	learn: 0.0057580	total: 1.47s	remaining: 2.81s
241:	learn: 0.0057216	total: 1.48s	remaining: 2.8s
242:	learn: 0.0056899	total: 1.49s	remaining: 2.79s
243:	learn: 0.0056609	total: 1.49s	remaining: 2.79s
244:	learn: 0.0056365	total: 1.5s	remaining: 2.78s
245:	learn: 0.0056049	total: 1.51s	remaining: 2.78s
246:	learn: 0.0055940	total: 1.51s	remaining: 2.77s
247:	learn: 0.0055631	total: 1.52s	remaining: 2.77s
248:	learn: 0.0055364	total: 1.52s	remaining: 2.76s
249:	learn: 0.0054871	total: 1.53s	remaining: 2.75s
250:	learn: 0.0054484	total: 1.54s	remaining: 2.75s
251:	learn: 0.0054212	total: 1.54s	remaining: 2.75s
252:	learn: 0.0053866	total: 1.55s	remaining: 2.74s
253:	learn: 0.0053564	total: 1.56s	remaining: 2.73s
254:	learn: 0.0053180	total: 1.56s	remaining: 2.73s
255:	learn: 0.0052825	total: 1.57s	remaining: 2.72s
256:	learn: 0.0052691	total: 1.57s	remaining: 2.71s
257:	learn: 0.0052457	total: 1.58s	remaining: 2.71s
258:	learn: 0.0052194	total: 1.59s	remaining: 2.71s
259:	learn: 0.0051902	total: 1.59s	remaining: 2.7s
260:	learn: 0.0051511	total: 1.6s	remaining: 2.69s
261:	learn: 0.0051197	total: 1.6s	remaining: 2.68s
262:	learn: 0.0050927	total: 1.61s	remaining: 2.67s

263:	learn: 0.0050694	total: 1.61s	remaining: 2.67s
264:	learn: 0.0050398	total: 1.62s	remaining: 2.66s
265:	learn: 0.0050212	total: 1.63s	remaining: 2.65s
266:	learn: 0.0049984	total: 1.63s	remaining: 2.64s
267:	learn: 0.0049692	total: 1.64s	remaining: 2.64s
268:	learn: 0.0049440	total: 1.64s	remaining: 2.63s
269:	learn: 0.0049134	total: 1.65s	remaining: 2.62s
270:	learn: 0.0048891	total: 1.65s	remaining: 2.62s
271:	learn: 0.0048692	total: 1.66s	remaining: 2.61s
272:	learn: 0.0048366	total: 1.66s	remaining: 2.6s
273:	learn: 0.0048058	total: 1.67s	remaining: 2.59s
274:	learn: 0.0047789	total: 1.67s	remaining: 2.58s
275:	learn: 0.0047550	total: 1.68s	remaining: 2.58s
276:	learn: 0.0047409	total: 1.68s	remaining: 2.57s
277:	learn: 0.0047169	total: 1.69s	remaining: 2.56s
278:	learn: 0.0047065	total: 1.7s	remaining: 2.56s
279:	learn: 0.0046886	total: 1.7s	remaining: 2.55s
280:	learn: 0.0046744	total: 1.71s	remaining: 2.55s
281:	learn: 0.0046570	total: 1.72s	remaining: 2.54s
282:	learn: 0.0046350	total: 1.72s	remaining: 2.54s
283:	learn: 0.0046153	total: 1.73s	remaining: 2.53s
284:	learn: 0.0045935	total: 1.74s	remaining: 2.53s
285:	learn: 0.0045657	total: 1.74s	remaining: 2.52s
286:	learn: 0.0045366	total: 1.75s	remaining: 2.52s
287:	learn: 0.0045129	total: 1.76s	remaining: 2.52s
288:	learn: 0.0044876	total: 1.77s	remaining: 2.51s
289:	learn: 0.0044578	total: 1.77s	remaining: 2.51s
290:	learn: 0.0044408	total: 1.78s	remaining: 2.5s
291:	learn: 0.0044233	total: 1.79s	remaining: 2.5s
292:	learn: 0.0044001	total: 1.79s	remaining: 2.49s
293:	learn: 0.0043848	total: 1.8s	remaining: 2.48s
294:	learn: 0.0043682	total: 1.8s	remaining: 2.48s
295:	learn: 0.0043485	total: 1.81s	remaining: 2.47s
296:	learn: 0.0043305	total: 1.82s	remaining: 2.47s
297:	learn: 0.0043110	total: 1.82s	remaining: 2.46s
298:	learn: 0.0042877	total: 1.83s	remaining: 2.45s
299:	learn: 0.0042735	total: 1.84s	remaining: 2.45s
300:	learn: 0.0042492	total: 1.84s	remaining: 2.44s
301:	learn: 0.0042322	total: 1.85s	remaining: 2.44s
302:	learn: 0.0042229	total: 1.85s	remaining: 2.43s
303:	learn: 0.0042037	total: 1.86s	remaining: 2.42s
304:	learn: 0.0041858	total: 1.87s	remaining: 2.42s
305:	learn: 0.0041675	total: 1.88s	remaining: 2.41s
306:	learn: 0.0041530	total: 1.88s	remaining: 2.41s
307:	learn: 0.0041363	total: 1.89s	remaining: 2.41s
308:	learn: 0.0041116	total: 1.9s	remaining: 2.4s
309:	learn: 0.0040952	total: 1.9s	remaining: 2.4s
310:	learn: 0.0040747	total: 1.91s	remaining: 2.39s

311:	learn: 0.0040576	total: 1.92s	remaining: 2.39s
312:	learn: 0.0040445	total: 1.94s	remaining: 2.4s
313:	learn: 0.0040243	total: 1.94s	remaining: 2.39s
314:	learn: 0.0040068	total: 1.95s	remaining: 2.38s
315:	learn: 0.0039919	total: 1.96s	remaining: 2.38s
316:	learn: 0.0039761	total: 1.96s	remaining: 2.37s
317:	learn: 0.0039600	total: 1.97s	remaining: 2.36s
318:	learn: 0.0039406	total: 1.97s	remaining: 2.36s
319:	learn: 0.0039256	total: 1.98s	remaining: 2.35s
320:	learn: 0.0039048	total: 1.99s	remaining: 2.35s
321:	learn: 0.0038843	total: 2s	remaining: 2.34s
322:	learn: 0.0038671	total: 2s	remaining: 2.34s
323:	learn: 0.0038516	total: 2.01s	remaining: 2.33s
324:	learn: 0.0038273	total: 2.02s	remaining: 2.33s
325:	learn: 0.0038143	total: 2.02s	remaining: 2.32s
326:	learn: 0.0037902	total: 2.03s	remaining: 2.31s
327:	learn: 0.0037783	total: 2.03s	remaining: 2.31s
328:	learn: 0.0037629	total: 2.04s	remaining: 2.3s
329:	learn: 0.0037415	total: 2.05s	remaining: 2.29s
330:	learn: 0.0037220	total: 2.05s	remaining: 2.29s
331:	learn: 0.0037091	total: 2.06s	remaining: 2.28s
332:	learn: 0.0036917	total: 2.07s	remaining: 2.28s
333:	learn: 0.0036756	total: 2.07s	remaining: 2.27s
334:	learn: 0.0036600	total: 2.08s	remaining: 2.27s
335:	learn: 0.0036473	total: 2.08s	remaining: 2.26s
336:	learn: 0.0036388	total: 2.09s	remaining: 2.25s
337:	learn: 0.0036234	total: 2.1s	remaining: 2.25s
338:	learn: 0.0036034	total: 2.11s	remaining: 2.25s
339:	learn: 0.0035847	total: 2.12s	remaining: 2.24s
340:	learn: 0.0035705	total: 2.13s	remaining: 2.24s
341:	learn: 0.0035571	total: 2.13s	remaining: 2.23s
342:	learn: 0.0035459	total: 2.14s	remaining: 2.23s
343:	learn: 0.0035375	total: 2.14s	remaining: 2.22s
344:	learn: 0.0035273	total: 2.15s	remaining: 2.21s
345:	learn: 0.0035108	total: 2.16s	remaining: 2.21s
346:	learn: 0.0034907	total: 2.17s	remaining: 2.2s
347:	learn: 0.0034772	total: 2.17s	remaining: 2.2s
348:	learn: 0.0034681	total: 2.18s	remaining: 2.19s
349:	learn: 0.0034574	total: 2.19s	remaining: 2.19s
350:	learn: 0.0034431	total: 2.19s	remaining: 2.18s
351:	learn: 0.0034317	total: 2.2s	remaining: 2.17s
352:	learn: 0.0034182	total: 2.21s	remaining: 2.17s
353:	learn: 0.0034182	total: 2.21s	remaining: 2.16s
354:	learn: 0.0034022	total: 2.22s	remaining: 2.16s
355:	learn: 0.0033879	total: 2.23s	remaining: 2.15s
356:	learn: 0.0033754	total: 2.23s	remaining: 2.15s
357:	learn: 0.0033501	total: 2.24s	remaining: 2.14s
358:	learn: 0.0033367	total: 2.25s	remaining: 2.13s

359:	learn: 0.0033244	total: 2.25s	remaining: 2.13s
360:	learn: 0.0033088	total: 2.26s	remaining: 2.12s
361:	learn: 0.0033087	total: 2.26s	remaining: 2.11s
362:	learn: 0.0033087	total: 2.27s	remaining: 2.11s
363:	learn: 0.0032907	total: 2.28s	remaining: 2.1s
364:	learn: 0.0032813	total: 2.28s	remaining: 2.1s
365:	learn: 0.0032709	total: 2.29s	remaining: 2.09s
366:	learn: 0.0032555	total: 2.3s	remaining: 2.08s
367:	learn: 0.0032474	total: 2.3s	remaining: 2.08s
368:	learn: 0.0032314	total: 2.31s	remaining: 2.07s
369:	learn: 0.0032192	total: 2.32s	remaining: 2.07s
370:	learn: 0.0032058	total: 2.32s	remaining: 2.06s
371:	learn: 0.0032058	total: 2.33s	remaining: 2.05s
372:	learn: 0.0031970	total: 2.33s	remaining: 2.05s
373:	learn: 0.0031823	total: 2.34s	remaining: 2.04s
374:	learn: 0.0031738	total: 2.35s	remaining: 2.04s
375:	learn: 0.0031627	total: 2.36s	remaining: 2.03s
376:	learn: 0.0031514	total: 2.37s	remaining: 2.03s
377:	learn: 0.0031438	total: 2.37s	remaining: 2.02s
378:	learn: 0.0031319	total: 2.38s	remaining: 2.02s
379:	learn: 0.0031235	total: 2.39s	remaining: 2.01s
380:	learn: 0.0031084	total: 2.39s	remaining: 2s
381:	learn: 0.0030999	total: 2.4s	remaining: 2s
382:	learn: 0.0030902	total: 2.41s	remaining: 1.99s
383:	learn: 0.0030902	total: 2.41s	remaining: 1.99s
384:	learn: 0.0030745	total: 2.42s	remaining: 1.98s
385:	learn: 0.0030638	total: 2.42s	remaining: 1.97s
386:	learn: 0.0030506	total: 2.43s	remaining: 1.97s
387:	learn: 0.0030433	total: 2.44s	remaining: 1.96s
388:	learn: 0.0030341	total: 2.45s	remaining: 1.96s
389:	learn: 0.0030342	total: 2.45s	remaining: 1.95s
390:	learn: 0.0030247	total: 2.46s	remaining: 1.94s
391:	learn: 0.0030177	total: 2.46s	remaining: 1.94s
392:	learn: 0.0030176	total: 2.48s	remaining: 1.93s
393:	learn: 0.0030085	total: 2.48s	remaining: 1.93s
394:	learn: 0.0029995	total: 2.49s	remaining: 1.92s
395:	learn: 0.0029994	total: 2.49s	remaining: 1.91s
396:	learn: 0.0029858	total: 2.5s	remaining: 1.91s
397:	learn: 0.0029723	total: 2.5s	remaining: 1.9s
398:	learn: 0.0029605	total: 2.51s	remaining: 1.89s
399:	learn: 0.0029445	total: 2.51s	remaining: 1.88s
400:	learn: 0.0029380	total: 2.52s	remaining: 1.88s
401:	learn: 0.0029380	total: 2.52s	remaining: 1.87s
402:	learn: 0.0029379	total: 2.53s	remaining: 1.86s
403:	learn: 0.0029295	total: 2.53s	remaining: 1.86s
404:	learn: 0.0029184	total: 2.54s	remaining: 1.85s
405:	learn: 0.0029099	total: 2.54s	remaining: 1.84s
406:	learn: 0.0029100	total: 2.55s	remaining: 1.84s

407:	learn: 0.0028953	total: 2.56s	remaining: 1.83s
408:	learn: 0.0028883	total: 2.56s	remaining: 1.82s
409:	learn: 0.0028762	total: 2.57s	remaining: 1.82s
410:	learn: 0.0028646	total: 2.58s	remaining: 1.81s
411:	learn: 0.0028563	total: 2.58s	remaining: 1.8s
412:	learn: 0.0028563	total: 2.59s	remaining: 1.8s
413:	learn: 0.0028466	total: 2.6s	remaining: 1.79s
414:	learn: 0.0028466	total: 2.6s	remaining: 1.78s
415:	learn: 0.0028379	total: 2.6s	remaining: 1.78s
416:	learn: 0.0028300	total: 2.61s	remaining: 1.77s
417:	learn: 0.0028213	total: 2.62s	remaining: 1.77s
418:	learn: 0.0028123	total: 2.63s	remaining: 1.76s
419:	learn: 0.0028051	total: 2.63s	remaining: 1.75s
420:	learn: 0.0027952	total: 2.64s	remaining: 1.75s
421:	learn: 0.0027856	total: 2.64s	remaining: 1.74s
422:	learn: 0.0027776	total: 2.65s	remaining: 1.74s
423:	learn: 0.0027686	total: 2.66s	remaining: 1.73s
424:	learn: 0.0027603	total: 2.66s	remaining: 1.72s
425:	learn: 0.0027569	total: 2.67s	remaining: 1.72s
426:	learn: 0.0027470	total: 2.67s	remaining: 1.71s
427:	learn: 0.0027373	total: 2.68s	remaining: 1.7s
428:	learn: 0.0027291	total: 2.69s	remaining: 1.7s
429:	learn: 0.0027206	total: 2.69s	remaining: 1.69s
430:	learn: 0.0027101	total: 2.7s	remaining: 1.68s
431:	learn: 0.0027000	total: 2.7s	remaining: 1.68s
432:	learn: 0.0026865	total: 2.71s	remaining: 1.67s
433:	learn: 0.0026862	total: 2.71s	remaining: 1.66s
434:	learn: 0.0026739	total: 2.72s	remaining: 1.66s
435:	learn: 0.0026663	total: 2.73s	remaining: 1.65s
436:	learn: 0.0026553	total: 2.73s	remaining: 1.65s
437:	learn: 0.0026414	total: 2.74s	remaining: 1.64s
438:	learn: 0.0026339	total: 2.75s	remaining: 1.63s
439:	learn: 0.0026294	total: 2.75s	remaining: 1.63s
440:	learn: 0.0026195	total: 2.76s	remaining: 1.62s
441:	learn: 0.0026126	total: 2.76s	remaining: 1.61s
442:	learn: 0.0026040	total: 2.77s	remaining: 1.61s
443:	learn: 0.0026039	total: 2.78s	remaining: 1.6s
444:	learn: 0.0025978	total: 2.78s	remaining: 1.59s
445:	learn: 0.0025874	total: 2.79s	remaining: 1.59s
446:	learn: 0.0025843	total: 2.79s	remaining: 1.58s
447:	learn: 0.0025749	total: 2.8s	remaining: 1.57s
448:	learn: 0.0025672	total: 2.8s	remaining: 1.57s
449:	learn: 0.0025594	total: 2.81s	remaining: 1.56s
450:	learn: 0.0025498	total: 2.81s	remaining: 1.55s
451:	learn: 0.0025497	total: 2.82s	remaining: 1.55s
452:	learn: 0.0025416	total: 2.83s	remaining: 1.54s
453:	learn: 0.0025416	total: 2.83s	remaining: 1.53s
454:	learn: 0.0025345	total: 2.84s	remaining: 1.53s

455:	learn: 0.0025235	total: 2.84s	remaining: 1.52s
456:	learn: 0.0025234	total: 2.85s	remaining: 1.51s
457:	learn: 0.0025126	total: 2.85s	remaining: 1.51s
458:	learn: 0.0025025	total: 2.86s	remaining: 1.5s
459:	learn: 0.0025022	total: 2.87s	remaining: 1.5s
460:	learn: 0.0024951	total: 2.87s	remaining: 1.49s
461:	learn: 0.0024829	total: 2.88s	remaining: 1.48s
462:	learn: 0.0024763	total: 2.88s	remaining: 1.48s
463:	learn: 0.0024681	total: 2.89s	remaining: 1.47s
464:	learn: 0.0024611	total: 2.89s	remaining: 1.46s
465:	learn: 0.0024545	total: 2.9s	remaining: 1.46s
466:	learn: 0.0024545	total: 2.9s	remaining: 1.45s
467:	learn: 0.0024545	total: 2.91s	remaining: 1.44s
468:	learn: 0.0024545	total: 2.91s	remaining: 1.43s
469:	learn: 0.0024544	total: 2.92s	remaining: 1.43s
470:	learn: 0.0024471	total: 2.92s	remaining: 1.42s
471:	learn: 0.0024470	total: 2.93s	remaining: 1.42s
472:	learn: 0.0024471	total: 2.94s	remaining: 1.41s
473:	learn: 0.0024470	total: 2.94s	remaining: 1.4s
474:	learn: 0.0024407	total: 2.95s	remaining: 1.4s
475:	learn: 0.0024408	total: 2.95s	remaining: 1.39s
476:	learn: 0.0024407	total: 2.96s	remaining: 1.38s
477:	learn: 0.0024330	total: 2.97s	remaining: 1.38s
478:	learn: 0.0024194	total: 2.97s	remaining: 1.37s
479:	learn: 0.0024097	total: 2.98s	remaining: 1.36s
480:	learn: 0.0024019	total: 2.99s	remaining: 1.36s
481:	learn: 0.0023959	total: 2.99s	remaining: 1.35s
482:	learn: 0.0023906	total: 3s	remaining: 1.35s
483:	learn: 0.0023860	total: 3s	remaining: 1.34s
484:	learn: 0.0023860	total: 3.01s	remaining: 1.33s
485:	learn: 0.0023861	total: 3.02s	remaining: 1.33s
486:	learn: 0.0023804	total: 3.02s	remaining: 1.32s
487:	learn: 0.0023707	total: 3.03s	remaining: 1.31s
488:	learn: 0.0023610	total: 3.04s	remaining: 1.31s
489:	learn: 0.0023525	total: 3.04s	remaining: 1.3s
490:	learn: 0.0023486	total: 3.05s	remaining: 1.3s
491:	learn: 0.0023486	total: 3.05s	remaining: 1.29s
492:	learn: 0.0023409	total: 3.06s	remaining: 1.28s
493:	learn: 0.0023363	total: 3.06s	remaining: 1.28s
494:	learn: 0.0023260	total: 3.07s	remaining: 1.27s
495:	learn: 0.0023166	total: 3.08s	remaining: 1.27s
496:	learn: 0.0023166	total: 3.08s	remaining: 1.26s
497:	learn: 0.0023108	total: 3.09s	remaining: 1.25s
498:	learn: 0.0023087	total: 3.1s	remaining: 1.25s
499:	learn: 0.0023022	total: 3.1s	remaining: 1.24s
500:	learn: 0.0023023	total: 3.11s	remaining: 1.24s
501:	learn: 0.0023020	total: 3.12s	remaining: 1.23s
502:	learn: 0.0022936	total: 3.12s	remaining: 1.22s

503:	learn: 0.0022935	total: 3.13s	remaining: 1.22s
504:	learn: 0.0022839	total: 3.13s	remaining: 1.21s
505:	learn: 0.0022785	total: 3.14s	remaining: 1.2s
506:	learn: 0.0022785	total: 3.15s	remaining: 1.2s
507:	learn: 0.0022784	total: 3.15s	remaining: 1.19s
508:	learn: 0.0022701	total: 3.16s	remaining: 1.19s
509:	learn: 0.0022596	total: 3.17s	remaining: 1.18s
510:	learn: 0.0022596	total: 3.17s	remaining: 1.17s
511:	learn: 0.0022521	total: 3.18s	remaining: 1.17s
512:	learn: 0.0022476	total: 3.18s	remaining: 1.16s
513:	learn: 0.0022383	total: 3.19s	remaining: 1.15s
514:	learn: 0.0022384	total: 3.19s	remaining: 1.15s
515:	learn: 0.0022383	total: 3.2s	remaining: 1.14s
516:	learn: 0.0022302	total: 3.21s	remaining: 1.14s
517:	learn: 0.0022302	total: 3.21s	remaining: 1.13s
518:	learn: 0.0022301	total: 3.22s	remaining: 1.12s
519:	learn: 0.0022301	total: 3.23s	remaining: 1.12s
520:	learn: 0.0022223	total: 3.23s	remaining: 1.11s
521:	learn: 0.0022159	total: 3.24s	remaining: 1.1s
522:	learn: 0.0022159	total: 3.24s	remaining: 1.1s
523:	learn: 0.0022102	total: 3.25s	remaining: 1.09s
524:	learn: 0.0022023	total: 3.25s	remaining: 1.08s
525:	learn: 0.0021989	total: 3.26s	remaining: 1.08s
526:	learn: 0.0021989	total: 3.27s	remaining: 1.07s
527:	learn: 0.0021989	total: 3.27s	remaining: 1.07s
528:	learn: 0.0021896	total: 3.28s	remaining: 1.06s
529:	learn: 0.0021863	total: 3.28s	remaining: 1.05s
530:	learn: 0.0021779	total: 3.29s	remaining: 1.05s
531:	learn: 0.0021780	total: 3.29s	remaining: 1.04s
532:	learn: 0.0021714	total: 3.3s	remaining: 1.03s
533:	learn: 0.0021653	total: 3.31s	remaining: 1.03s
534:	learn: 0.0021652	total: 3.31s	remaining: 1.02s
535:	learn: 0.0021596	total: 3.32s	remaining: 1.02s
536:	learn: 0.0021595	total: 3.33s	remaining: 1.01s
537:	learn: 0.0021539	total: 3.34s	remaining: 1s
538:	learn: 0.0021500	total: 3.34s	remaining: 999ms
539:	learn: 0.0021418	total: 3.35s	remaining: 992ms
540:	learn: 0.0021418	total: 3.35s	remaining: 986ms
541:	learn: 0.0021316	total: 3.36s	remaining: 980ms
542:	learn: 0.0021239	total: 3.37s	remaining: 974ms
543:	learn: 0.0021173	total: 3.37s	remaining: 968ms
544:	learn: 0.0021095	total: 3.38s	remaining: 961ms
545:	learn: 0.0021095	total: 3.38s	remaining: 955ms
546:	learn: 0.0021022	total: 3.39s	remaining: 948ms
547:	learn: 0.0020977	total: 3.4s	remaining: 942ms
548:	learn: 0.0020977	total: 3.4s	remaining: 936ms
549:	learn: 0.0020977	total: 3.41s	remaining: 930ms
550:	learn: 0.0020976	total: 3.41s	remaining: 923ms

551:	learn: 0.0020890	total: 3.42s	remaining: 917ms
552:	learn: 0.0020824	total: 3.43s	remaining: 911ms
553:	learn: 0.0020824	total: 3.43s	remaining: 904ms
554:	learn: 0.0020759	total: 3.44s	remaining: 898ms
555:	learn: 0.0020711	total: 3.44s	remaining: 891ms
556:	learn: 0.0020711	total: 3.44s	remaining: 885ms
557:	learn: 0.0020711	total: 3.45s	remaining: 878ms
558:	learn: 0.0020673	total: 3.46s	remaining: 872ms
559:	learn: 0.0020672	total: 3.46s	remaining: 865ms
560:	learn: 0.0020672	total: 3.46s	remaining: 858ms
561:	learn: 0.0020671	total: 3.47s	remaining: 852ms
562:	learn: 0.0020611	total: 3.47s	remaining: 845ms
563:	learn: 0.0020610	total: 3.48s	remaining: 839ms
564:	learn: 0.0020610	total: 3.48s	remaining: 832ms
565:	learn: 0.0020555	total: 3.49s	remaining: 826ms
566:	learn: 0.0020555	total: 3.49s	remaining: 819ms
567:	learn: 0.0020482	total: 3.5s	remaining: 813ms
568:	learn: 0.0020425	total: 3.5s	remaining: 806ms
569:	learn: 0.0020378	total: 3.51s	remaining: 800ms
570:	learn: 0.0020379	total: 3.51s	remaining: 794ms
571:	learn: 0.0020378	total: 3.52s	remaining: 787ms
572:	learn: 0.0020333	total: 3.52s	remaining: 781ms
573:	learn: 0.0020334	total: 3.53s	remaining: 774ms
574:	learn: 0.0020333	total: 3.53s	remaining: 768ms
575:	learn: 0.0020251	total: 3.54s	remaining: 761ms
576:	learn: 0.0020251	total: 3.54s	remaining: 755ms
577:	learn: 0.0020186	total: 3.55s	remaining: 749ms
578:	learn: 0.0020134	total: 3.55s	remaining: 742ms
579:	learn: 0.0020123	total: 3.56s	remaining: 736ms
580:	learn: 0.0020122	total: 3.56s	remaining: 730ms
581:	learn: 0.0020122	total: 3.57s	remaining: 724ms
582:	learn: 0.0020122	total: 3.58s	remaining: 718ms
583:	learn: 0.0020122	total: 3.58s	remaining: 712ms
584:	learn: 0.0020114	total: 3.59s	remaining: 705ms
585:	learn: 0.0020114	total: 3.59s	remaining: 699ms
586:	learn: 0.0020115	total: 3.6s	remaining: 692ms
587:	learn: 0.0020115	total: 3.6s	remaining: 686ms
588:	learn: 0.0020114	total: 3.61s	remaining: 680ms
589:	learn: 0.0020044	total: 3.61s	remaining: 674ms
590:	learn: 0.0020044	total: 3.62s	remaining: 667ms
591:	learn: 0.0019966	total: 3.63s	remaining: 661ms
592:	learn: 0.0019966	total: 3.63s	remaining: 656ms
593:	learn: 0.0019966	total: 3.64s	remaining: 649ms
594:	learn: 0.0019909	total: 3.65s	remaining: 643ms
595:	learn: 0.0019909	total: 3.65s	remaining: 637ms
596:	learn: 0.0019839	total: 3.66s	remaining: 631ms
597:	learn: 0.0019770	total: 3.67s	remaining: 625ms
598:	learn: 0.0019770	total: 3.67s	remaining: 619ms

599:	learn: 0.0019750	total: 3.68s	remaining: 613ms
600:	learn: 0.0019663	total: 3.69s	remaining: 607ms
601:	learn: 0.0019611	total: 3.69s	remaining: 601ms
602:	learn: 0.0019611	total: 3.7s	remaining: 595ms
603:	learn: 0.0019611	total: 3.7s	remaining: 589ms
604:	learn: 0.0019611	total: 3.71s	remaining: 582ms
605:	learn: 0.0019611	total: 3.71s	remaining: 576ms
606:	learn: 0.0019553	total: 3.72s	remaining: 570ms
607:	learn: 0.0019495	total: 3.73s	remaining: 564ms
608:	learn: 0.0019438	total: 3.73s	remaining: 558ms
609:	learn: 0.0019437	total: 3.74s	remaining: 552ms
610:	learn: 0.0019437	total: 3.74s	remaining: 545ms
611:	learn: 0.0019418	total: 3.75s	remaining: 539ms
612:	learn: 0.0019416	total: 3.76s	remaining: 533ms
613:	learn: 0.0019416	total: 3.76s	remaining: 527ms
614:	learn: 0.0019343	total: 3.77s	remaining: 521ms
615:	learn: 0.0019342	total: 3.78s	remaining: 515ms
616:	learn: 0.0019288	total: 3.78s	remaining: 509ms
617:	learn: 0.0019215	total: 3.79s	remaining: 503ms
618:	learn: 0.0019215	total: 3.8s	remaining: 497ms
619:	learn: 0.0019153	total: 3.8s	remaining: 491ms
620:	learn: 0.0019154	total: 3.81s	remaining: 485ms
621:	learn: 0.0019106	total: 3.82s	remaining: 479ms
622:	learn: 0.0019033	total: 3.82s	remaining: 473ms
623:	learn: 0.0019033	total: 3.83s	remaining: 467ms
624:	learn: 0.0019034	total: 3.84s	remaining: 461ms
625:	learn: 0.0019033	total: 3.84s	remaining: 454ms
626:	learn: 0.0019033	total: 3.85s	remaining: 448ms
627:	learn: 0.0019033	total: 3.85s	remaining: 442ms
628:	learn: 0.0019032	total: 3.86s	remaining: 436ms
629:	learn: 0.0019032	total: 3.86s	remaining: 429ms
630:	learn: 0.0018981	total: 3.87s	remaining: 423ms
631:	learn: 0.0018937	total: 3.88s	remaining: 417ms
632:	learn: 0.0018938	total: 3.88s	remaining: 411ms
633:	learn: 0.0018888	total: 3.88s	remaining: 405ms
634:	learn: 0.0018860	total: 3.89s	remaining: 398ms
635:	learn: 0.0018861	total: 3.9s	remaining: 392ms
636:	learn: 0.0018860	total: 3.9s	remaining: 386ms
637:	learn: 0.0018860	total: 3.91s	remaining: 380ms
638:	learn: 0.0018860	total: 3.91s	remaining: 373ms
639:	learn: 0.0018804	total: 3.92s	remaining: 367ms
640:	learn: 0.0018804	total: 3.92s	remaining: 361ms
641:	learn: 0.0018803	total: 3.93s	remaining: 355ms
642:	learn: 0.0018803	total: 3.93s	remaining: 349ms
643:	learn: 0.0018792	total: 3.94s	remaining: 343ms
644:	learn: 0.0018754	total: 3.95s	remaining: 337ms
645:	learn: 0.0018752	total: 3.95s	remaining: 330ms
646:	learn: 0.0018752	total: 3.96s	remaining: 324ms

647:	learn: 0.0018726	total: 3.97s	remaining: 318ms
648:	learn: 0.0018728	total: 3.97s	remaining: 312ms
649:	learn: 0.0018727	total: 3.98s	remaining: 306ms
650:	learn: 0.0018727	total: 3.98s	remaining: 300ms
651:	learn: 0.0018727	total: 3.99s	remaining: 294ms
652:	learn: 0.0018669	total: 4s	remaining: 288ms
653:	learn: 0.0018653	total: 4s	remaining: 281ms
654:	learn: 0.0018597	total: 4.01s	remaining: 275ms
655:	learn: 0.0018597	total: 4.01s	remaining: 269ms
656:	learn: 0.0018595	total: 4.02s	remaining: 263ms
657:	learn: 0.0018519	total: 4.03s	remaining: 257ms
658:	learn: 0.0018458	total: 4.03s	remaining: 251ms
659:	learn: 0.0018458	total: 4.04s	remaining: 245ms
660:	learn: 0.0018456	total: 4.04s	remaining: 238ms
661:	learn: 0.0018400	total: 4.05s	remaining: 232ms
662:	learn: 0.0018400	total: 4.05s	remaining: 226ms
663:	learn: 0.0018339	total: 4.06s	remaining: 220ms
664:	learn: 0.0018338	total: 4.06s	remaining: 214ms
665:	learn: 0.0018337	total: 4.07s	remaining: 208ms
666:	learn: 0.0018337	total: 4.07s	remaining: 202ms
667:	learn: 0.0018264	total: 4.08s	remaining: 195ms
668:	learn: 0.0018264	total: 4.08s	remaining: 189ms
669:	learn: 0.0018192	total: 4.09s	remaining: 183ms
670:	learn: 0.0018192	total: 4.09s	remaining: 177ms
671:	learn: 0.0018191	total: 4.1s	remaining: 171ms
672:	learn: 0.0018190	total: 4.11s	remaining: 165ms
673:	learn: 0.0018191	total: 4.11s	remaining: 159ms
674:	learn: 0.0018190	total: 4.12s	remaining: 152ms
675:	learn: 0.0018190	total: 4.12s	remaining: 146ms
676:	learn: 0.0018191	total: 4.13s	remaining: 140ms
677:	learn: 0.0018145	total: 4.13s	remaining: 134ms
678:	learn: 0.0018087	total: 4.14s	remaining: 128ms
679:	learn: 0.0018087	total: 4.15s	remaining: 122ms
680:	learn: 0.0018088	total: 4.15s	remaining: 116ms
681:	learn: 0.0018086	total: 4.16s	remaining: 110ms
682:	learn: 0.0018085	total: 4.16s	remaining: 104ms
683:	learn: 0.0018085	total: 4.17s	remaining: 97.5ms
684:	learn: 0.0018084	total: 4.17s	remaining: 91.4ms
685:	learn: 0.0018085	total: 4.18s	remaining: 85.4ms
686:	learn: 0.0018085	total: 4.19s	remaining: 79.3ms
687:	learn: 0.0018085	total: 4.19s	remaining: 73.2ms
688:	learn: 0.0018085	total: 4.2s	remaining: 67.1ms
689:	learn: 0.0018085	total: 4.21s	remaining: 61ms
690:	learn: 0.0018084	total: 4.21s	remaining: 54.9ms
691:	learn: 0.0018084	total: 4.22s	remaining: 48.8ms
692:	learn: 0.0018084	total: 4.23s	remaining: 42.7ms
693:	learn: 0.0018085	total: 4.23s	remaining: 36.6ms
694:	learn: 0.0018083	total: 4.24s	remaining: 30.5ms

```

695:    learn: 0.0018083      total: 4.25s    remaining: 24.4ms
696:    learn: 0.0018083      total: 4.26s    remaining: 18.3ms
697:    learn: 0.0018083      total: 4.26s    remaining: 12.2ms
698:    learn: 0.0018083      total: 4.27s    remaining: 6.11ms
699:    learn: 0.0018083      total: 4.28s    remaining: 0us

```

```
[111]: <catboost.core.CatBoostClassifier at 0x2498e872990>
```

4.4.2 Validating the final model

```

[113]: # predicting and validating the results of the y_hat
y_predict_OFF = cat_boost_official.predict(X_test_processed)

# accuracies and F1_scores for the model
accuracy_OFF = accuracy_score(y_test, y_predict_OFF)
f1_score_OFF = f1_score(y_test, y_predict_OFF)

# printing the accuracies for the model prediction
print(f"Accuracy of CatBoost ML Model : {accuracy_OFF}")
print(f"F1 Score of CatBoost ML Model : {f1_score_OFF}")

```

```

Accuracy of CatBoost ML Model : 0.9904411764705883
F1 Score of CatBoost ML Model : 0.9537366548042704

```

4.4.3 Model Save

```

[116]: # saving the model

# Create directory if it doesn't exist
folder_path = "../models/final_model"
os.makedirs(folder_path, exist_ok=True)

# Save path
file_path = os.path.join(folder_path, "personal_loan_ML_catbst_model.pkl")

# Save the model
joblib.dump(model, file_path)

```

```
[116]: ['../models/final_model\\personal_loan_ML_catbst_model.pkl']
```

1.5.5 4.5 Model Training (NN)/(DL)

4.5.1 Hyperparameter Tuning for Best NN model

```

[119]: # function for NN model tuning

# function initialization
def build_model(hp):
    # model initializing

```

```

model = tf.keras.Sequential()

# Input layer
model.add(tf.keras.layers.Input(shape=(X_train_resampled.shape[1],)))

# Tunable hidden layers - up to 4 layers
num_layers = hp.Int('num_layers', min_value=1, max_value=4, step=1)

for i in range(num_layers):
    # units , activation functions, regularizers options choice LIST
    units = hp.Int(f'units_{i}', min_value=64, max_value=512, step=64)
    activation_choice = hp.Choice(f'activation_{i}', ['relu', 'swish', 'leaky_relu'])
    l2_reg = hp.Float(f'l2_reg_{i}', min_value=1e-5, max_value=1e-2, sampling='log')

    # Dense layer without activation
    model.add(tf.keras.layers.Dense(units=units,
                                     activation=None,
                                     kernel_regularizer=tf.keras.regularizers.
↳l2(l2_reg)))

    # Activation layer based on choice
    if activation_choice == 'leaky_relu':
        model.add(tf.keras.layers.LeakyReLU(alpha=0.01))
    else:
        model.add(tf.keras.layers.Activation(activation_choice))

    # Add Dropout **only once after second layer**
    if i == 1:
        if hp.Boolean('add_dropout'):
            dropout_rate = hp.Float('dropout_rate', min_value=0.2,
↳max_value=0.5, step=0.1)
            model.add(tf.keras.layers.Dropout(rate=dropout_rate))

# Output layer
model.add(tf.keras.layers.Dense(1, activation='sigmoid'))

# Compile the model with tunable optimizer
optimizer_choice = hp.Choice('optimizer', ['adam', 'nadam'])
optimizer = tf.keras.optimizers.get(optimizer_choice)

# initiating the model
model.compile(optimizer=optimizer,
              loss='binary_crossentropy',
              metrics=['accuracy'])

```

```
return model
```

```
[131]: # Tuner initializing
tuner = kt.Hyperband(
    build_model,
    objective='val_accuracy',
    max_epochs=25,
    factor=3,
    directory='../models/NN_model_trials',
    project_name='personal_loan_NN_test2',
    overwrite=True
)
```

```
[132]: print(tuner)
```

```
<keras_tuner.src.tuners.hyperband.Hyperband object at 0x0000024A2591FCD0>
```

4.5.2 Fit trainset for Tuning

```
[133]: # early stopping for preventing wasting time and wasting unwanted trials
# Define early stopping callback
early_stopping = tf.keras.callbacks.EarlyStopping(monitor='loss', patience=3,
↪restore_best_weights=True)
```

```
[134]: # Run hyperparameter search
tuner.search(X_train_resampled, y_train_resampled, epochs=35,
↪validation_data=(X_test_processed, y_test),
           verbose=1, callbacks=[early_stopping])
```

```
Trial 30 Complete [00h 00m 17s]
val_accuracy: 0.9779411554336548
```

```
Best val_accuracy So Far: 0.9823529124259949
Total elapsed time: 00h 06m 58s
```

```
[135]: # Get the best hyperparameters
best_NN_hps = tuner.get_best_hyperparameters(num_trials=1)[0]

# printing the best hyper parameters for the neural network
print(best_NN_hps.values)
```

```
{'num_layers': 4, 'units_0': 320, 'activation_0': 'swish', 'l2_reg_0':
0.00021304799057902336, 'optimizer': 'adam', 'tuner/epochs': 25,
'tuner/initial_epoch': 9, 'tuner/bracket': 2, 'tuner/round': 2, 'units_1': 64,
'activation_1': 'relu', 'l2_reg_1': 1e-05, 'add_dropout': False, 'units_2': 64,
'activation_2': 'relu', 'l2_reg_2': 1e-05, 'units_3': 64, 'activation_3':
'relu', 'l2_reg_3': 1e-05, 'tuner/trial_id': '0012', 'dropout_rate': 0.2}
```

```
[139]: for i , j in best_NN_hps.values.items():
        print(f"{i} ->\t {j}")
```

```
num_layers ->      4
units_0 ->        320
activation_0 ->    swish
l2_reg_0 ->        0.00021304799057902336
optimizer ->      adam
tuner/epochs ->    25
tuner/initial_epoch -> 9
tuner/bracket ->    2
tuner/round ->     2
units_1 ->        64
activation_1 ->    relu
l2_reg_1 ->        1e-05
add_dropout ->    False
units_2 ->        64
activation_2 ->    relu
l2_reg_2 ->        1e-05
units_3 ->        64
activation_3 ->    relu
l2_reg_3 ->        1e-05
tuner/trial_id ->      0012
dropout_rate ->    0.2
```

4.5.3 Training the best neural network model

```
[140]: # Build and train the best model
best_NN_model = tuner.hypermodel.build(best_NN_hps)
best_NN_model.fit(
    X_train_resampled,
    y_train_resampled,
    epochs=150,
    batch_size=32, # 32 batch_size for better generalization, I hope
    validation_data=(X_test_processed, y_test),
    verbose=1
)
```

```
Epoch 1/150
178/178          3s 5ms/step -
accuracy: 0.8810 - loss: 0.3242 - val_accuracy: 0.9610 - val_loss: 0.0975
Epoch 2/150
178/178          1s 3ms/step -
accuracy: 0.9683 - loss: 0.0876 - val_accuracy: 0.9669 - val_loss: 0.0904
Epoch 3/150
178/178          0s 2ms/step -
accuracy: 0.9703 - loss: 0.0890 - val_accuracy: 0.9559 - val_loss: 0.1227
Epoch 4/150
```

178/178 1s 3ms/step -
 accuracy: 0.9785 - loss: 0.0626 - val_accuracy: 0.9632 - val_loss: 0.0855
 Epoch 5/150
 178/178 1s 3ms/step -
 accuracy: 0.9782 - loss: 0.0643 - val_accuracy: 0.9610 - val_loss: 0.0949
 Epoch 6/150
 178/178 1s 3ms/step -
 accuracy: 0.9793 - loss: 0.0592 - val_accuracy: 0.9721 - val_loss: 0.0741
 Epoch 7/150
 178/178 0s 2ms/step -
 accuracy: 0.9777 - loss: 0.0576 - val_accuracy: 0.9610 - val_loss: 0.1165
 Epoch 8/150
 178/178 0s 3ms/step -
 accuracy: 0.9821 - loss: 0.0536 - val_accuracy: 0.9713 - val_loss: 0.0798
 Epoch 9/150
 178/178 0s 3ms/step -
 accuracy: 0.9845 - loss: 0.0470 - val_accuracy: 0.9794 - val_loss: 0.0717
 Epoch 10/150
 178/178 0s 2ms/step -
 accuracy: 0.9796 - loss: 0.0567 - val_accuracy: 0.9757 - val_loss: 0.0749
 Epoch 11/150
 178/178 0s 3ms/step -
 accuracy: 0.9874 - loss: 0.0406 - val_accuracy: 0.9779 - val_loss: 0.0704
 Epoch 12/150
 178/178 1s 3ms/step -
 accuracy: 0.9895 - loss: 0.0383 - val_accuracy: 0.9757 - val_loss: 0.0748
 Epoch 13/150
 178/178 1s 3ms/step -
 accuracy: 0.9887 - loss: 0.0404 - val_accuracy: 0.9684 - val_loss: 0.0861
 Epoch 14/150
 178/178 1s 3ms/step -
 accuracy: 0.9903 - loss: 0.0367 - val_accuracy: 0.9743 - val_loss: 0.0813
 Epoch 15/150
 178/178 1s 3ms/step -
 accuracy: 0.9888 - loss: 0.0346 - val_accuracy: 0.9684 - val_loss: 0.0947
 Epoch 16/150
 178/178 0s 2ms/step -
 accuracy: 0.9896 - loss: 0.0372 - val_accuracy: 0.9757 - val_loss: 0.0802
 Epoch 17/150
 178/178 0s 2ms/step -
 accuracy: 0.9866 - loss: 0.0418 - val_accuracy: 0.9684 - val_loss: 0.0956
 Epoch 18/150
 178/178 0s 3ms/step -
 accuracy: 0.9889 - loss: 0.0373 - val_accuracy: 0.9743 - val_loss: 0.0780
 Epoch 19/150
 178/178 0s 2ms/step -
 accuracy: 0.9893 - loss: 0.0350 - val_accuracy: 0.9713 - val_loss: 0.0874
 Epoch 20/150

178/178 0s 2ms/step -
 accuracy: 0.9930 - loss: 0.0309 - val_accuracy: 0.9787 - val_loss: 0.0810
 Epoch 21/150
 178/178 1s 3ms/step -
 accuracy: 0.9890 - loss: 0.0323 - val_accuracy: 0.9632 - val_loss: 0.1044
 Epoch 22/150
 178/178 0s 2ms/step -
 accuracy: 0.9900 - loss: 0.0378 - val_accuracy: 0.9779 - val_loss: 0.0775
 Epoch 23/150
 178/178 0s 3ms/step -
 accuracy: 0.9918 - loss: 0.0321 - val_accuracy: 0.9728 - val_loss: 0.0975
 Epoch 24/150
 178/178 0s 3ms/step -
 accuracy: 0.9930 - loss: 0.0291 - val_accuracy: 0.9816 - val_loss: 0.0803
 Epoch 25/150
 178/178 0s 3ms/step -
 accuracy: 0.9912 - loss: 0.0330 - val_accuracy: 0.9794 - val_loss: 0.0816
 Epoch 26/150
 178/178 0s 3ms/step -
 accuracy: 0.9917 - loss: 0.0326 - val_accuracy: 0.9794 - val_loss: 0.0839
 Epoch 27/150
 178/178 1s 3ms/step -
 accuracy: 0.9907 - loss: 0.0336 - val_accuracy: 0.9809 - val_loss: 0.0832
 Epoch 28/150
 178/178 0s 2ms/step -
 accuracy: 0.9929 - loss: 0.0274 - val_accuracy: 0.9728 - val_loss: 0.0878
 Epoch 29/150
 178/178 0s 2ms/step -
 accuracy: 0.9860 - loss: 0.0607 - val_accuracy: 0.9728 - val_loss: 0.0726
 Epoch 30/150
 178/178 1s 3ms/step -
 accuracy: 0.9887 - loss: 0.0402 - val_accuracy: 0.9772 - val_loss: 0.0802
 Epoch 31/150
 178/178 0s 3ms/step -
 accuracy: 0.9919 - loss: 0.0305 - val_accuracy: 0.9779 - val_loss: 0.0941
 Epoch 32/150
 178/178 0s 3ms/step -
 accuracy: 0.9922 - loss: 0.0308 - val_accuracy: 0.9699 - val_loss: 0.0878
 Epoch 33/150
 178/178 0s 2ms/step -
 accuracy: 0.9918 - loss: 0.0288 - val_accuracy: 0.9721 - val_loss: 0.0971
 Epoch 34/150
 178/178 0s 3ms/step -
 accuracy: 0.9920 - loss: 0.0280 - val_accuracy: 0.9787 - val_loss: 0.0782
 Epoch 35/150
 178/178 0s 2ms/step -
 accuracy: 0.9939 - loss: 0.0274 - val_accuracy: 0.9809 - val_loss: 0.0814
 Epoch 36/150

178/178 0s 2ms/step -
 accuracy: 0.9872 - loss: 0.0433 - val_accuracy: 0.9728 - val_loss: 0.0850
 Epoch 37/150
 178/178 0s 3ms/step -
 accuracy: 0.9910 - loss: 0.0277 - val_accuracy: 0.9816 - val_loss: 0.0950
 Epoch 38/150
 178/178 0s 2ms/step -
 accuracy: 0.9954 - loss: 0.0274 - val_accuracy: 0.9779 - val_loss: 0.0801
 Epoch 39/150
 178/178 0s 2ms/step -
 accuracy: 0.9924 - loss: 0.0271 - val_accuracy: 0.9772 - val_loss: 0.0968
 Epoch 40/150
 178/178 0s 2ms/step -
 accuracy: 0.9873 - loss: 0.0421 - val_accuracy: 0.9772 - val_loss: 0.1081
 Epoch 41/150
 178/178 0s 2ms/step -
 accuracy: 0.9935 - loss: 0.0305 - val_accuracy: 0.9772 - val_loss: 0.0890
 Epoch 42/150
 178/178 1s 3ms/step -
 accuracy: 0.9901 - loss: 0.0314 - val_accuracy: 0.9779 - val_loss: 0.0851
 Epoch 43/150
 178/178 1s 3ms/step -
 accuracy: 0.9938 - loss: 0.0236 - val_accuracy: 0.9779 - val_loss: 0.0893
 Epoch 44/150
 178/178 0s 2ms/step -
 accuracy: 0.9940 - loss: 0.0272 - val_accuracy: 0.9816 - val_loss: 0.0891
 Epoch 45/150
 178/178 1s 4ms/step -
 accuracy: 0.9933 - loss: 0.0263 - val_accuracy: 0.9757 - val_loss: 0.0939
 Epoch 46/150
 178/178 0s 3ms/step -
 accuracy: 0.9940 - loss: 0.0260 - val_accuracy: 0.9809 - val_loss: 0.1112
 Epoch 47/150
 178/178 0s 2ms/step -
 accuracy: 0.9950 - loss: 0.0233 - val_accuracy: 0.9831 - val_loss: 0.0819
 Epoch 48/150
 178/178 0s 3ms/step -
 accuracy: 0.9926 - loss: 0.0305 - val_accuracy: 0.9765 - val_loss: 0.1009
 Epoch 49/150
 178/178 0s 2ms/step -
 accuracy: 0.9940 - loss: 0.0259 - val_accuracy: 0.9809 - val_loss: 0.0864
 Epoch 50/150
 178/178 0s 2ms/step -
 accuracy: 0.9906 - loss: 0.0286 - val_accuracy: 0.9787 - val_loss: 0.0860
 Epoch 51/150
 178/178 0s 2ms/step -
 accuracy: 0.9939 - loss: 0.0231 - val_accuracy: 0.9787 - val_loss: 0.0971
 Epoch 52/150

178/178 0s 2ms/step -
 accuracy: 0.9933 - loss: 0.0230 - val_accuracy: 0.9816 - val_loss: 0.1033
 Epoch 53/150
 178/178 1s 3ms/step -
 accuracy: 0.9935 - loss: 0.0259 - val_accuracy: 0.9772 - val_loss: 0.0964
 Epoch 54/150
 178/178 0s 2ms/step -
 accuracy: 0.9945 - loss: 0.0259 - val_accuracy: 0.9728 - val_loss: 0.0894
 Epoch 55/150
 178/178 0s 3ms/step -
 accuracy: 0.9933 - loss: 0.0251 - val_accuracy: 0.9772 - val_loss: 0.1061
 Epoch 56/150
 178/178 1s 3ms/step -
 accuracy: 0.9945 - loss: 0.0254 - val_accuracy: 0.9809 - val_loss: 0.1067
 Epoch 57/150
 178/178 1s 3ms/step -
 accuracy: 0.9957 - loss: 0.0207 - val_accuracy: 0.9824 - val_loss: 0.1002
 Epoch 58/150
 178/178 1s 3ms/step -
 accuracy: 0.9958 - loss: 0.0212 - val_accuracy: 0.9765 - val_loss: 0.1045
 Epoch 59/150
 178/178 1s 3ms/step -
 accuracy: 0.9898 - loss: 0.0368 - val_accuracy: 0.9809 - val_loss: 0.0851
 Epoch 60/150
 178/178 1s 3ms/step -
 accuracy: 0.9963 - loss: 0.0217 - val_accuracy: 0.9794 - val_loss: 0.0971
 Epoch 61/150
 178/178 1s 3ms/step -
 accuracy: 0.9941 - loss: 0.0255 - val_accuracy: 0.9794 - val_loss: 0.1044
 Epoch 62/150
 178/178 0s 2ms/step -
 accuracy: 0.9968 - loss: 0.0206 - val_accuracy: 0.9824 - val_loss: 0.1092
 Epoch 63/150
 178/178 0s 2ms/step -
 accuracy: 0.9932 - loss: 0.0301 - val_accuracy: 0.9816 - val_loss: 0.0965
 Epoch 64/150
 178/178 0s 2ms/step -
 accuracy: 0.9962 - loss: 0.0197 - val_accuracy: 0.9728 - val_loss: 0.1074
 Epoch 65/150
 178/178 0s 2ms/step -
 accuracy: 0.9928 - loss: 0.0270 - val_accuracy: 0.9794 - val_loss: 0.1013
 Epoch 66/150
 178/178 0s 2ms/step -
 accuracy: 0.9920 - loss: 0.0282 - val_accuracy: 0.9765 - val_loss: 0.1023
 Epoch 67/150
 178/178 0s 2ms/step -
 accuracy: 0.9947 - loss: 0.0213 - val_accuracy: 0.9757 - val_loss: 0.0926
 Epoch 68/150

178/178 0s 2ms/step -
accuracy: 0.9946 - loss: 0.0223 - val_accuracy: 0.9794 - val_loss: 0.1064
Epoch 69/150

178/178 0s 3ms/step -
accuracy: 0.9911 - loss: 0.0320 - val_accuracy: 0.9809 - val_loss: 0.0933
Epoch 70/150

178/178 0s 2ms/step -
accuracy: 0.9965 - loss: 0.0226 - val_accuracy: 0.9801 - val_loss: 0.1017
Epoch 71/150

178/178 0s 2ms/step -
accuracy: 0.9948 - loss: 0.0208 - val_accuracy: 0.9765 - val_loss: 0.1380
Epoch 72/150

178/178 1s 3ms/step -
accuracy: 0.9866 - loss: 0.0615 - val_accuracy: 0.9772 - val_loss: 0.0800
Epoch 73/150

178/178 1s 3ms/step -
accuracy: 0.9934 - loss: 0.0249 - val_accuracy: 0.9765 - val_loss: 0.0901
Epoch 74/150

178/178 0s 3ms/step -
accuracy: 0.9945 - loss: 0.0258 - val_accuracy: 0.9824 - val_loss: 0.0915
Epoch 75/150

178/178 0s 3ms/step -
accuracy: 0.9960 - loss: 0.0201 - val_accuracy: 0.9787 - val_loss: 0.1023
Epoch 76/150

178/178 1s 4ms/step -
accuracy: 0.9897 - loss: 0.0321 - val_accuracy: 0.9787 - val_loss: 0.0875
Epoch 77/150

178/178 1s 3ms/step -
accuracy: 0.9945 - loss: 0.0238 - val_accuracy: 0.9735 - val_loss: 0.0911
Epoch 78/150

178/178 0s 3ms/step -
accuracy: 0.9935 - loss: 0.0240 - val_accuracy: 0.9794 - val_loss: 0.0934
Epoch 79/150

178/178 1s 3ms/step -
accuracy: 0.9959 - loss: 0.0222 - val_accuracy: 0.9757 - val_loss: 0.1001
Epoch 80/150

178/178 0s 2ms/step -
accuracy: 0.9911 - loss: 0.0326 - val_accuracy: 0.9801 - val_loss: 0.0968
Epoch 81/150

178/178 0s 2ms/step -
accuracy: 0.9972 - loss: 0.0192 - val_accuracy: 0.9750 - val_loss: 0.1036
Epoch 82/150

178/178 0s 2ms/step -
accuracy: 0.9936 - loss: 0.0242 - val_accuracy: 0.9801 - val_loss: 0.0976
Epoch 83/150

178/178 0s 2ms/step -
accuracy: 0.9940 - loss: 0.0249 - val_accuracy: 0.9787 - val_loss: 0.1062
Epoch 84/150

178/178 0s 3ms/step -
 accuracy: 0.9969 - loss: 0.0196 - val_accuracy: 0.9779 - val_loss: 0.1305
 Epoch 85/150
 178/178 0s 2ms/step -
 accuracy: 0.9958 - loss: 0.0217 - val_accuracy: 0.9787 - val_loss: 0.0982
 Epoch 86/150
 178/178 0s 2ms/step -
 accuracy: 0.9964 - loss: 0.0186 - val_accuracy: 0.9794 - val_loss: 0.0913
 Epoch 87/150
 178/178 0s 3ms/step -
 accuracy: 0.9947 - loss: 0.0211 - val_accuracy: 0.9772 - val_loss: 0.1166
 Epoch 88/150
 178/178 1s 3ms/step -
 accuracy: 0.9940 - loss: 0.0222 - val_accuracy: 0.9743 - val_loss: 0.1056
 Epoch 89/150
 178/178 0s 2ms/step -
 accuracy: 0.9898 - loss: 0.0340 - val_accuracy: 0.9809 - val_loss: 0.0954
 Epoch 90/150
 178/178 0s 2ms/step -
 accuracy: 0.9937 - loss: 0.0217 - val_accuracy: 0.9750 - val_loss: 0.1088
 Epoch 91/150
 178/178 0s 3ms/step -
 accuracy: 0.9938 - loss: 0.0235 - val_accuracy: 0.9772 - val_loss: 0.1058
 Epoch 92/150
 178/178 1s 3ms/step -
 accuracy: 0.9927 - loss: 0.0315 - val_accuracy: 0.9721 - val_loss: 0.0994
 Epoch 93/150
 178/178 0s 2ms/step -
 accuracy: 0.9941 - loss: 0.0246 - val_accuracy: 0.9787 - val_loss: 0.0895
 Epoch 94/150
 178/178 0s 3ms/step -
 accuracy: 0.9943 - loss: 0.0230 - val_accuracy: 0.9801 - val_loss: 0.0956
 Epoch 95/150
 178/178 0s 2ms/step -
 accuracy: 0.9945 - loss: 0.0221 - val_accuracy: 0.9787 - val_loss: 0.1098
 Epoch 96/150
 178/178 0s 2ms/step -
 accuracy: 0.9960 - loss: 0.0189 - val_accuracy: 0.9794 - val_loss: 0.1045
 Epoch 97/150
 178/178 0s 2ms/step -
 accuracy: 0.9904 - loss: 0.0347 - val_accuracy: 0.9765 - val_loss: 0.1198
 Epoch 98/150
 178/178 0s 2ms/step -
 accuracy: 0.9935 - loss: 0.0246 - val_accuracy: 0.9772 - val_loss: 0.1224
 Epoch 99/150
 178/178 0s 3ms/step -
 accuracy: 0.9970 - loss: 0.0192 - val_accuracy: 0.9765 - val_loss: 0.1060
 Epoch 100/150

178/178 1s 4ms/step -
 accuracy: 0.9941 - loss: 0.0232 - val_accuracy: 0.9794 - val_loss: 0.1215
 Epoch 101/150
 178/178 1s 4ms/step -
 accuracy: 0.9952 - loss: 0.0253 - val_accuracy: 0.9794 - val_loss: 0.1016
 Epoch 102/150
 178/178 1s 3ms/step -
 accuracy: 0.9973 - loss: 0.0177 - val_accuracy: 0.9787 - val_loss: 0.1250
 Epoch 103/150
 178/178 1s 3ms/step -
 accuracy: 0.9943 - loss: 0.0237 - val_accuracy: 0.9801 - val_loss: 0.1237
 Epoch 104/150
 178/178 0s 2ms/step -
 accuracy: 0.9958 - loss: 0.0195 - val_accuracy: 0.9787 - val_loss: 0.1202
 Epoch 105/150
 178/178 0s 2ms/step -
 accuracy: 0.9958 - loss: 0.0185 - val_accuracy: 0.9787 - val_loss: 0.1186
 Epoch 106/150
 178/178 0s 3ms/step -
 accuracy: 0.9967 - loss: 0.0170 - val_accuracy: 0.9743 - val_loss: 0.1395
 Epoch 107/150
 178/178 1s 3ms/step -
 accuracy: 0.9925 - loss: 0.0279 - val_accuracy: 0.9765 - val_loss: 0.1027
 Epoch 108/150
 178/178 0s 2ms/step -
 accuracy: 0.9948 - loss: 0.0220 - val_accuracy: 0.9787 - val_loss: 0.1107
 Epoch 109/150
 178/178 0s 2ms/step -
 accuracy: 0.9941 - loss: 0.0227 - val_accuracy: 0.9772 - val_loss: 0.1079
 Epoch 110/150
 178/178 0s 2ms/step -
 accuracy: 0.9956 - loss: 0.0176 - val_accuracy: 0.9801 - val_loss: 0.1159
 Epoch 111/150
 178/178 0s 2ms/step -
 accuracy: 0.9953 - loss: 0.0195 - val_accuracy: 0.9735 - val_loss: 0.1124
 Epoch 112/150
 178/178 0s 2ms/step -
 accuracy: 0.9951 - loss: 0.0227 - val_accuracy: 0.9787 - val_loss: 0.0957
 Epoch 113/150
 178/178 0s 2ms/step -
 accuracy: 0.9965 - loss: 0.0207 - val_accuracy: 0.9772 - val_loss: 0.1002
 Epoch 114/150
 178/178 0s 2ms/step -
 accuracy: 0.9956 - loss: 0.0187 - val_accuracy: 0.9757 - val_loss: 0.1245
 Epoch 115/150
 178/178 0s 3ms/step -
 accuracy: 0.9955 - loss: 0.0190 - val_accuracy: 0.9765 - val_loss: 0.1184
 Epoch 116/150

178/178 0s 3ms/step -
 accuracy: 0.9938 - loss: 0.0250 - val_accuracy: 0.9824 - val_loss: 0.1020
 Epoch 117/150
 178/178 1s 3ms/step -
 accuracy: 0.9962 - loss: 0.0184 - val_accuracy: 0.9765 - val_loss: 0.1122
 Epoch 118/150
 178/178 1s 3ms/step -
 accuracy: 0.9933 - loss: 0.0226 - val_accuracy: 0.9787 - val_loss: 0.1059
 Epoch 119/150
 178/178 0s 3ms/step -
 accuracy: 0.9959 - loss: 0.0219 - val_accuracy: 0.9809 - val_loss: 0.1068
 Epoch 120/150
 178/178 0s 2ms/step -
 accuracy: 0.9935 - loss: 0.0243 - val_accuracy: 0.9809 - val_loss: 0.1021
 Epoch 121/150
 178/178 1s 3ms/step -
 accuracy: 0.9972 - loss: 0.0179 - val_accuracy: 0.9809 - val_loss: 0.1120
 Epoch 122/150
 178/178 0s 2ms/step -
 accuracy: 0.9972 - loss: 0.0181 - val_accuracy: 0.9779 - val_loss: 0.1268
 Epoch 123/150
 178/178 0s 2ms/step -
 accuracy: 0.9959 - loss: 0.0197 - val_accuracy: 0.9765 - val_loss: 0.1098
 Epoch 124/150
 178/178 0s 3ms/step -
 accuracy: 0.9967 - loss: 0.0172 - val_accuracy: 0.9801 - val_loss: 0.1089
 Epoch 125/150
 178/178 0s 2ms/step -
 accuracy: 0.9980 - loss: 0.0161 - val_accuracy: 0.9779 - val_loss: 0.1198
 Epoch 126/150
 178/178 0s 2ms/step -
 accuracy: 0.9952 - loss: 0.0224 - val_accuracy: 0.9779 - val_loss: 0.1063
 Epoch 127/150
 178/178 1s 3ms/step -
 accuracy: 0.9960 - loss: 0.0190 - val_accuracy: 0.9801 - val_loss: 0.0971
 Epoch 128/150
 178/178 0s 3ms/step -
 accuracy: 0.9962 - loss: 0.0187 - val_accuracy: 0.9750 - val_loss: 0.1199
 Epoch 129/150
 178/178 0s 3ms/step -
 accuracy: 0.9955 - loss: 0.0222 - val_accuracy: 0.9809 - val_loss: 0.1109
 Epoch 130/150
 178/178 0s 3ms/step -
 accuracy: 0.9966 - loss: 0.0192 - val_accuracy: 0.9794 - val_loss: 0.1125
 Epoch 131/150
 178/178 1s 3ms/step -
 accuracy: 0.9949 - loss: 0.0201 - val_accuracy: 0.9728 - val_loss: 0.1304
 Epoch 132/150

178/178 0s 3ms/step -
 accuracy: 0.9932 - loss: 0.0269 - val_accuracy: 0.9779 - val_loss: 0.1056
 Epoch 133/150
 178/178 1s 3ms/step -
 accuracy: 0.9926 - loss: 0.0290 - val_accuracy: 0.9801 - val_loss: 0.1128
 Epoch 134/150
 178/178 0s 3ms/step -
 accuracy: 0.9946 - loss: 0.0210 - val_accuracy: 0.9809 - val_loss: 0.0989
 Epoch 135/150
 178/178 0s 2ms/step -
 accuracy: 0.9950 - loss: 0.0201 - val_accuracy: 0.9765 - val_loss: 0.1013
 Epoch 136/150
 178/178 0s 3ms/step -
 accuracy: 0.9942 - loss: 0.0231 - val_accuracy: 0.9765 - val_loss: 0.1273
 Epoch 137/150
 178/178 0s 3ms/step -
 accuracy: 0.9956 - loss: 0.0202 - val_accuracy: 0.9787 - val_loss: 0.1117
 Epoch 138/150
 178/178 1s 4ms/step -
 accuracy: 0.9974 - loss: 0.0164 - val_accuracy: 0.9794 - val_loss: 0.1229
 Epoch 139/150
 178/178 0s 3ms/step -
 accuracy: 0.9949 - loss: 0.0195 - val_accuracy: 0.9809 - val_loss: 0.1141
 Epoch 140/150
 178/178 0s 3ms/step -
 accuracy: 0.9966 - loss: 0.0175 - val_accuracy: 0.9779 - val_loss: 0.1219
 Epoch 141/150
 178/178 0s 3ms/step -
 accuracy: 0.9955 - loss: 0.0228 - val_accuracy: 0.9801 - val_loss: 0.1005
 Epoch 142/150
 178/178 0s 2ms/step -
 accuracy: 0.9955 - loss: 0.0202 - val_accuracy: 0.9779 - val_loss: 0.1050
 Epoch 143/150
 178/178 0s 2ms/step -
 accuracy: 0.9962 - loss: 0.0184 - val_accuracy: 0.9794 - val_loss: 0.1164
 Epoch 144/150
 178/178 0s 2ms/step -
 accuracy: 0.9955 - loss: 0.0176 - val_accuracy: 0.9801 - val_loss: 0.1281
 Epoch 145/150
 178/178 0s 2ms/step -
 accuracy: 0.9963 - loss: 0.0183 - val_accuracy: 0.9816 - val_loss: 0.1048
 Epoch 146/150
 178/178 0s 3ms/step -
 accuracy: 0.9961 - loss: 0.0185 - val_accuracy: 0.9816 - val_loss: 0.1274
 Epoch 147/150
 178/178 0s 2ms/step -
 accuracy: 0.9957 - loss: 0.0203 - val_accuracy: 0.9772 - val_loss: 0.1105
 Epoch 148/150

```

178/178          1s 3ms/step -
accuracy: 0.9945 - loss: 0.0210 - val_accuracy: 0.9743 - val_loss: 0.0966
Epoch 149/150
178/178          1s 3ms/step -
accuracy: 0.9932 - loss: 0.0290 - val_accuracy: 0.9794 - val_loss: 0.0945
Epoch 150/150
178/178          1s 3ms/step -
accuracy: 0.9966 - loss: 0.0160 - val_accuracy: 0.9735 - val_loss: 0.1047

```

[140]: <keras.src.callbacks.history.History at 0x24a43fd6210>

```

[142]: # saving the final neural network model
best_NN_model.save("../models/final_model/personal_loan_NN_model1.keras")

```

1.6 Task 5 : Which Model?

```

[ ]: # Give your reasoning on which is the best model in this case and why it
     ↪ performs better

```

While Comparing both the **Machine Learning** and **Deep Learning** Models:

- it turns out that Deep Learning Model works better with even better accuracies of
- above **97%** whereas
- the ML algorithm that finally got chosen was Cat Boosting Classifier
- with **95.37%** accuracy
- overall the neural network model performed better with Tensorflow